SC2006 Visual examination reference

A4 chapter guide, four older papers grouped by question type and four recent worked papers. Revised 4 November 2026.

Use: read the ELI5 introduction, inspect the diagram, then use the rules and traps to build a case-specific answer. Tutorial material already covered in lecture chapters appears once.

Historical format: all eight supplied papers allow two hours for four compulsory 25-mark questions.

Question	What to find first	Chapter / page
Q1	Actors, goals, use-case relationships and behaviour; one paper adds stakeholders/requirements.	Ch. 2–3, pp. 2–5
Q2	Conceptual classes, state/activity/dialog diagrams, development process and Scrum reasoning.	Ch. 4–5, 7–8; pp. 7, 9, 13, 15
Q3	Architecture tied to the case, NFRs, design-pattern roles, trade-offs and responsibility/LLM questions.	Ch. 9–11, 16; pp. 18, 20, 22, 31
Q4	ECs, full boundary triplets, one-invalid defensive rows and expected outcomes; testing/maintenance extensions.	Ch. 12–13, 16; pp. 23, 26, 31

Print: A4, actual size, one page per sheet side. All pages are portrait. Colour labels remain readable in greyscale.

Status: worked answers are unofficial; assumptions are labelled. Use original PDFs for exact questions and proper UML notation in examination drawings. Historical formats do not guarantee the next paper.

Contents

Chapter 1: Software engineering and the SDLC . . . 2
Chapter 2: Requirements elicitation . . . 2
Chapter 3: Use-case modelling . . . 5
Chapter 4: Conceptual models and class diagrams . . . 7
Chapter 5: State machines and activity diagrams . . . 9
Chapter 6: Software Requirements Specification . . . 11
Chapter 7: Software process models . . . 13
Chapter 8: Agile, XP and Scrum . . . 15
Chapter 9: Architecture and system design . . . 18
Chapter 10: Object design and design patterns . . . 20
Chapter 11: Model-View-Controller . . . 22
Chapter 12: Testing fundamentals and basis paths . . . 23
Chapter 13: Equivalence classes and boundary values . . . 26
Chapter 14: GES: difficult constraints . . . 29
Chapter 15: Hard tutorial questions . . . 30
Chapter 16: AI engineering and agentic systems . . . 31
Chapter 17: Examination answer checklist . . . 33
Source index . . . 33
Older papers: cases and question map . . . 34
Older Q1: Use-case models and descriptions . . . 34
Older Q2: Conceptual models, behaviour and process . . . 38
Older Q3: Architecture, NFRs and patterns . . . 42
Older Q4: Input testing and applied testing/maintenance . . . 45
Paper 5: AY2024/25 Semester 1 — Collaborative task management . . . 49
Paper 6: AY2024/25 Semester 2 — Disaster Warning System (DWS) . . . 53
Paper 7: AY2025/26 Semester 1 — Smart Home Control System . . . 57
Paper 8: AY2025/26 Semester 2 — Restaurant Recommendation System . . . 61

Chapter 1: Software engineering and the SDLC

ELI5

Building software is like building a playground. First ask who needs it and what they need; then draw plans, choose the structure, build it, check it and keep repairing it. The drawings and checklists are work products. A process explains how the team revisits those activities, not merely a row of boxes.

Answer check: Trace the case from requirement to model, design and test. Process models arrange activities; architectures arrange components; patterns solve recurring design problems. Test design starts before execution.

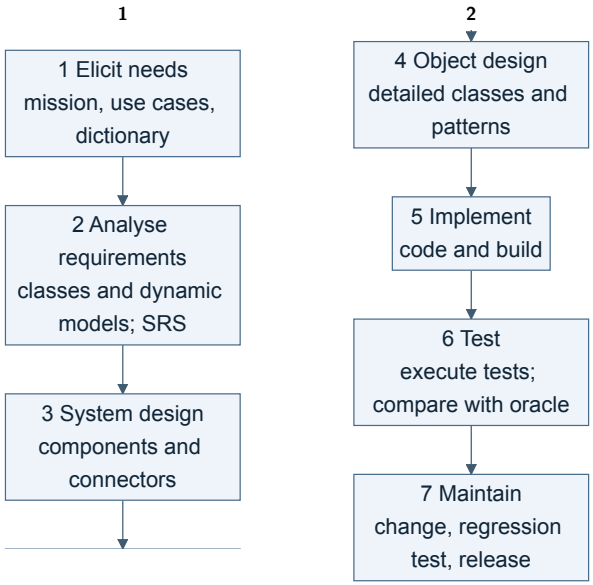


Figure 1: 1. The spine of the course: SDLC → artefacts — follow numbered panels; crossing arrows continue.

Milestones and work products (L1 work-product table)

Phase / Work products / milestones

Requirements specification: SRS, prototype, requirements review

Design: Software Design Document, Interface Design Document, models, **test cases**; preliminary & final design reviews

Implementation: Source code, system build, technical documentation, user manual

Testing: Test execution, test report, user acceptance test

Maintenance: Change request, bug report, patch release, upgrade

Definition to memorise

> **Software engineering** is *the production of maintainable, fault-free software that meets the user's requirements and is delivered on time and within budget.*

The four process activities (also the exam-safe summary)

Specification (what it should do) → **Design & implementation** (organisation + code) → **Validation** (does it do what the customer wants) → **Evolution** (change in response to need).

> **Trap:** process models arrange SDLC activities; architectures arrange components/connectors; design patterns solve recurring object-collaboration problems.

Requirement → analysis → design traceability chain

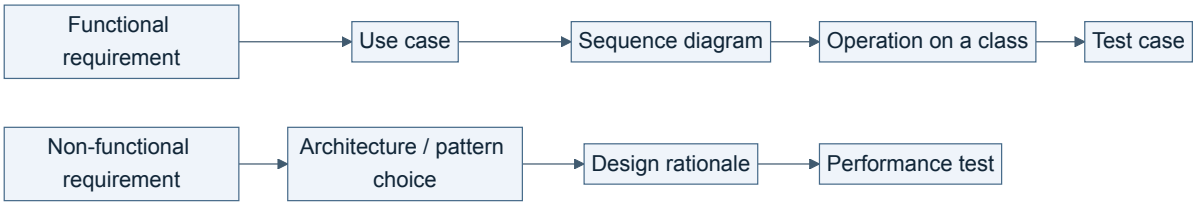


Figure 2: Requirement → analysis → design traceability chain

Chapter 2: Requirements elicitation

ELI5

A customer saying 'make a great bicycle' is not yet a testable order. Ask who rides it, where, what it must do and how you will know it is good enough. 'Has brakes' is a capability; 'stops within an agreed distance under stated conditions' is a quality target.

Answer check: Identify stakeholders, atomise requirements, attach measurable acceptance conditions and mark missing policy TBD. Security-related actions can be FRs; a digit alone does not make an NFR.

Visual: classify a requirement

Read what the statement asserts, then check whether you can verify it.

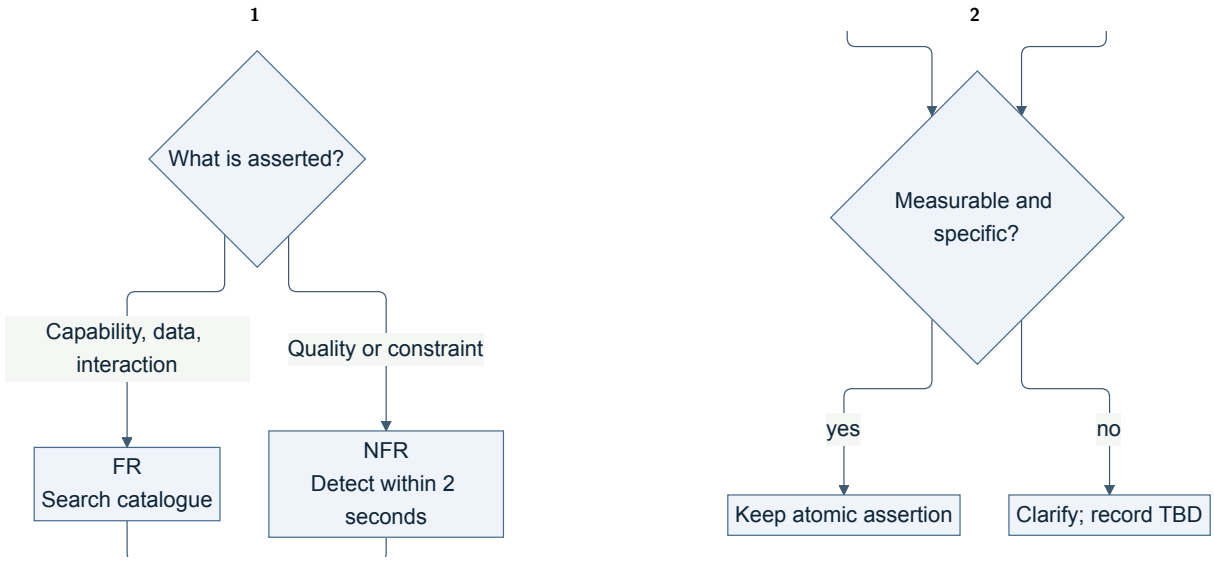


Figure 3: classify a requirement — follow numbered panels; crossing arrows continue.

2.1 Project mission statement

Two or three sentences that define:

1. **The problem** — what will be done (scope and limits).
2. **The stakeholders** — developers and users.
3. **The outcomes and benefits** — plus a completion criterion.

A mission statement may appear in the SRS product description (Writing SRS pp. 7–8).

2.2 Elicitation techniques (six — memorise the list)

Technique / ***Use when***

Interview: You need depth from one stakeholder; open questions

Observation: Users cannot articulate what they do ("watch the work")

Workshop: Many stakeholders must agree; resolve conflicts fast

Legacy product study: Replacing an existing system; inherit its quirks

Competitive product study: No access to users; derive features from rivals

Prototype: Requirements are visual/interaction-heavy; "a picture is worth a thousand words"

Two goals behind all six: **learn the problem domain** and **study user tasks**.

Trap: prototypes can elicit needs or validate them; state which purpose applies.

2.3 Functional vs non-functional requirements

	Functional	Non-functional
Definition	Interactions between the system and its environment; maps program inputs to program outputs	Properties the system must have, not directly related to functional behaviour
Answers	*What must the system do?*	*How well must it do it?*
Sub-kinds	System functionality · Information to be processed · Interface with other systems	Usability · Reliability · Performance · Supportability (also security, availability, scalability, maintainability, portability)
Example	"The library member must be able to search the library catalog."	"When a book is placed on the checkout pad, the system must detect it within 2 seconds."
Example	"The system must display the temperature in degrees centigrade in the range −10 °C to +130 °C to one decimal place."	"Help messages must be displayed in the local language according to the user's locale."
Example	"The system must be able to control up to six robot arms simultaneously."	"After a system reboot, the full system functionality must be restored within 5 minutes."

Classification drill (high-yield). Decide which of these are FR vs NFR:

Statement	Answer	Why
The system must display the current time in 24-hour format	**FR**	Information to be processed
The database must be replaceable with any commercial product supporting standard SQL	**NFR** (supportability)	Portability/replaceability property
The system must verify membership with the University Account System before a loan	**FR**	Interface with another system
80% of first-time users must enter a simple search query within 2 minutes	**NFR** (usability)	Measurable quality target

Continued on next page

Statement	Answer	Why
The system must log the event when the email server is unreachable	**FR**	Functionality (though see Section 3 — may be an exception flow)
The system must be user-friendly	**NFR, but not verifiable as written**	Usability intent; needs a measurable criterion

> **Trap:** the "control up to six robot arms" example is **functional** (number of controllable devices is a capability), while "response within 2 seconds" is **performance/NFR**. Do not classify by the presence of a number — classify by **what is being asserted**.

2.4 Atomic, verifiable, traceable

Atomic — one testable assertion per numbered statement. Nest sub-requirements.

> *Original:* "When a computer is added, the tracking system requires the user to specify its type and allow the user to provide a description. Both these fields must be text of length > 0 and < 512."

```
1.1   When a computer is added to the tracking
↪  system, the user must enter the computer
↪  type.
1.1.1 The computer type data must be text of at
↪  least one character and less than 512
↪  characters.
1.2   When a computer is added to the tracking
↪  system, the user must enter a description.
1.2.1 The description of the computer must be
↪  text of at least one character and less
↪  than 512 characters.
```

Verifiable — sufficiently specific to be testable, with an oracle.

NO Not verifiable / YES Verifiable

NO Not verifiable: The user interface must be user-friendly.

YES Verifiable: 80% of first-time users must be able to enter a simple search query within 2 minutes of starting to use the system.

NO Not verifiable: The system must control more than one drill press.

YES Verifiable: The system must control up to seven drill presses concurrently.

NO Not verifiable: The company must not hire too-young or too-old people.

YES Verifiable: 0–15 and 55–99: do not hire; 16–17: part-time; 18–54: full-time.

Traceability — *the ability to track requirements from their expression in the SRS to their realisation in design, documentation, source code, and their verification in reviews and tests; and to track dependencies among requirements, system functions and system components.*

> **Trap:** traceability is **not** just "numbering requirements". Numbering is a prerequisite (modifiability); traceability is the **link** forwards (to design/code/test) and backwards (to source).

2.5 Data dictionary (problem-domain glossary)

The table below is the lecture's **illustrative university domain**, not a statement of NTU's academic calendar.

- Ensures *consistent, unambiguous terminology* that all stakeholders agree on.
- Defines **problem-domain** terms (Programme of Study, Course, Lecture, Tutorial, Class, Laboratory, Seminar) — **not** implementation terms (table, foreign key, thread).

Term / Definition

Programme of Study: A university programme a student enrols into. Three levels: undergraduate, master, PhD.

Semester: Four per academic year. Semesters 1 and 4 have 13 teaching weeks; semesters 2 and 3 have 5.

Course: A basic unit of teaching; compulsory or elective; must include lectures and tutorials; some have labs.

Lecture: Traditional class in lecture theatres; video recorded and published in the LMS.

Tutorial: Interactive, small-size class in technology-enhanced rooms; **not** video recorded.

> **Trap:** the data dictionary is part of the SRS (Section 5 of the SRS contents). It is not a database schema and not a class diagram.

2.6 Requirements validation

Technique / What it checks

Review (walkthrough, inspection, critical review) with a checklist: completeness, consistency, unambiguity, correctness

Prototype: Stakeholders: the system specified meets their needs/ desires (**correct**) and requirements are **prioritised**. Development team: requirements and underlying assumptions are **properly understood**.

> **Trap:** review $\neq$ testing. Review is **static** (document inspection); testing is **dynamic** (execution). Both appear as "validation" in different lectures — read the context.

2.7 The iterative elicitation loop (memorise the steps)

Step 1 — Project team alone. Review the client's specification (and legacy/competitive products) to:

1. **Atomise** requirements.
2. Identify **actors**.
3. Identify main **use cases** (functional requirements).
4. Identify **non-functional** requirements.
5. Initiate the **data dictionary**.
6. Identify **uncertainties** $\rightarrow$ **TBD** (To Be Determined). *Never guess user requirements.*

Step 2 — Project team meets client (interview / observation / workshop). Agenda agreed up-front:

1. Present and walk through the initial requirements + data dictionary → get agreement.
2. Clarify the TBD issues.
3. Discuss new requirements the client raises.

Repeat 1–2 **until** all uncertainties and ambiguities in functional, non-functional requirements and data-dictionary terms are agreed with the client. A prototype may be produced to show intended functionality. **Minimise the number of meetings by making meetings productive.**

> **Trap:** "Do not guess the user requirements" is an explicit lecture instruction — the correct answer to a missing requirement is **TBD + ask the client**, not "assume a sensible default".

Chapter 3: Use-case modelling

ELI5

A use case is a promise about a user's goal, like 'borrow a book'. An actor is a role standing outside the system. The diagram is the menu of goals; the description is the recipe showing what the user and system do, including what happens when something goes wrong.

Answer check: Start with the boundary and external roles; name goals, then describe actor/system steps and alternatives. Include is required reuse; extend is conditional extra behaviour. Neither means "happens next. Authentication may be a precondition without a fresh login each time. Keep denial/conflict paths inside the behaviour when they must be explained.

Visual: decide between include and extend

First identify a meaningful unit of behaviour; do not turn every sentence into a use case.

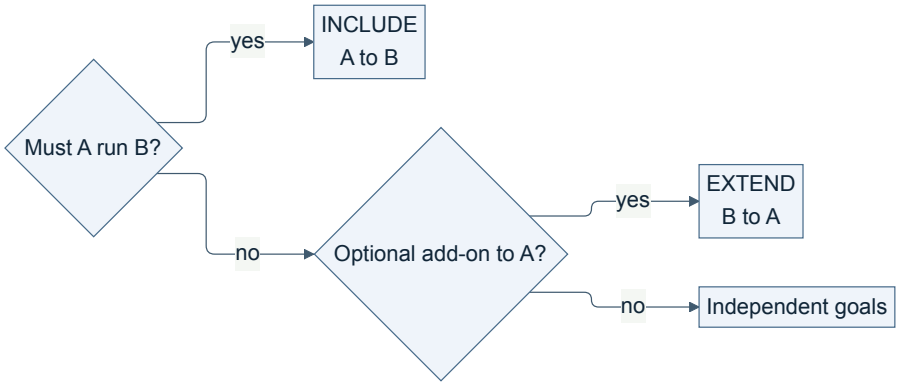


Figure 4: decide between include and extend

3.1 UML in one table

UML = *a graphical notation with textual annotation for specifying, documenting and communicating structure, functionality and dynamic behaviour of complex software systems.* Booch + Jacobson + Rumbaugh; the lecture gives UML 1.1 adoption in 1997 and refers to UML 2.5 (2015). These are historical references in the supplied slides, not a claim about the current UML release.

Category / Diagrams

Structure (static, "irrespective of time"): Class, Object, Package, Component, Composite Structure, Deployment (L3 list)

Behaviour (dynamic): **Use Case**, **Activity**, **State Machine**, **Sequence**, **Communication**, Interaction Overview, Timing

Course modelling focus: **use case, class, sequence, state machine, activity** (communication/object appear in the SRS figure list).

Trap: UML is **not** a programming language and **not** a method. "Only use the diagrams that are useful and necessary to define the system" — select diagrams for the modelling need rather than drawing every diagram type.

3.2 Use case model = static + dynamic

> **Use case model = Use case diagram (static) + Use case descriptions (dynamic).**

The diagram identifies relationships; the description specifies the behaviour. Supply both when requested.

3.3 Diagram elements

Element	Notation	Meaning
Actor	Stick figure, labelled with role (not a person's name)	An **external** entity that interacts with the system
Use case	Ellipse, labelled with a verb phrase	A unit of functionality performed by the system yielding a result/value for an actor
Association	Solid line actor↔use case	Actor participates in that use case

Continued on next page

Element	Notation	Meaning
System boundary	Rectangle around use cases	Separates system from environment
Generalization	Solid line, hollow triangle pointing to the parent	Actor or use-case specialisation
«include»	Dashed arrow from base → included	Base **always** performs the included behaviour
«extend»	Dashed arrow from extending → base	Extending behaviour is **optional/conditional**

3.4 «include» vs «extend» — decomposition, reuse and optional behaviour

	«include»	«extend»
Trigger	Always, as part of the base	Under an explicit condition / at an extension point
Purpose	Functional decomposition, or **reuse** of common behaviour	Add **optional** functionality
Base alone is...	**Incomplete** without the included UC	**Complete by itself**
Arrow direction	base —-► included	extending —-► base
Course example	Manage Incident **includes** Create / Handle / Close Incident; Open Incident and Allocate Resources **include** View Map	Report Emergency ◄—- **extends** —- Help

Trap #1 (arrow direction). Students reverse «extend» arrows constantly. Mnemonic: **"include points to the helper; extend points to the host."**

Trap: conditional behaviour can remain an alternative/exception flow. Use extend only for a separate optional unit; required reused behaviour uses include.

3.5 The use case description — mandatory fields

1. **Participating actors** (with the **initiating actor** identified).
   - A use case can have more than one participating actor.
   - The **initiating actor triggers** the use case.
   - If triggered by a **system event or another use case**, the initiating actor is **NULL**.
2. **Entry conditions (pre-condition)** — system state that must be satisfied to execute.
3. **Exit conditions (post-condition)** — system state that must be reached afterwards.
4. **Flow of events** (a.k.a. normal / main success flow) — alternating actor and system steps: step 1 = how the actor triggers; step 2 = how the system responds; ...
5. **Alternative flows** — variations or errors that **return to the normal flow** (e.g. invalid card/PIN → back to step 2; insufficient funds → back to step 6).
6. **Exceptions** — exceptional situations that **cause the use case to fail** (e.g. three invalid PIN attempts → card suspended; ATM out of funds → refuses card).

Template also captures: Use Case ID, Name, Created/ Updated by, dates, Actor, Description, Priority, Frequency of Use, Includes, Special Requirements, Assumptions, Notes and Issues.

3.6 Worked example — "Withdraw Money using ATM"

Participating actors: Bank Customer (**initiating**), Bank (secondary/external system). **Entry conditions:** (1) customer has an account with the bank **and** (2) has an ATM card and PIN. **Exit conditions:** (1) customer receives the requested cash **or** (2) customer receives an explanation from the ATM why cash could not be dispensed.

Normal flow (note the A/S alternation):

#	Actor step	System step
1	Inputs the card into the ATM	
2		Requests input of a six-digit PIN
3	Enters a PIN	
4		Verifies card and PIN with account info in the Bank
5		If verified, requests the amount to withdraw
6	Enters an amount	
7		Verifies the amount is available in the account
8		If sufficient funds, outputs money, card and receipt
9	Removes card, receipt and money	

Alternative flows

- **AF-S5 (invalid card and PIN):** 1. ATM displays "Invalid card and PIN. Please try again!" for 2 s. 2. ATM **returns to step 2**.
- **AF-S8 (insufficient funds):** 1. ATM displays "Insufficient funds... enter a smaller amount!" for 2 s. 2. ATM **returns to step 6**.

Exceptions

- **EX1 (invalid PIN three times):** 1. Displays "Card is suspended! Please call customer service (1633)..." for 2 s. 2. Outputs the card. → *Use case fails.*
- **EX2 (ATM has insufficient funds):** 1. Displays "This ATM is off-service due to insufficient funds!" at the start screen. 2. Does not accept the card.

Trap: the AF/EX distinction is graded. **AF = recoverable, returns to a step. EX = terminal, the goal is not achieved.** "Insufficient funds in *account*" is recoverable (AF); "insufficient funds in *ATM*" is not (EX). Read carefully which one the question describes.

3.7 Deriving use cases from functional requirements

Use the Library Management System drill (L3):

#	Functional requirement	Actor(s)	Use case	Association
1	Member searches for materials	Member	Search Material	—
2	Member loans materials	Member	Loan Material	—
3	System verifies membership with University Account System **before** loan	Member, University Account System	Verify Member	«include» from Loan Material
4	Member returns materials via drop-off kiosk	Member, Library Drop-off Kiosk	Return Material	—
5	System sends overdue notice via CITS Email Server	CITS Email System	Send Overdue Notice	—
6	**When** network to CITS fails, log the event	— (system event)	Log Network Issues	«extend» on Send Overdue Notice

Method (do this in the exam, in order):

1. **Identify actors** — scan for *people/roles* and *external systems*; underline nouns.
2. **Identify use cases** — scan for *verb phrases describing a goal* ("search", "loan", "return", "send notice"). One goal = one use case.
3. **Identify associations** — draw actor↔UC lines only where the actor participates.
4. **Add include/extend/generalization** — only where justified; annotate with one sentence of reasoning.
5. **Write the description** for the use cases the question names.

Lecture comparison. L3, pp. 20–22, shows three candidate library diagrams, including different choices of actor and authentication behaviour. Do not treat every candidate as an approved solution. Trace each element to the stated requirements: membership verification is required before a loan; Login is not stated for the library example. L4, pp. 12–16, separately uses Login in the course registration example. Explain the system boundary and reuse choice.

Trap — actors vs system internals. "Database", "Sensor" (if part of the system's own hardware), "UI" are **not** actors. But an **external** University Account System, CITS Email Server, Billing System, Library Drop-off Kiosk **are** actors.

Trap — actor generalization. If Student and Professor share Login, model an abstract **User** actor with generalization arrows from Student/Professor to User, then associate Login with User. Do not draw duplicate Logins.

Chapter 4: Conceptual models and class diagrams

ELI5

Think of a cinema: a TicketScreen talks to the customer, a BookingController organises the booking and a Ticket stores a booking fact. A class is a kind of thing; an object is one particular instance. A link's numbers tell you how many objects can be connected, not which class owns the drawing.

Answer check: Separate boundaries, coordinating controls and persistent entities. Conceptual questions may prohibit attributes/operations; add them only for detailed design. State invariants that multiplicities cannot express, such as one active hold despite many historical holds.

Visual: assign each class a responsibility

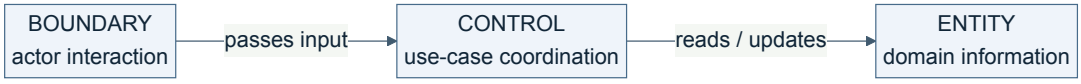


Figure 5: assign each class a responsibility

The boundary/control/entity arrows show a common collaboration, not every allowed relationship; derive responsibilities from the case.

4.1 Elicitation vs analysis (L5 distinction)

	Requirements elicitation	Requirements analysis
Purpose	Find out **what customers want**	Produce a system model that is **correct, complete, consistent, unambiguous** based on the use cases
Output	Description of the system in terms of **actors and use cases**	**Conceptual model** (system structure) + **dynamic model** (system behaviour)
Perspective	Customer's	Software engineer's
Artefacts	SRS, use case diagram + descriptions, data dictionary, prototype	Class diagram (boundary/control/entity), sequence diagrams, state machines, activity diagrams

Formalising requirements with analysis models: **clarifies** structure and dynamics, **validates** requirements, **underpins** solution modelling.

4.2 Conceptual model vocabulary

Element	Definition	Example
Object	Anything that has a **state** and exhibits **behaviour**	Bus, Dog, Company; *LoginForm, UserVerifier, UserList*
Operation	Procedure through which objects communicate	Bus: Stop, Start; Company: Sell, Quote; UserVerifier: authenticate
Attribute	Variable holding state information	Bus: Seats, Colour; Company: Name, Employees

Documented in a **class diagram**: Object → Class; Roles of the object → Attributes and methods of the class.

4.3 Class notation



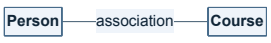
Figure 6: 4.3 Class notation

4.4 Relationships — notation and reading

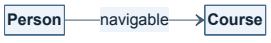
What the relationships look like

The following are concrete class-diagram sketches. Read the symbol **at the end of the line** carefully: that is where the relationship is attached.

Association: structural link; navigation unspecified



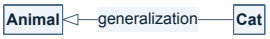
Navigability: reach the arrow-end class



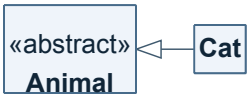
Non-navigable: Person cannot reach Course



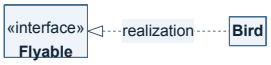
Generalization: triangle at parent; no multiplicity



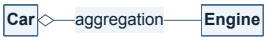
Abstract superclass: cannot instantiate



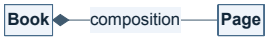
Realization: implements an interface



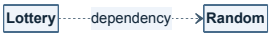
Aggregation: parts can outlive the whole



Composition: exclusive ownership and lifecycle



Dependency: temporary use



Multiplicity: near-end class is counted



Mermaid cannot draw UML's non-navigable x; place it at the Course end in an examination drawing.

Figure 7: Relationship notation atlas: read the symbol at each end.

Multiplicity cheatsheet

Notation / Meaning

`1`: Exactly one

`0..1`: Zero or one (optional)

`*` **or** `0..*`: Zero or more

`1..*`: One or more

`3`: Exactly three

`1..6`: Between one and six

How to read a multiplicity correctly: the number is written **at the end near the class it counts**. `Person "1" -- "1" Head` means one Head per Person and one Person per Head. `Polygon "1" -- "*" Point` means zero or more Points per Polygon and one Polygon per Point.

Notation: abstract class names are italicised or marked `{abstract}`; interface realisation uses a dashed line and hollow triangle.

Traps and things to check

Choose one relationship type per line. Generalization has a hollow triangle at the parent, no multiplicities or role names. Association multiplicities belong at both counted ends. A plain line leaves navigability unspecified; an x prohibits reaching its target end. Composition adds exclusive composite ownership and lifecycle responsibility; a part may be detached before deletion. Use hollow-triangle generalization despite L5's inconsistent filled-arrow example.

4.5 Stereotypes: entity / boundary / control

Stereotype	Role	Typical examples
«entity»	Information tracked by the system	`Account`, `Silo`, `Shipment`
«boundary»	Interaction between an actor and the system	`LoginForm`, `EntryGate`, `FingerprintReader`
«control»	Logic that realises a use case (coordination)	`FacilityAccessSystem`, `SheetManager`, `RegistrationController`

Worked example — **EnterSecureFacility**:

1. User presses finger on FingerprintReader. 2. FingerprintReader scans and sends image to FacilityAccessSystem. 3. FacilityAccessSystem validates fingerprint and unlocks EntryGate.
2. EntryGate signals the system it has opened. 5. FacilityAccessSystem locks EntryGate, logs the entry event and increments occupant count.

Classes: `Employee` («entity»), `Fingerprint` («entity»), `FingerprintReader` («boundary»), `EntryGate` («boundary»), `FacilityAccessSystem` («control»). Relations with stereotypes: `«read»`, `«call»`, `«signal»` (usage dependencies between them).

4.6 Four-step analysis procedure

1. Identify the **classes**.
2. Find the **attributes** for each class.
3. Find the **methods/operations** for each class.
4. Find the **associations** between classes.

How to find classes:

1. **Recurring nouns/concepts** in the use cases.
2. **Real-world entities** the system must track.
3. **Application/problem-domain terms** in the data dictionary. Then **review definitions and attributes with stakeholders**.

> **Trap:** do **not** promote every noun to a class. If a noun has no attributes, no identity and no behaviour of its own (e.g. "amount", "date", "grade"), it is an **attribute**, not a class. Conversely "grade" in GES has an enumerated domain and constrains storage rules → it can justify a class/type in the design model.

Chapter 5: State machines and activity diagrams

ELI5

A traffic light has states such as Red and Green; 'change colour' is an action. A pizza-order workflow has activities such as take order, bake and deliver. Use a state machine to explain one object's changing condition; use an activity diagram to explain how work moves among participants.

Answer check: Model the named object/workflow. Check guards, equality, cancellation, timeouts, retry counts and late replies. A dialog map models interaction/screens. Do not force a final node onto permanent monitoring; do not rename actions as states.

Visual: choose the diagram that answers the question

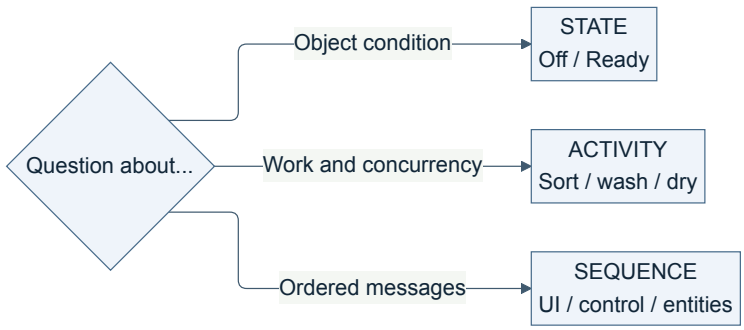


Figure 8: choose the diagram that answers the question

5.1 State machine diagram (a.k.a. statechart, finite state automaton, dialog map)

- Models a system/object by the **states** it can be in.
- **State** = a *stable condition* that exists for a period (or all) of time until an **action/event** drives it to another stable state.
- Finite number of states; transitions determined by previous state and actions.
- Notation: rounded rectangle, name at top, optional **entry / action**, **exit / action**, **do / activity**.
- A diagram begins with a **start node** (filled circle) and ends with an **end node — unless** the system loops forever, in which case there is no end node.
- A transition is **assumed instantaneous** in response to an event. Full label:

```
event(arguments) [guard] / action
```

Dialog map = a state machine used to prototype navigation. Since most UIs show one screen/window at a time (a state) with defined conditions to move (transitions), dialog maps:

- find **missing or incorrect navigation pathways early**, when they are cheap to fix;
- spot **reuse opportunities and redundancies** without premature screen design.

Worked states: see the case-specific dialog/state answers in the past papers. A self-transition exits and re-enters its state, so entry actions run again.

Action semantics: do may complete before state exit; entry/exit run on every entry/exit, including re-entry. Decision guards must be mutually exclusive and cover intended cases (or use `[else]`).

5.2 Activity diagram

- **Activity** = execution of one or more basic operations (rounded rectangle).
- **Control flow** = arrows.
- **Fork** = one input, two or more **concurrent** outputs (**thick bar**).
- **Join** = two or more flows come together into one (**thick bar**).
- **Decision/branch** = one input, several guarded outputs (**diamond**).
- **Merge** = several branches into one (**diamond**).

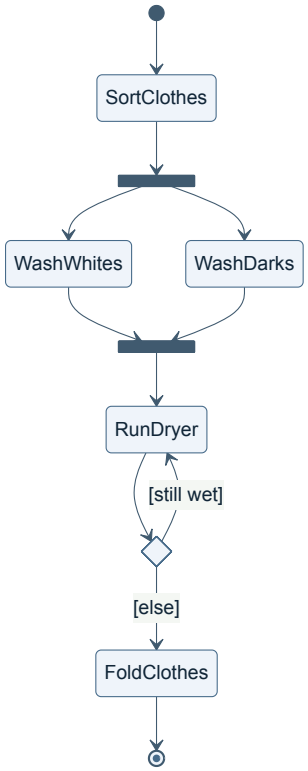


Figure 9: 5.2 Activity diagram

UML check: use action nodes, initial/final nodes and guarded diamonds. A join waits for parallel inputs; a merge rejoins alternatives.

5.3 State machine vs activity diagram (L6 comparison)

	State machine	Activity
Models	The **states** of a system/object and events that move it	The **workflow of activities** (a flow chart)
Node meaning	A stable condition	A unit of work
Focus	Event-driven **reactive** behaviour	Procedural **control flow**, concurrency
Typical use	UI navigation (dialog map), protocol/device states	Business process, algorithm, use-case flow

Trap: fork/join are **bars**; decision/merge are **diamonds**. A diamond for concurrency or a bar for a conditional branch uses the wrong notation.

5.4 The requirements elicitation & analysis activity diagram (memorise the loop)

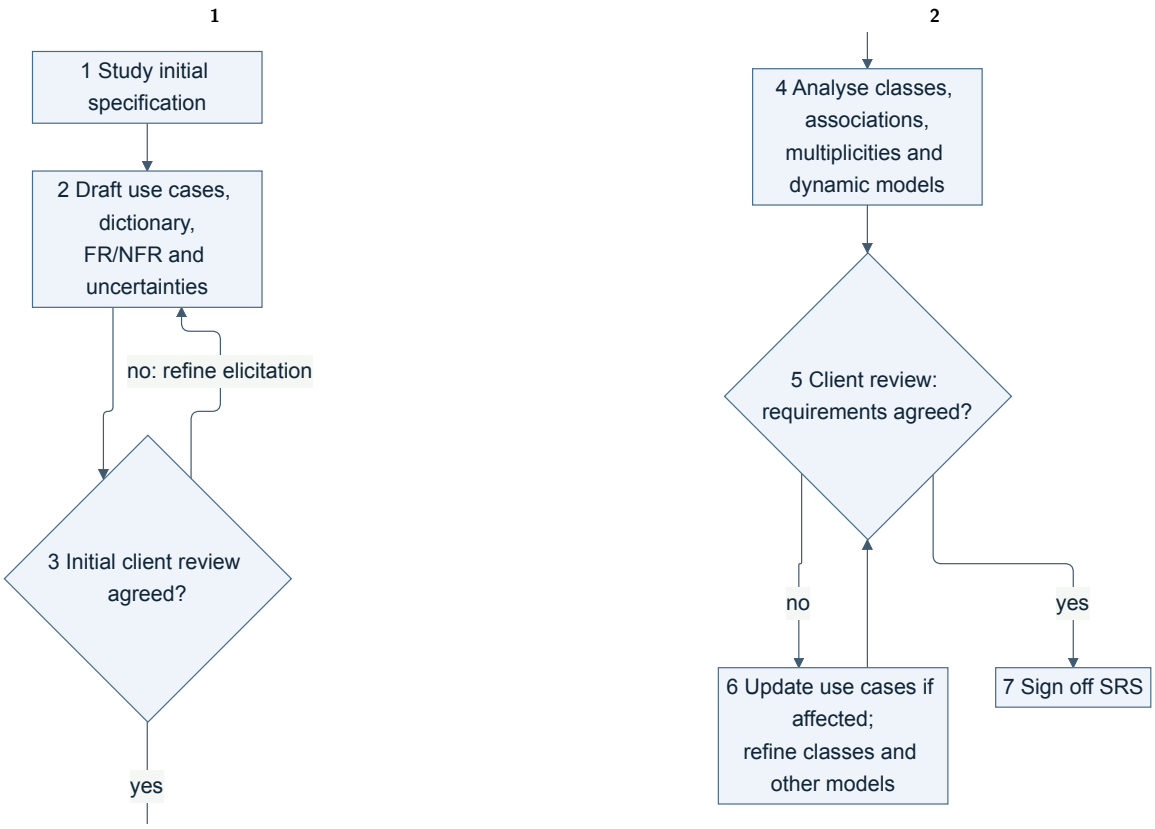


Figure 10: 5.4 The requirements elicitation & analysis activity diagram — follow numbered panels; crossing arrows continue.

Chapter 6: Software Requirements Specification

ELI5

An SRS is an agreement about the cake you ordered: size, flavour, delivery and how to check it. It should not secretly replace your order with the baker's preferred recipe. Someone reading the agreement should be able to tell whether the delivered product meets it.

Answer check: Specify required behaviour, interfaces and qualities; leave arbitrary design choices out. Use identifiers plus trace links, measurable conditions and explicit TBDs. Adding "shall" alone does not repair an ambiguous requirement.

Visual: see the SRS as five compartments

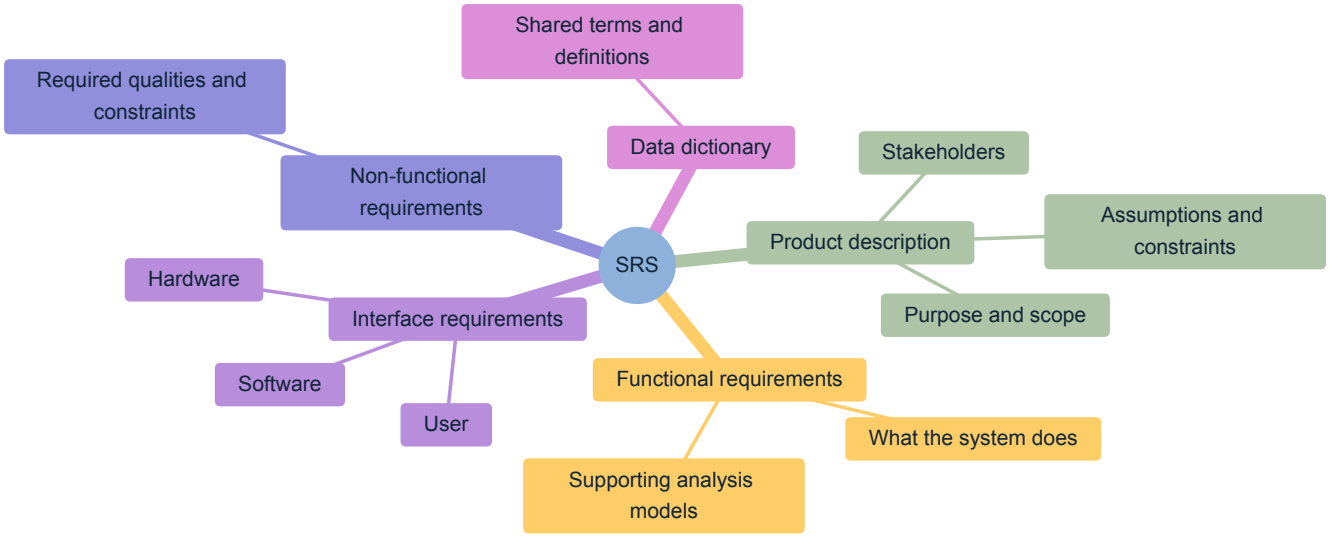


Figure 11: see the SRS as five compartments

6.1 What the SRS is (six definitions that get paraphrased into exam questions)

1. The **output of requirements elicitation and analysis**.
2. An organisation's **written understanding** of a customer's requirements and dependencies at a point in time, prior to design/development.
3. A **two-way insurance policy**: both client and organisation understand each other's requirements.
4. A statement in **precise, clear language** of the functions and capabilities the system must provide, plus required constraints.
5. A **blueprint** for completing the project with as little cost growth as possible.
6. **Detailed functional and non-functional requirements — it does NOT contain design suggestions.**

6.2 Purpose of the SRS (4 items)

1. Give the customer assurance that the organisation understands the problems to be solved and the necessary software behaviour.
2. **Decompose** the problem into component parts.
3. Serve as **input to the design specification** — so it must contain sufficient functional detail for a design solution to be devised.
4. Serve as the **parent/master document for testing and validation**.

6.3 Contents (5 sections — memorise the numbering)

Section	Section	Contents
1	**Product description**	a) Purpose of the system (mission statement) · b) Scope · c) Users and stakeholders · d) Assumptions and constraints
2	**Functional requirements**	Precise numbered statements + activity diagrams, use case diagrams with descriptions, class diagrams, sequence diagrams, communication diagrams, state machine diagrams, UI prototype
3	**Non-functional requirements**	Numbered statements covering performance, standards, reliability, availability, security, maintainability, portability
4	**Interface requirements**	a) **User** (screen formats, layouts, report/menu content, function keys) · b) **Hardware** (ports, instruction sets) · c) **Software** (interfaces to DBMS, OS, maths packages)
5	**Data dictionary**	Unambiguously defines all terms, phrases and abbreviations used in the SRS

Standards reference: **IEEE 830-1998**, *IEEE Recommended Practice for Software Requirements Specifications*.

Who writes it: it should be in **natural language, unambiguous, with diagrams as necessary**; it must be understood by the development team; programmers/managers are seldom the best writers, so it is good practice to **involve a technical writer**.

6.4 The nine language-quality characteristics

#	Characteristic	Test you can apply
1	**Correct**	Only user representatives can confirm; include them in inspections

Continued on next page

#	Characteristic	Test you can apply
2	**Feasible**	Implementable within known capabilities/limits of system and environment
3	**Necessary**	Can you identify its origin (customer need, external interface, standard)? If not, maybe not necessary
4	**Unambiguous**	Multiple readers must reach **one** interpretation
5	**Verifiable**	Can you devise a test / inspection / demonstration for it?
6	**Complete**	Nothing missing; organise hierarchically so reviewers can spot gaps
7	**Consistent**	No conflict with other requirements or higher-level requirements
8	**Modifiable**	Uniquely labelled, expressed separately, change history maintained
9	**Traceable**	Link each requirement to its source and its realisation

6.5 Word list (memorise both columns)

YES Use these words

YES Use* / *Meaning

Shall: Dictates provision of a functional capability

Must / Must not: Establishes performance requirement or constraint

Is required to: Imperative in SRS statements

Are applicable: Includes standards/documentation by reference

Responsible for: Imperative for pre-defined architectures

Will: Cites what the operational/development **environment** will provide

NO Avoid these words

NO Avoid* / *Why

adequate: vague threshold

as a minimum: unbounded

be capable of. . .: avoids commitment

as applicable / as appropriate: optional

effective: unmeasurable

if possible / if practical: optional

timely: unmeasurable

normal: undefined baseline

> The two lists are **independent** — the lecture presents them side by side, so a row on the left is not the "opposite" of the row on the right.

> **Trap (shall vs will). Shall** binds the *system*; **will** describes what the *environment* supplies. "The system **shall** . . . " is a requirement; "The user **will** provide a valid card" is an assumption about the environment. Swapping them blurs responsibility for the behaviour.

Chapter 7: Software process models

ELI5

Waterfall is like planning a whole house before construction; incremental delivery is like opening usable rooms in stages; reuse/configuration is like assembling suitable existing modules. None is automatically best. Choose according to change, risk, available components and the need for predictable agreement.

Answer check: Choose using change, risk, component fit and constraints. Give a case-specific benefit and trade-off. Waterfall includes testing; agile includes planning. Customer involvement does not eliminate the cost of late change.

Visual: compare how the work is arranged

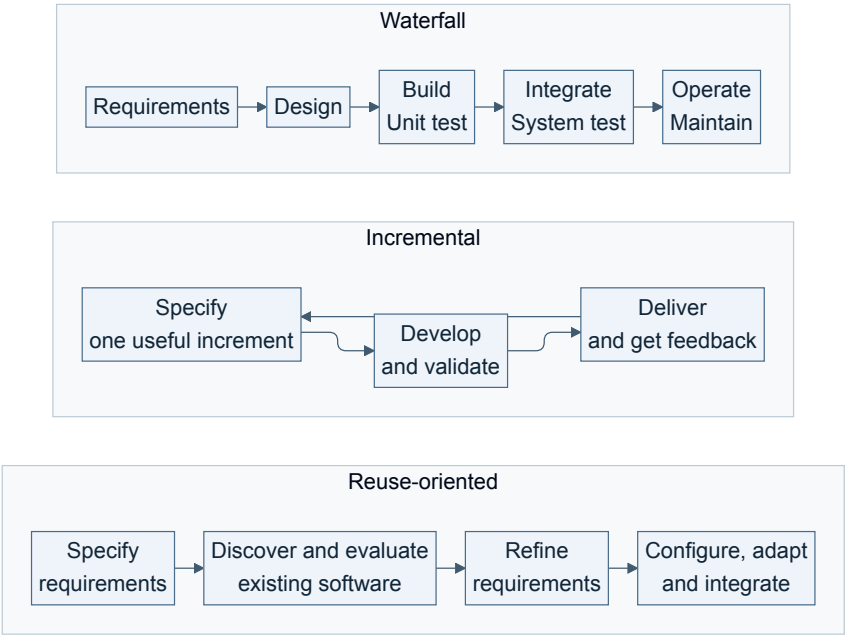


Figure 12: compare how the work is arranged

7.1 Definitions

> A **software process** is *a structured set of activities required to develop a software system.*

All processes involve specification, design/implementation, validation and evolution (Chapter 1). Their arrangement and feedback differ.

> A **software process model** is *an abstract representation of a process — a description from some particular perspective.*

Process descriptions include, besides activities and their ordering:

- **Products** — outcomes of a process activity;
- **Roles** — responsibilities of the people involved;
- **Pre- and post-conditions** — statements true before/after an activity or product.

7.2 Plan-driven vs agile

	Plan-driven	Agile
Planning	All activities planned **in advance**; progress measured against the plan	Planning is **incremental**; process easily changed to reflect changing requirements
Iteration	Iteration occurs **within** activities	Specification, design, implementation, testing are **interleaved**; outputs negotiated during development
Outputs	Decided **up front**	Decided through **negotiation** during the process

Most practical processes include **elements of both**. **There are no right or wrong software processes.**

7.3 The three general models

Model	Shape	When used	Key benefit	Key problem
Waterfall	Separate, distinct, sequential phases	Requirements **well understood** and changes limited; large systems-engineering projects developed at several sites	Plan-driven coordination across sites	**Difficulty of accommodating change** once under way; inflexible partitioning
Incremental	Specification, development, validation **inter-leaved**; may be plan-driven **or** agile	Most business systems	Lower cost of accommodating change; easier customer feedback; earlier delivery of value	**Process not visible** (managers need deliverables); **structure degrades** as increments are added unless you refactor
Integration & configuration (reuse-oriented)	Assemble from existing components/COTS	Systems buildable from existing assets; standard for many business systems	Reduced cost and risk; faster delivery	**Requirements compromises inevitable**; loss of control over evolution of reused elements

Waterfall phases (exact names): Requirements analysis and definition → System and software design → Implementation and unit testing → Integration and system testing → Operation and maintenance.

Visibility note: managers need regular deliverables, but documenting every version can be costly; refactoring controls the structural degradation identified in the table.

Reuse-oriented phases (5): requirements specification → software discovery and evaluation → requirements refinement → application system configuration → component adaptation and integration.

Types of reusable software (3): stand-alone application systems (COTS) configured for a particular environment; collections of objects packaged for a component framework (.NET, J2EE); web services developed to service standards and remotely invocable.

> **Trap #1.** "Incremental = agile" is **false** — incremental development *may be plan-driven*.

7.4 Why costs are high / why maintenance dominates (Tutorial 4)

Prepare a *reasoned* answer, not a slogan:

- **Costs**: requirements churn, integration of heterogeneous components, testing effort grows super-linearly with interfaces, coordination cost in teams, environment/tooling/long lifetimes.
- **Maintenance**: software is soft — it *can* change, so it *must* change (new requirements, new platforms, bug fixes). Systems can live for decades; ongoing fixes and adaptation accumulate over that lifetime. Unlike hardware, there is no manufacturing cost, so evolution dominates total cost.
- **Characteristics affecting development**: size (source/binary/users/developers/lifetime/versions), complexity (environment, technologies, hardware), cost and schedule constraints, non-functional requirements (performance, availability, security, scalability, extensibility).
- **Two distinct differences from hardware manufacturing**: (1) software is **designed, not manufactured** — copying is free; (2) software **does not wear out** — it degrades by change, and hardware failures follow a bathtub curve.

Template W — change/process answer: clarify and record change; assess effects on requirements, design, code, tests, cost and schedule; agree revised scope/baselines; replan and verify. Early prototypes/reviews and increments expose change sooner, but do not remove late rework. Incremental delivery can be plan-driven or agile. For reuse, evaluate fixed-component fit, prototype risky integrations, negotiate acceptable compromises and isolate mismatches behind adapters. **Comparison:** Waterfall favours stable baselines, sequential delivery and requirements/review/acceptance involvement; Agile favours reprioritised usable increments, frequent customer feedback and earlier adaptation.

Chapter 8: Agile, XP and Scrum

ELI5

A sprint is a short team promise to produce a usable increment and learn from it. The backlog is the prioritised queue; the Product Owner orders value; developers choose and organise feasible work; the Scrum Master helps the process work. Velocity is the team's measured delivery pace, not a demand to work faster.

Answer check: Use the Sprint Goal, actual capacity and completed velocity. Developers retain ownership; the Scrum Master facilitates. Negotiate scope transparently instead of extending the timebox or lowering quality. Incomplete items earn no completed velocity.

Visual: follow one sprint through its feedback loops

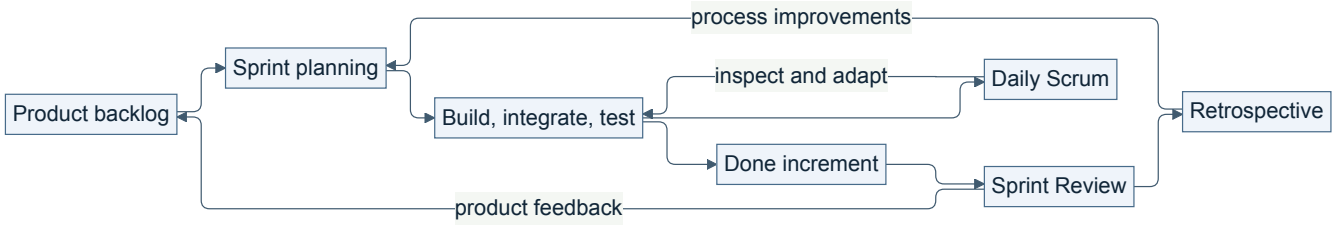


Figure 13: follow one sprint through its feedback loops

8.1 Why agile emerged

Rapid delivery became the dominant requirement: requirements change too fast for stable specs; plan-driven development does not meet these needs; agile methods (late 1990s) aimed to **radically reduce delivery time for working software**.

Agile development characteristics:

- program specification, design and implementation are **inter-leaved**;
- the system is developed as **a series of versions/increments**, with stakeholders involved in specification and evaluation of each version;
- **frequent delivery** of new versions for evaluation;
- **extensive tool support** (e.g. automated testing);

- **minimal documentation** — focus on working code.

8.2 Agile manifesto (4 values — quote them exactly)

> We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:
>
> - **Individuals and interactions** over processes and tools
> - **Working software** over comprehensive documentation
> - **Customer collaboration** over contract negotiation
> - **Responding to change** over following a plan
>
> That is, while there is value in the items on the right, we value the items on the left more.

Trap: it does **not** say "no documentation". Both sides have value; the left is *valued more*. Saying "agile means no documentation" is inconsistent with the manifesto.

8.3 The five agile principles

Principle / Description

Customer involvement: Customers closely involved throughout; provide and prioritise requirements, evaluate iterations

Incremental delivery: Software developed in increments; customer specifies requirements for each increment

People, not process: Recognise and exploit team skills; let teams develop their own ways of working

Embrace change: Expect requirements to change; design to accommodate change

Maintain simplicity: Focus on simplicity in software and process; actively eliminate complexity

Applicability: product development (small/medium product for sale) and custom in-house development with a committed customer and few external rules/regulations.

Across organisations: managers may be risk-averse; large organisations have quality procedures incompatible with agile bureaucracy; agile works best with **high-skill teams** but large orgs have a wide skill range; cultural resistance where conventional processes are entrenched.

8.4 Extreme Programming (XP)

Extreme approach: new versions possibly **several times per day**; increments delivered **every 2 weeks**; **all tests run for every build**, and the build is accepted only if tests pass.

Ten practices in L9, pp. 15–16:

Practice / Description

Incremental planning: Requirements on **story cards**; stories for a release chosen by time available and relative priority; developers break stories into **tasks**

Small releases: Minimal useful functionality providing business value first; releases frequent, adding functionality

Simple design: Enough design for current requirements and no more

Test-first development: Automated unit test framework; tests written **before** functionality

Refactoring: Continuous improvement of code as soon as improvements are found

Pair programming: Two developers at one machine, checking each other's work

Collective ownership: Pairs work on all areas; no islands of expertise; anyone can change anything

Continuous integration: Task integrated into the whole system as soon as complete; after any integration all unit tests must pass

Sustainable pace: Large amounts of overtime unacceptable — reduces quality and medium-term productivity

On-site customer: A representative end-user available **full time**; a member of the team, responsible for bringing requirements

Related lecture topics: **user stories for specification** and **automated test harnesses**. These are not two extra entries in the ten-row practice table.

Key influential practices (4): user stories for specification, refactoring, test-first development, pair programming.

XP refactoring check: improve structure continuously because future changes are hard to predict; remove duplication and clarify responsibility. It improves understanding/changeability and can reduce documentation needs, but architectural refactoring is expensive.

Pairing check: rotating pairs share knowledge, ownership and informal review, reducing key-person risk and encouraging refactoring. Productivity gains are evidence-dependent, not guaranteed.

TDD check: executable, self-checking tests clarify requirements; run old and new tests after each addition for regression protection.

> **Trap:** TDD $\neq$ "testing at the end". TDD = test **first**, watch it fail, then implement. The "tests are written as programs rather than data" point is what makes automation possible.
>
> **Trap:** XP is technically focused and **hard to integrate with management practice** in most organisations — consequently the *method as originally defined is not widely used*, even though its practices are.

8.5 Scrum

Scrum focuses on **managing iterative development** rather than on specific technical practices. The following describes the supplied L9 slides; do not treat their historical terminology as a statement of the current Scrum standard. The slides themselves contain variants: p. 32 says no more than 7 developers; p. 41 says 5–9. Neither is needed to infer the headcount in Tutorial 5 Q6.

Three phases:

1. **Outline planning** — establish general objectives, design the software architecture.
2. **Sprint cycles** — each develops an increment.
3. **Project closure** — wrap up, complete documentation (help frames, user manuals), assess lessons learned.

Terminology (memorise verbatim-ish):

Term / Definition

Development team: A **self-organising** group, **no more than 7 people**, responsible for developing the software and essential project documents

Potentially shippable product increment: The increment delivered from a sprint; ideally in a finished state needing no further work

(e.g. testing) before inclusion in the final product — *in practice not always achievable*

Product backlog: A list of "to do" items the Scrum team must tackle: feature definitions, requirements, user stories, or supplementary tasks such as architecture definition or user documentation

Product owner: An individual (or small group) who identifies features/requirements, **prioritises** them, and continuously reviews the backlog to ensure the project meets critical business needs; may be a customer, product manager, or stakeholder representative

Scrum (meeting): A **daily** meeting of the Scrum team reviewing progress and prioritising work for that day; ideally short, face-to-face, whole team

ScrumMaster: Ensures the Scrum process is followed, guides the team in effective use of Scrum, interfaces with the rest of the company, protects the team from outside interference. **Should NOT be thought of as a project manager** (the developers are adamant; others may disagree)

Sprint: A development iteration, usually **2–4 weeks**

Velocity: An estimate of how much product-backlog effort the team can cover in one sprint; basis for estimating what fits and for measuring improvement

Sprint cycle: fixed length 2–4 weeks → planning starts from the **product backlog** → **selection phase** (whole team + customer select features for the sprint) → team self-organises and develops, **isolated from customer and organisation**, all communication channelled through the ScrumMaster, whose role is to **protect the team from external distractions** → at the end, work reviewed and presented to stakeholders → next sprint begins.

Teamwork: the Scrum Master facilitates impediment resolution and communication; the Daily Scrum makes progress, problems and next steps visible for short-term replanning.

Benefits: manageable chunks, adaptation to unstable needs, team visibility/communication, timely customer feedback, and trust/positive culture.

Practical Scrum material (L9, pp. 38–61; some pages are image-only):

- **Product backlog:** owned by the PO; ordered requirements/features/tasks for the product.
- **Sprint backlog:** selected product backlog items **plus the plan** for delivering the increment and Sprint Goal; owned by the Development Team. The slides protect the selected sprint scope and goal, while task estimates and remaining work are updated daily. Unfinished work returns to the product backlog for reconsideration; it earns no completed velocity points.
- **Product increment:** integrates Done work with previous increments; tested and potentially releasable. Potentially shippable does not mean a release must occur every sprint.
- **Sprint planning:** PO presents ordered items; developers decide how much fits and how to build it. Tasks are estimated in hours (the slide suggests 1–16 hours); points size backlog items, not hours.
- **Daily Scrum:** 15-minute time box; yesterday's progress, today's plan and impediments.
- **Sprint Review:** inspect the product with stakeholders and revise the product backlog. The slide gives a time box of 1 hour per week of sprint length.
- **Sprint Retrospective:** inspect people, process and tools; agree improvements and actions for the next sprint. The slide gives 45 minutes per week, after the Review.
- **Burndown chart:** remaining work against time; compare actual progress with planned work.

Good backlog = DEEP (L9, pp. 54–55; Tutorial 5 Q3): Detailed appropriately (small, detailed near-term items; larger, less detailed later items), **Emergent** (continually changes), **Estimated** (effort estimates inform prioritisation), **Prioritised** (especially work for upcoming sprints). Refinement continuously adds detail, estimates and reprioritises items; p. 56 suggests up to 10% of team time as a guideline.

User stories (L9, pp. 57–61): "As a [role], I want [action], so that [benefit]." An **epic** is a large body of work split into smaller stories. The **3Cs** are **Card** (intent), **Conversation** (shared understanding) and **Confirmation** (acceptance criteria). Acceptance criteria define when work is finished; they can be functional or non-functional. State intent, remain independent of implementation and avoid unnecessary design detail. INVEST below evaluates the story itself.

8.6 INVEST (Tutorial 5 — memorise)

Letter	Meaning	Test
I	**Independent**	Can be developed/released without depending on another story
N	**Negotiable**	Detail is negotiated between team and PO, not fixed upfront
V	**Valuable**	Delivers value to a user/customer
E	**Estimable**	Team can size it
S	**Small**	Fits in a sprint
T	**Testable**	Has an oracle; can be verified

8.7 Velocity & cost arithmetic (Tutorial 5 Q6)

Total backlog = **64 points**. Scrum Team = Product Owner + ScrumMaster + **6** developers = **8** people. Cost per member per sprint = **$2K**. **8 sprints** needed.

```
Velocity  = total points / number of sprints = 64 / 8
↪  = 8 points per sprint
```

```
Cost      = people * cost per person per sprint *
↪  sprints
          = 8 * $2,000 * 8 = $128,000
```

Trap: do **not** drop the PO and ScrumMaster. The question says *"each member in the Scrum Team"* and then enumerates them. Dropping them gives 6 × 2K × 8 = **$96,000** (wrong). State your assumption explicitly if you think the question is ambiguous — make

the calculation auditable.

New-team velocity: use a trial sprint/history from comparable work as a provisional estimate, then measure completed points and adjust for capacity. It is not assigned hours or a mandated deadline.

Template S — Scrum scenario answer: identify the Sprint Goal and customer value; use completed velocity/real capacity; expose impediments and let developers solve technical work; PO/developers negotiate scope while protecting quality and timebox; review/retrospect and adapt. Forecast remaining sprints = remaining effort / representative velocity. Split oversized stories; investigate dependencies early. Urgent work normally enters a future backlog; only the PO cancels an obsolete goal. A Done increment is tested, integrated and potentially shippable.

Chapter 9: Architecture and system design

ELI5

Architecture is a map of major parts and their communication. Choose the map using the case's important qualities and constraints.

Answer check: State the NFR, draw components/ connectors, explain the mechanism, then name a cost or failure mode.

Visual: recognise the architectural organisation

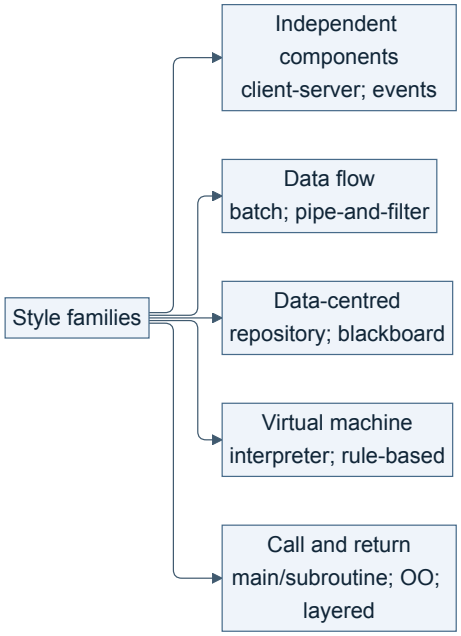


Figure 14: recognise the architectural organisation

9.1 Why design is hard here

Scale, users, developers, versions, long lifetimes and changing technology increase complexity; cost, schedule, maintenance and NFRs constrain choices.

9.2 Definition

Architecture = components + connectors + constraints. It abstracts implementation details, shows structure/interactions and demonstrates properties of interest. Connectors can be distributed across components rather than exist as separate runtime objects.

	Definition	Examples
Component	A unit of software that performs some function **at run time**	programs, objects, processes, clients, servers, databases
Connector	Interaction among components: communication, coordination or cooperation	shared variable access, procedure calls, RPC, communication protocols, data streams, transaction streams

9.3 Architectural style

A style defines a family through component/connector vocabulary, composition constraints and semantic models. Instances can vary in count, interaction mechanism and component function. Event systems use implicit or explicit invocation; styles may coexist.

9.5 Layered architecture

Upper layers call lower services. Benefits: abstraction, extension and lower-layer reuse. Costs: traversal overhead, difficult layer boundaries and poor fit for some systems. A direct lower-to-concrete-upper callback breaks the dependency direction; Observer abstracts that up-call.

9.6 Client–server

Separate client/server computation supports concurrency, scalability, integration and change. Network protocols must handle delivery, failure and coordination; correctness is harder than a local call.

9.7 Pipe-and-filter

Filters transform streams through pipes without shared state. Good for batch/stream transformations; weak for interactive shared-state behaviour.

9.8 Data-centred (repository / blackboard)

Components communicate through a central repository/ blackboard. It integrates independently changing producers/ consumers, but creates shared coupling, contention and a potential single failure point.

9.9 Design principles (L12 and L14)

Principle	Rule and failure check
SRP / high cohesion	One reason to change; related responsibilities stay together. A God class or input parser that also writes the database fails this check.
OCP / identify variation	Encapsulate what varies; extend behind a stable contract. A type-switch scattered through clients forces modification.
Interfaces / low coupling	Depend on abstractions and minimise cross-module dependencies; avoid fields tied to concrete vendors.
DRY	Keep one authoritative representation of knowledge; duplicated creation logic is a warning.
Least Knowledge	Talk to immediate collaborators; avoid chains such as a.getB().getC().doIt().
Separation / modularity	Separate UI, rules and data; use abstraction and modular boundaries to support reuse. Business rules in event handlers defeat this.
Composition over inheritance	Assemble behaviour rather than grow fragile subclass hierarchies.

9.10 Design rationale frame (design rationale)

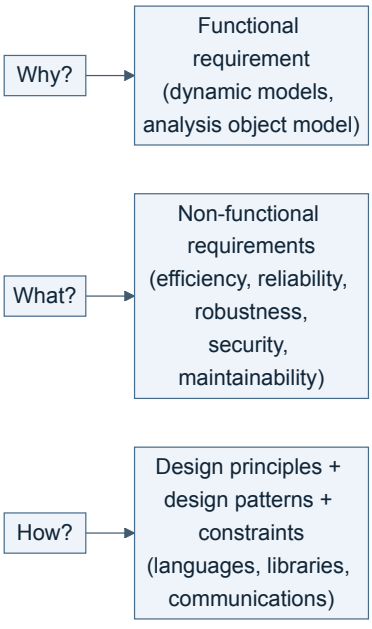


Figure 15: 9.10 Design rationale frame (design rationale)

Template A — architecture justification: [case NFR and why it matters]; [style, components and connectors]; [mechanism supporting the NFR]; [cost/failure mode and assumption]. Compare alternatives against the same requirement/ workload; class or edge count alone is not quality evidence.

9.11 The design process (descriptive model)

Model requirements; propose alternatives; validate against requirements; revise mismatches; elaborate, compare and select a blueprint.

Chapter 10: Object design and design patterns

ELI5

Strategy changes behaviour; Factory creates the chosen object; Observer tells subscribers about changes; Facade offers one simple entry point.

Answer check: Name the problem and variation, map roles to the case, show relationships, then give a benefit and cost.

Pattern selection at a glance

Pattern	Use when / mechanism / cost
Strategy	Behaviour/algorithm varies; Context delegates to a stable interface. Extra indirection and selection.
Factory	Concrete type is known at runtime; centralise creation. Selection/configuration still needs maintenance.
Observer	Subscribers change over time; attach/detach/notify through an abstraction. Order, overhead and life-cycle cleanup matter.
Façade	A subsystem is complex; offer one higher-level entry point. Poor/excessive facades add rework and overhead.

10.1 What object design adds over system design

System design* / *Object design

System design: Architecture, subsystems, layers, styles

Object design: **Detailed** class diagram, interfaces, patterns

System design: Decides *where* responsibility lives

Object design: Decides *how* objects collaborate

Object-design activities: reuse; interface specification (operations, arguments, types, exceptions); restructuring for design goals; speed/memory optimisation.

10.2 Why patterns at all?

A pattern names a reusable response to recurring design forces. Its mechanism and consequences matter more than its name.

10.3 The four elements (L13 definition)

Name; problem; solution; consequences. State when it applies, collaborating roles/relationships, and the resulting benefits/costs.

10.4 GoF 23 — grouping (know the buckets)

Category* / *Patterns

Creational (5): Abstract Factory, Builder, **Factory Method**, Prototype, **Singleton**

Structural (7): Adapter, Bridge, Composite, **Decorator**, **Façade**, Flyweight, Proxy

Behavioural (11): Chain of Responsibility, **Command**, Interpreter, Iterator, Mediator, Memento, **Observer**, State, **Strategy**, Template Method, Visitor

10.5 Strategy

Strategy: Context holds the Strategy abstraction and delegates operation(). Concrete strategies implement interchangeable behaviour, selected at runtime. Benefit: independent extension/encapsulation. Cost: indirection and a selection decision. The Factory below is optional when Strategy alone suffices.

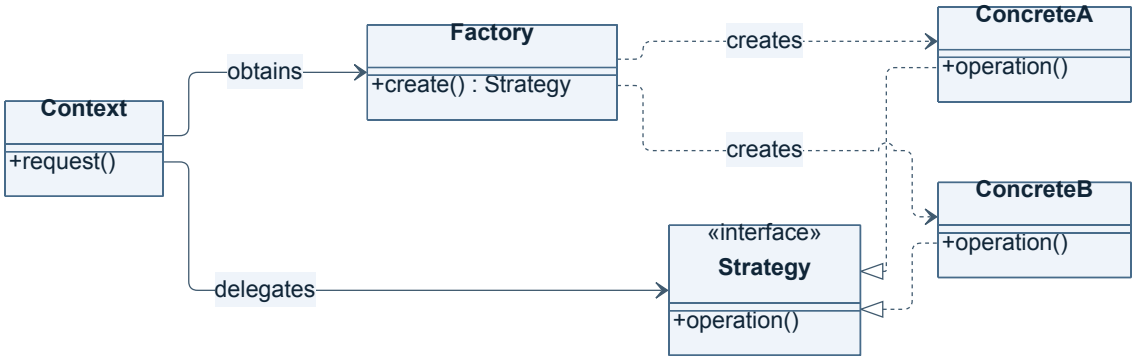


Figure 16: Canonical Strategy + Factory: substitute the paper's case roles. Factory creates; Context delegates.

Template P — combined-pattern answer: identify independent behaviour and creation variation; map Context, Strategy, Factory and concrete implementations; show create and delegate relationships; explain extension behind a stable interface plus indirection/configuration cost. A switch-based factory still changes for a new type; registration/configuration or dynamic loading can move selection out of client code.

10.6 Observer (L13; Tutorial 7)

Observer: Subject holds subscribers through the Observer abstraction; attach/detach manage membership, notify calls update. Runtime subscribers and selective interests decouple publisher from concrete listeners; callbacks or messages avoid blind polling. Also called Subscriber–Publisher or Consumer–Producer.

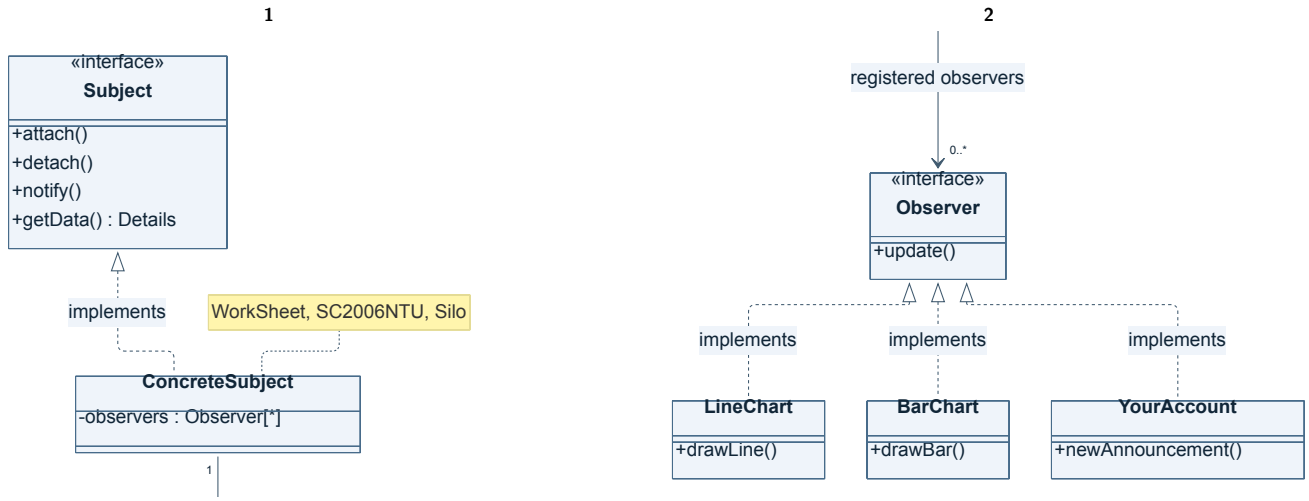


Figure 17: 10.6 Observer (L13; Tutorial 7) — follow numbered panels; crossing arrows continue.

	Pull	Push
Subject sends	Simple notification (`update()`)	Detailed change data (`update(data)`)
Observer then	Calls back `getData()` — **two-way**	Needs no callback — **one-way**
Coupling	Observer must know the subject's getters	Subject must know what observers need; may over-send
Efficiency	Observer fetches only if interested	Wastes data when observer is uninterested

Trade-off: independent reuse and broadcast versus notification overhead, update storms, unspecified order and subscription-cleanup leaks. Selective notification filters registered interests; either push or pull can be used.

10.7 Factory (Static Factory)

Simple/Static Factory centralises runtime selection and construction, removes duplicate creation code and can provide lazy or singleton instances. It differs from GoF Factory Method (subclass-overridden creation) and Abstract Factory (families of related products).

Examples: JDBC creates vendor-specific Connections; Swing selects Look&Feel; Eclipse loads registered ViewPart subclasses. Dynamic loading supports new configured classes without hard-coding them in the client. Construction remains a separate responsibility from Strategy delegation.

10.8 Façade

Façade provides a unified higher-level interface to a subsystem. It reduces client dependencies and simplifies use. Costs: a poor facade needs rework; excessive facades add complexity/overhead. Direct subsystem access can still exist. Adapter instead converts an incompatible interface.

10.9 Other patterns you must be able to *name* for a scenario

Pattern	Scenario signal	Sketch
Singleton	"exactly one instance", configuration/registry, lazy init	private ctor, static `getInstance()`
Abstract Factory	"create **families** of related objects" (controller *or* look-and-feel for a view)	`AbstractFactory.createButton()/ createMenu()`
Factory Method	Subclasses decide which class to instantiate	`creator.factoryMethod()`
DAO	Isolate persistence from domain logic	`CustomerDAO.find(id)`
Decorator	Attach **additional** responsibilities dynamically (e.g. scrolling on a view)	`ScrollingView wraps View`
Composite	Nested views (Panel contains Button; nested Panels)	tree of `Component`
Command	Undo/redo of user actions	`Command.execute()/undo()`
Adapter	Convert one interface into another expected by the client	wrapper class
Proxy	Surrogate controlling access (remote/lazy/protection)	same interface as real subject

Template C — combine/criticise patterns: combine only for distinct compatible forces; shared classes may play several roles if responsibilities stay clear. Give one concrete benefit and one cost (interfaces, indirection, lifecycle/configuration or tests). MVC combines Observer and Strategy; a smaller design can be preferable when variation is absent.

Chapter 11: Model-View-Controller

ELI5

One spreadsheet Model can feed a table and chart. Controllers interpret input; a Model change notifies Views so they refresh consistently.

Answer check: Trace input, Model update, notification and View refresh. MVC is triangular; it can coexist with layered services/persistence.

11.1 The problem MVC solves

Use MVC for interactive systems with changing input/display behaviour and multiple synchronised views of shared information. It separates presentation and interaction from core rules/data.

11.2 Components

Component	Responsibility	Stereotype	Example
Model	Contains the **processing (operations) and the data**	«control» + «entity»	`WorkSheet`, `Sheet Manager`
View	Defines and manages **how data is presented**; each View provides a specific presentation of the same Model; each View **observes** the Model	«boundary»	`LineChart`, `BarChart`
Controller	Manages **user interaction**: captures input (mouse clicks, key presses) and passes interactions to View and Model; each View is associated with a Controller; user interacts **solely through Controllers**	«boundary»/«control»	`EditWS`, `EditWSCtrler`

11.3 The triangular structure (know the arrows by heart)

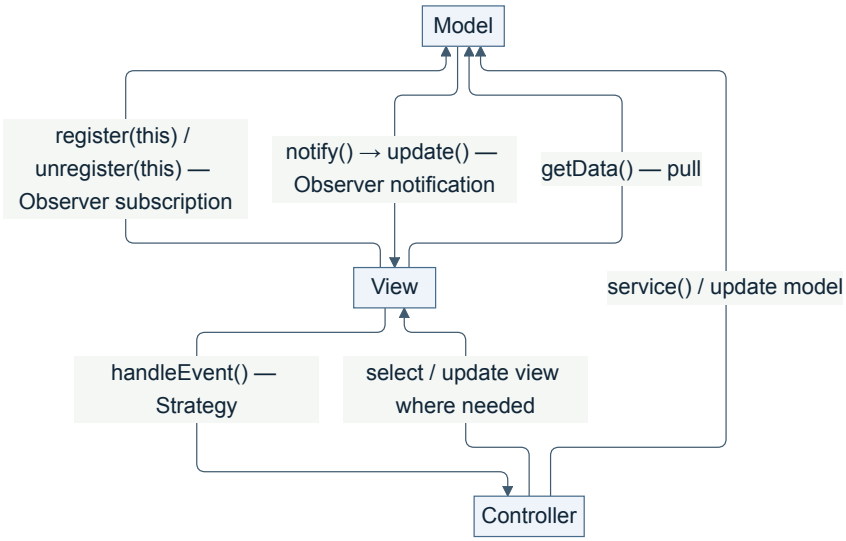


Figure 18: 11.3 The triangular structure (know the arrows by heart)

Arrow check: View registers with Model; Controller invokes Model services; Model notifies subscribed Views; Views pull data and redraw. A subscribed Controller may also receive Model notifications (L15 pp. 22, 25). Observing the Model is different from observing the Controller.

11.4 Which patterns live inside MVC

Relationship	Pattern	Why
View ↔ Model; subscribed Controller ↔ Model	**Observer**	Multiple views, loose coupling, automatic update
View ↔ Controller	**Strategy**	View delegates interface behaviour; controller swappable → different responses to input; also View ↔ Look & Feel
Controller creation / Look & Feel	**Abstract Factory**	Create a controller or LAF for a view
Nested views	**Composite**	Panel contains Button; nested Panels
Extra UI functionality (scrolling)	**Decorator**	Attach additional UI functionality to a view

Continued on next page

Relationship	Pattern	Why
Undo/redo of user actions	**Command**	Support undo/redo

11.5 Why MVC over plain layering (the exam argument)

Template M — MVC alternative: map case state/rules to Model, presentations to Views and input handling to Controllers. Use Observer for consistent refresh; Strategy separates input/look-and-feel variation; Abstract Factory creates related UI objects. Benefits: multiple-view consistency, separate UI/domain change, testability. Costs: notification/lifecycle and multi-artifact complexity. It avoids delayed/excessive polling and concrete view-to-view callbacks; layering can remain underneath.

11.6 Pros / cons

Pros / Cons

Pros: Simultaneous development — multiple views per model; models can have multiple views

Cons: Code navigability and a pronounced **learning curve**

Pros: High cohesion — related actions grouped on a controller; views for a model grouped

Cons: Framework navigation is complex: new layers of abstraction, users must adapt to MVC's decomposition criteria

Pros: Low coupling among models, views and controllers

Cons: Multi-artifact consistency — one logical change touches three artefacts

Traps: MVC is not three deployment tiers. Its Controller is not automatically the analysis control stereotype: the course puts View/Controller in presentation boundaries, and entities plus business controls in Model. Controller updates Model; Model notifies subscribers; Views pull data and redraw.

Chapter 12: Testing fundamentals and basis paths

ELI5

A test is a planned experiment. The input is what you try; the oracle says what should happen; the log records what did happen. Black-box testing asks whether behaviour meets the promise; white-box testing uses the code's branches and paths to choose experiments.

Answer check: State the test level, input and independently justified oracle. For CFG work, keep graph conventions, node/edge counts and feasible traces consistent. CC is a basis size, not a minimum statement/branch suite. Generated tests are not evidence of execution.

Visual: separate test selection from code coverage

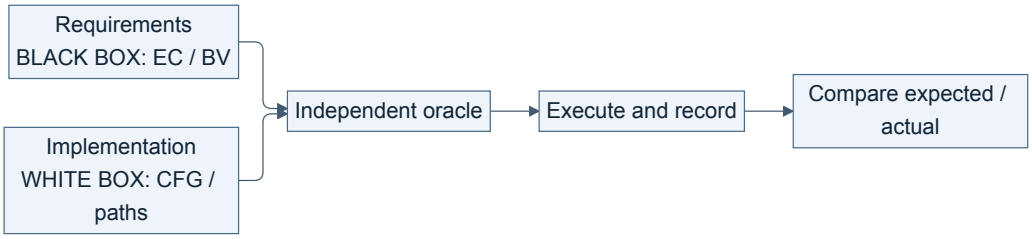


Figure 19: separate test selection from code coverage

12.1 Definitions

> A **software bug** is *an error, flaw, failure or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.*

> **Software testing** is *an investigation conducted to provide stakeholders with information about the quality of the software product or service under test.*

Three properties that decide many MCQs:

1. It is **dynamic execution** (you run the software).
2. It aims **to find bugs, not to prove that no bugs exist** (Dijkstra).
3. It reduces the **perceived risk** of the system not working to an acceptable value.

Not testing: testing $\neq$ debugging. Testing finds the failure; debugging locates and fixes the fault.

12.2 Why exhaustive testing is impossible

For a 16-bit integer there are $\mathbf{2^{16} = 65\,536}$ inputs. Even that is affordable once, but for realistic signatures the input space is astronomically large, so we must **select** inputs cleverly. That selection is exactly what **EC** and **BV** testing formalise.

Selection rule: random samples can miss the small regions where a defect changes output; use EC/BV to target those regions.

12.3 Black box vs white box

	Black box	White box
What to analyse	**Requirements and specifications**	**Implementation details**, internal paths and structure
Example requirement	"The income of an applicant must be between $1500 and $8000 per month"	—
Techniques	**Equivalence class testing, boundary value testing**, state-based testing	**Control flow testing** (CFG, basis paths), data flow testing
Pros	Directs the tester to a **very small subset** of inputs likely to find bugs; applicable at **all levels**	Gives a **coverage** notion
Cons	**Cannot infer code coverage from black-box inputs alone**; EC/requirement coverage can still be tracked	Needs code; misses "missing requirement" bugs

12.4 Levels of testing (4)

Level / ***Scope***

Unit testing: Test a unit individually (the level used in this course)

Integration testing: Test units **in combination**

System testing: Everything: functionality, usability, reliability, performance, portability, security. . .

Acceptance testing (UAT): Customer accepts the software and pays

12.5 Ingredients of a test case (5 — memorise)

Ingredient / ***Description***

Name: Distinct name of the test case

Location: Path to the test program and its inputs

Input: Input data or commands

Oracle: **Expected** test output

Log: **Actual** output produced by the test

JUnit mapping: `@Test public void testQualifiedApplication()` = **Name**; the class/package = **Location**; the arguments to `analyzeApplication(...)` = **Input**; the expected argument in `assertEquals(ApplicationResult.Approval, result)` = **Oracle**; the recorded actual `result` and test report = **Log**.

> **Trap:** *oracle* = expected; *log* = actual. Keep this distinction when naming test-case components.
>
> **Oracle check:** derive expected behaviour from requirements; separately validate wrong requirements with stakeholders.

12.6 Execution order: cascading vs independent tests

	Cascading (dependent)	Independent
Shape	Create → Read → Update → Delete → Read-deleted (each builds on the last)	Self-contained cases for each feature
YES Benefit	**Smaller and simpler** test cases; realistic end-to-end sequence	**Parallel** test execution; failures are isolated
NO Cost	**One failure invalidates subsequent tests**	**Larger and more complex** test cases (each sets up its own state)

12.7 Basis path testing (white box)

> **Basis path testing** (structured testing) is a white-box method for designing test cases intended to exercise a **basis set of linearly independent control-flow paths**.

Process:

1. Draw the **Control Flow Graph (CFG)**.
2. Compute the **Cyclomatic Complexity (CC)**.
3. Derive a set of **linearly independent paths** (the basis set) — there are **CC** of them.
4. Generate a test case for **each** basis path.

Cyclomatic complexity (three equivalent formulas):

```
CC = E - N + 2        (E = edges, N = nodes; per
↪  connected component, P components -> +2P)
```

```
CC = number of binary decision nodes + 1   (one
↪  connected CFG)
CC = number of bounded regions + 1
```

For a connected, single-entry/single-exit CFG, CC is the number of paths in a **graph basis**. It is **not** a minimum for statement or branch coverage. Some graph paths are infeasible because of program data, so a structural basis is not automatically an executable test suite. L16, p. 10, loosely says "all possible paths"; p. 11 illustrates a basis set instead. Loops can create infinitely many execution paths, so these are different coverage goals.

Worked CFG: Section 12.9 distinguishes graph bases from feasible execution and minimum coverage suites.

12.8 Coverage criteria

Criterion	Requirement	Implication
Statement coverage	Every **node** executed at least once	Weakest; misses branch-only bugs
Branch (decision) coverage	Every **edge** (true and false of every decision) taken at least once	**100% branch ⇒ 100% statement**, but not vice versa
Path coverage	Every **path** executed	Usually infeasible (loops ⇒ infinite paths)
Basis-path coverage	A feasible independent path set exercised; report infeasible structural paths	Practical middle ground

> **Trap:** "statement coverage implies branch coverage" is **false** (a `if` with no `else` whose body always executes gives 100% statement but not 100% branch). The reverse *is* true.

12.9 Worked white-box example you should be able to reproduce — Tutorial 9

```
public int sum(int n, int upperbound){
 1  int result, i;
 2  result = 0;
 3  i = 0;
 4  if(n < 0) {
 5    n = -n;
 6  }
 7  while(i<n && result <= upperbound){
 8    i = i + 1;
 9    result = result + i;
10  }
11  if(result <= upperbound){
12    System.out.println("The sum is " + result);
13  } else {
14    System.out.println("The sum is too large!");
15  }
16  return result;
}
```

CFG — 12 nodes (1, 2, 3, 4, 5, 7, 8, 9, 11, 12, 14, 16); the compound `while` guard is **one** node:

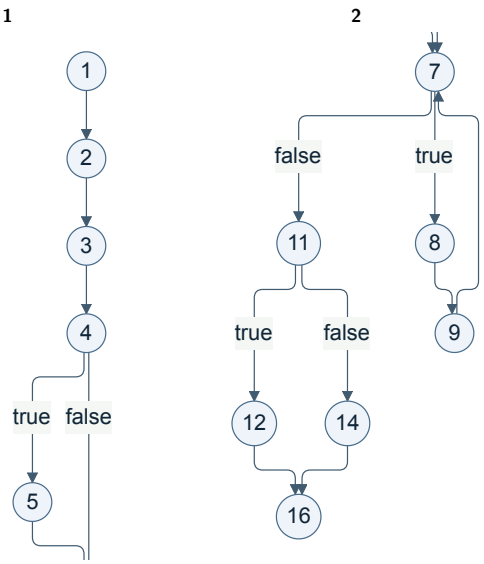


Figure 20: 12.9 Worked white-box example you should be able to reproduce — Tutorial 9 — follow numbered panels; crossing arrows continue.

Exact edge list: 1→2, 2→3, 3→4, 4→5, 4→7, 5→7, 7→8, 7→11, 8→9, 9→7, 11→12, 11→14, 12→16, 14→16.

N = 12, E = 14 ⇒ CC = E − N + 2 = 14 − 12 + 2 = 4. *(Cross-check by predicates: `if(n<0)`, the `while`, `if(result <= upperbound)` = 3 decisions ⇒ 3 + 1 = 4.)*

One feasible basis set for the combined-guard CFG (CC = 4):

#	Path	Covering input	Expected output
1	1,2,3,4,7,11,12,16	n = 0, ub = 100	prints "The sum is 0", returns 0
2	1,2,3,4,7,8,9,7,11,12,16	n = 1, ub = 100	prints "The sum is 1", returns 1
3	1,2,3,4,5,7,8,9,7,11,12,16	n = −1, ub = 100	prints "The sum is 1", returns 1
4	1,2,3,4,7,8,9,7,11,14,16	n = 1, ub = 0	prints "The sum is too large!", returns 1

Statement and branch coverage need only two tests, not four:

Input	Executed path	Expected print / return
`n = −1, upperbound = 100`	`1,2,3,4,5,7,8,9,7,11,12,16`	`The sum is 1` / `1`
`n = 5, upperbound = 1`	`1,2,3,4,7,8,9,7,8,9,7,11,14,16`	`The sum is too large!` / `3`

Together these cover all executable statements and both outcomes of each of the three combined decisions. Two is minimal because the final `if` takes only one outcome per call. Four paths are needed for the **illustrated basis set**, a stronger, different target. The method returns the sum **after** it has crossed the bound; it does not clamp the return value to `upperbound`.

Compound-condition convention: treating the whole `while` guard as node 7 gives **N = 12, E = 14, CC = 4**. Splitting short-circuit `&&` into nodes 7a (`i<n`) and 7b (`result<= upperbound`) gives **N = 13, E = 16, CC = 5**. In the split graph, 7a false skips 7b; 7a true reaches 7b; either false exits to 11. State the convention and make the formula match the graph.

The same two coverage tests above also exercise both outcomes of both operands in the split graph: the first ends with `i<n` false; the second ends with `i<n` true and `result<= upperbound` false. If a fifth basis path is requested for that graph, add `n = 1, upperbound = −1`: it skips the body because 7b is false immediately, prints `The sum is too large!` and returns 0.

Trace carefully: `n = 4` with a large bound executes four iterations. A single iteration can be obtained with `n = 1` and a non-negative bound; `n = 2` needs a bound that stops the loop after the first addition. A claimed basis path must either match its stated input exactly or be labelled schematic.

Chapter 13: Equivalence classes and boundary values

ELI5

Instead of trying every possible age, sort inputs into buckets that should behave alike. Then test the fence edges: just below, exactly on and just above. If you test two broken fields together, the first error may hide the second, so defensive tests usually break one parameter at a time.

Answer check: State units, representation and inclusive endpoints; partition valid/invalid behaviour, then select full boundary triplets. Combine valid inputs and isolate one invalid parameter per defensive row. Give an oracle. String length, syntax, registration and dependency failures are separate checks.

Visual: build tests in the right order

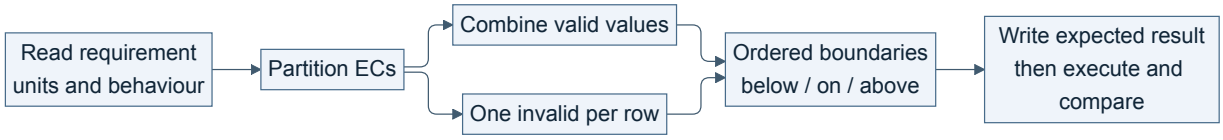


Figure 21: build tests in the right order

13.1 Two assumptions behind black-box testing

1. **A verifiable requirement / API specification exists** — make software testable from inception. *Not verifiable:* "The company must not hire too-young or too-old people." *Verifiable:* "0–15 and 55–99: do not hire; 16–17: part-time; 18–54: full-time."
2. **The code must be testable** — a 100-branch `if (age==0) ... if (age==99)` chain makes black-box testing ineffective compared with range conditions.

13.2 Equivalence classes

A set of input values forms an **equivalence class (EC)** if:

- they are all **supposed to produce the same output**;
- if one value catches a bug, the others **probably will too**;
- if one value does not catch a bug, the others **probably will not either**.

13.3 Partition rules

Input type	Valid ECs	Invalid ECs
Range of values ($1500 \leq x \leq 80000$)	**ONE** valid EC (or more, if the requirement has several valid ranges — e.g. hiring rules have 4)	**TWO** invalid ECs ($x \leq 1499$ and $x \geq 80001$)
Discrete values (applicant type $\in$ {Person}, property type $\in$ {HDB, Condo})	**ONE** valid EC (the listed set)	**ONE** invalid EC (everything else)

Worked partitions:

Parameter	Valid EC(s)	Invalid EC(s)
Mortgage income (1500–80000)	$1500 \leq x \leq 80000$	$x \leq 1499$; $x \geq 80001$
#mortgages (0–5)	$0 \leq x \leq 5$	$x \leq -1$; $x \geq 6$
Password length (4–8)	$4 \leq x \leq 8$	$x \leq 3$; $x \geq 9$
Password characters (a–z, A–Z, 0–9)	{a…z, A…Z, 0…9}	everything else (+, −, (,), …)
Applicant type (Person only)	{Person}	{Company, Trust, Partnership, …}
Property type (HDB, Condo)	{HDB, Condo}	{Townhouse, MobileHome, …}
Hiring age	0–15 (no hire) ; 16–17 (part) ; 18–54 (full) ; 55–99 (no hire)	age ≤ -1 ; age ≥ 100

Trap: counting the *hiring* rule as "one valid EC" is wrong — there are **four** valid ECs because the lecture partitions **four contiguous ranges**. The two "no hire" ranges share an output but have different boundaries; four ranges do not mean four distinct output values.

13.4 Designing test cases from ECs — the two golden rules

1. **Valid tests:** you **may combine** valid values of **several** parameters in one test case — choose one value from one EC of each parameter.
2. **Invalid tests: exactly ONE invalid value** per test case (one parameter, one EC); all other parameters take **valid** values.

Why only one invalid per case? So that the failure can

be attributed: you learn the error handling and behaviour for **each type of invalid input**. With four invalid values in one row, a rejection tells you nothing about which input caused it — and error handling may mask each other.

Small ECs: testing every member can be practical for small sets/ranges, but does not make each member a separate EC.

Testing by contract vs defensive testing

	Testing by contract	Defensive testing
Cases	**Only valid** inputs (satisfying pre-conditions)	**Both valid and invalid** inputs
Assumption	Users use the code as intended	Users will not; test that invalid input does not cause unforeseen errors or abnormal termination
Preferred by	—	**Testers** (if it breaks, the tester is blamed)

13.5 Boundary value testing (BVT)

Visual: see the mortgage-income partitions and their edges

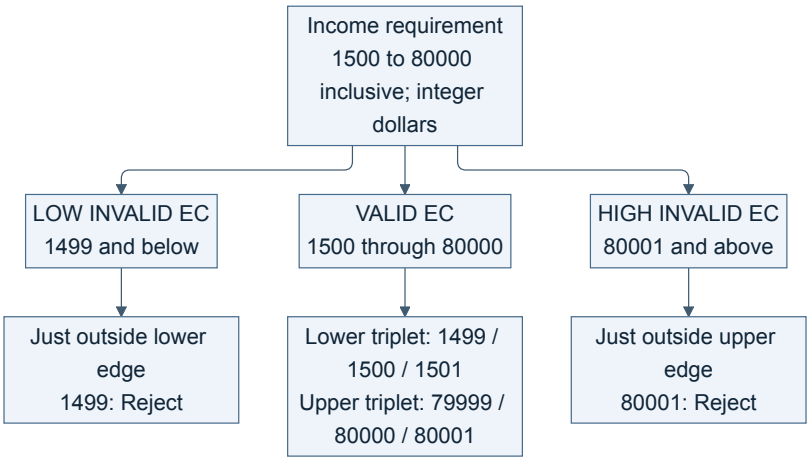


Figure 22: see the mortgage-income partitions and their edges

> For **each boundary of each range-of-values EC**, choose **three** values: one **on-the-boundary**, one **just-below**, one **just-above**. The size of the "just" step depends on the **unit** of the value.

In the lecture's terminology, range-of-values parameters include integer counts. "Discrete values" means **unordered categories** such as Person/Company and HDB/Condo, which have no numeric BVs. Integer quantities such as mortgage count **do** have BVs.

Worked BVs (full triplets, with the lecture's reduced valid/invalid selection):

Requirement	Lower boundary	Upper boundary	Valid BVs	Invalid BVs
Income 1500–80000	1499, **1500**, 1501	79999, **80000**, 80001	{1500, 80000}	{1499, 80001}
#mortgages 0–5	$-$**1**, 0, 1	4, **5**, 6	{0, 5}	{$-$1, 6}
Password length 4–8	3, **4**, 5	7, **8**, 9	{4, 8}	{3, 9}
Hiring age 0–15	$-$1, **0**, 1	14, **15**, 16	{0, 15} (see de-dup below)	{$-$1, 100}
Hiring age 16–17	15, **16**, 17	16, **17**, 18	...	...
Hiring age 18–54	17, **18**, 19	53, **54**, 55	...	...
Hiring age 55–99	54, **55**, 56	98, **99**, 100	...	...

Following the **course's reduced BV selection**, the hiring sets are: **valid BVs = {0, 15, 16, 17, 18, 54, 55, 99}**; **invalid BVs = {$-$1, 100}**.

The course's test-reduction rule (L18, pp. 11–15):

> Remove the just-below / just-above value from an EC's BVs (do **not** create a duplicate test case) if that value **falls into another EC** — including the *current* EC.

Concretely: a just-below/just-above value is used as an **invalid test value only if it lies outside all valid ECs**. If it lands in another valid EC, that EC's own on-the-boundary value already covers it. (For 0–15, 1 is omitted because it stays inside the same EC; 16 is already a boundary of the next EC.) This is **more than removing equal duplicate values**: the slides also omit distinct interior neighbours. State the full triplets when asked for BVs, then identify the reduced test selection. Keeping all distinct triplet values is a valid stronger test suite; do not claim those values are duplicates.

13.6 Designing test cases from BVs

Same two rules as ECs, with BVs supplying the values:

- **Valid test cases:** combination of **on-the-boundary (valid) values**, one from each EC, combined across parameters.

- **Invalid test cases: one** just-below/just-above (invalid) value per test case, with valid values for all other parameters.

For two range parameters A and B, the lecture uses these three test shapes:

1. **Valid A + Valid B** → combination of valid inputs.
2. **Valid A + Invalid B** → one invalid.
3. **Invalid A + Valid B** → one invalid.

13.7 Worked full example — Mortgage application

Requirements

- Income must be **$1500–$80000/month** (L17/L18 worked problem, not L16's separate $8000 example).
- Number of mortgages per applicant must be **0–5**.
- Only **Person** applicants are approved.
- Only **HDB** and **Condo** properties are approved.

ECs

Parameter	Valid EC(s)	Invalid EC(s)	BVs
Income	1500–80000	$\leq$1499 ; $\geq$80001	valid {1500, 80000}; invalid {1499, 80001}
#mortgage	0–5	≤ -1 ; ≥ 6	valid {0, 5}; invalid {−1, 6} (you may additionally enumerate 1–4)
Applicant type	{Person}	{Company, Trust, ...}	none (discrete)
Property type	{HDB, Condo}	{Townhouse, MobileHome, ...}	none (discrete)

Valid test cases (combining valid values; enumerating the small discrete/range ECs)

#	Income	#mortgage	Applicant	Property	Expected
1	$1500	0	Person	Condo	Approval
2	$2000	1	Person	Condo	Approval
3	$5000	2	Person	Condo	Approval
4	$12345	3	Person	HDB	Approval
5	$76543	4	Person	HDB	Approval
6	$80000	5	Person	HDB	Approval

Note how one row already covers the single valid EC of **every** parameter (the lecture's point), and the remaining rows exist only to enumerate the small ECs' values.

Invalid test cases (exactly one invalid per row, everything else valid)

#	Income	#mortgage	Applicant	Property	Expected
7	**$100**	1	Person	Condo	Reject
8	**$81000**	2	Person	HDB	Reject
9	$5000	**−1**	Person	Condo	Reject
10	$10000	**8**	Person	HDB	Reject
11	$12345	3	**Company**	HDB	Reject
12	$76543	4	Person	**Mobilehome**	Reject

Avoid: a row containing low income, eight mortgages, Company and Mobilehome simultaneously. Rejection cannot isolate which invalid class was exercised.

13.8 Summary table (put this in your head — it is a slide)

	Range of values	Discrete values
ECs	ONE or more valid EC; **TWO** invalid ECs	ONE valid EC; ONE invalid EC
BVs	For each EC: lower boundary → on / just-below / just-above; upper boundary → on / just-below / just-above	**NO boundary values**

Trap #1 (off-by-one). For $0 \leq x \leq 5$, BVs are **−1, 0, 1** and **4, 5, 6** — *not* 0, 0.5, 1 (that is only for continuous/real-valued inputs where the unit allows halves).

Endpoint check: inclusive endpoints are valid; classify each neighbour against the specification, not merely its position in a triplet.

Adjacent ECs: deduplicate shared boundary/neighbour values; do not repeat the same input for both classes.

Sensor check: distinguish out-of-range input from in-range-but-business-rejected readings (Section 13.9).

13.9 Worked example — GES silo (Tutorial 8)

Given: humidity 0–100%, normal 35–50%; temperature −10–100 °C, normal 40–60 °C. The question says a silo cannot accept grain if either reading "exceeds normal levels".

State the interpretation: the solution below permits acceptance only when **both readings are inside their inclusive normal ranges**. The wording does not explicitly resolve below-normal readings. Confirm that interpretation with the client; if "exceeds" means only above the upper bound, the below-normal oracle changes. Assume integer percentage points and whole degrees for ±1 BVs. Real-valued sensors require a specified resolution; −10 °C here is a stated input limit, not a universal physical minimum.

Parameter	Valid normal input: permit acceptance	Valid sensor input: block acceptance	Outside stated sensor range
Humidity	35–50	0–34; 51–100	≤ -1; ≥ 101
Temperature	40–60	−10–39; 61–100	≤ -11; ≥ 101

"Valid sensor input" and "eligible to accept grain" are different predicates. The three in-range ECs have specified business outcomes under the stated interpretation. Out-of-range input needs a validation policy; the tutorial does **not** specify an error code, message or recovery action.

Full boundary triplets for all in-range ECs:

Parameter	Lower/upper boundaries of its ECs	Triplets (on ±1)
Humidity	0, 34, 35, 50, 51, 100	(−1,0,1); (33,34,35); (34,35,36); (49,50,51); (50,51,52); (99,100,101)
Temperature	−10, 39, 40, 60, 61, 100	(−11,−10,−9); (38,39,40); (39,40,41); (59,60,61); (60,61,62); (99,100,101)

Following the lecture's reduced BV selection gives humidity {−1,0,34,35,50,51,100,101} and temperature {−11,−10,39, 40,60,61,100,101}. Interior neighbours may additionally be retained for stronger boundary checks.

#	Humidity	Temperature	Expected outcome under the stated interpretation
1	35	40	Permit acceptance: both lower normal boundaries
2	50	60	Permit acceptance: both upper normal boundaries
3	0	45	Block: humidity below normal, valid sensor input
4	34	45	Block: humidity just below normal
5	51	45	Block: humidity just above normal
6	100	45	Block: humidity above normal, at sensor limit
7	45	−10	Block: temperature below normal, at sensor limit
8	45	39	Block: temperature just below normal
9	45	61	Block: temperature just above normal
10	45	100	Block: temperature above normal, at sensor limit
11	−1	45	Out-of-range humidity; validation response TBD
12	101	45	Out-of-range humidity; validation response TBD
13	45	−11	Out-of-range temperature; validation response TBD
14	45	101	Out-of-range temperature; validation response TBD

Rows 3–10 isolate one abnormal reading; rows 11–14 isolate one invalid sensor input. Acceptance also depends on capacity/ type/grade in the complete GES, but Tutorial 8 isolates the sensor rule.

Chapter 14: GES: difficult constraints

ELI5

The Grain Elevator System is a practice world for turning one story into many consistent models. A proposed shipment is a plan; a recorded quantity is what actually arrived. Do not add a proposal to the stored amount before the arrival is measured and accepted.

Answer check: Keep proposed and actual grain quantities distinct. Check exact thresholds and started-block charges. A warning is not automatically input rejection. Do not invent sensor-failure, resolution or below-normal policies.

14.1 Domain facts

Fact / Value

Grain types: Wheat, barley, long grain rice, short grain rice, oats, hops

Grades: High and low for each grain type

Non-empty silo: Stores only one grain type and grade

Empty silo: May be assigned any accepted grain type and grade

Silos: 12: 1–6 hold 8 000 bushels each; 7–12 hold 12 000 each

Total capacity (derived): $6 \times 8\,000 + 6 \times 12\,000 = \mathbf{120\,000}$ **bushels**

Truck load: 150–180 bushels

Rail car capacity: 2 000 bushels

Arrival: Trucks from growers/sellers only; plate number, driver and seller recorded

Departure: Rail cars to processing plants/buyers only; serial number, conductor and buyer recorded

Monitoring: Humidity and temperature sensors send readings periodically

Abnormal alert: **Sensors** alert both GES and an external grain dryer; affected silo cannot accept more grain until both readings return to normal

The supplement supplies no numeric normal ranges; **Tutorial 8** supplies those thresholds.

14.2 Hard modelling checks

- **Proposal versus fact:** allocation is a proposed amount per arrival/silo pair. Stock and shipment logs change only after the manager reports actual deposits, whether accepting or overruling the proposal. Partial acceptance is allowed; no-capacity and zero-quantity logging need an agreed policy.
- **Overrule is not a bypass:** capacity, compatible grain type/grade and permitted environmental conditions still constrain the chosen silo. Log quantity, type/grade, time, manager, direction, vehicle and the relevant people/organisations.
- **Reports:** distinguish capacity remaining for the currently stored type from uncommitted capacity. A pending delivery is not inventory already stored.

14.3 Boundary and use-case traps

GES means tracking software: Manager and directly interacting sensors are actors; Seller/Buyer/Driver/Conductor are recorded entities. Sensors alert the external dryer directly. Arrival includes allocation; optional overrule extends the arrival behaviour. Actual-deposit reporting is required in both paths. Routine use-case descriptions follow Chapter 3's template.

14.4 Class-model traps

An arrival can use several silos; each silo receives many arrivals over time. An Allocation class carries the proposed amount for that pair. Keep actual deposits distinct. An Arrival uses a Truck; a Departure uses a RailCar. Do not attach both vehicle types to every shipment or assume all business roles inherit Person.

14.5 Silo condition state machine (requirement E)

Model **environmental condition separately from empty/non-empty/full**. An empty silo can be abnormal; unloading does not itself restore normal conditions. Sensors, rather than GES, send the dryer alert.

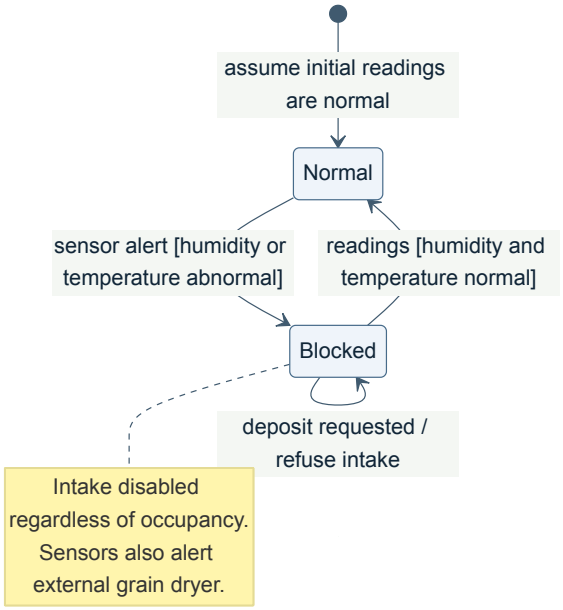


Figure 23: 14.5 Silo condition state machine (requirement E)

The initial normal reading is a stated model assumption. Unknown initial readings and sensor-failure policy remain TBD. "Intake disabled" is a constraint; `refuse intake` is an event response, not a continuously running `do/` activity. Returning to Normal removes the sensor block; capacity and type/grade constraints still apply.

14.6 Hard arrival invariant

Plan, then record reality: offer load; compute eligible silo proposal; accept/overrule; unload; report actual amounts and shipment details; update stock/log; acknowledge. Routine class/use-case/sequence drawings are omitted here; the invariants above are the part that cannot be copied blindly from a generic model.

Chapter 15: Hard tutorial questions

ELI5

The hard part is keeping two independent changes separate: which plants want an event, and how each plant receives it. A subscription list chooses who; a channel strategy chooses how.

Other hard exercises, answered once: Tutorial 8's normal/sensor-range ECs and ambiguous below-normal policy are in Section 13.9. Tutorial 9's compound conditions, feasible traces and two-test coverage suite are in Section 12.9. The routine projector and basic tutorial walkthroughs are omitted.

Hard tutorial: selective Observer + channel Strategy

Q1. GES notifies processing plants on grain arrival/departure; plants register **interests in specific grain types**; notification via preferred channel **email, SMS, or both**; interests and channels **vary over time**; must be **easily extended** to new channels.

- **Design problem:** one-to-many notification with runtime-variable subscribers and *two* varying dimensions (interest + channel).
- **Patterns:** **Observer** (subscription + notification: `attach/detach/notify`, `update` with interests → **selective notification**) + **Strategy** for the communication means (`EmailNotifier`, `SMSNotifier`, `Composite Notifier` for "both") + **an optional Factory** to create the notifier for a plant's current preference.
- **Trap:** "combination of these means" ⇒ the **Composite** pattern is also defensible for email+SMS; say so and justify.

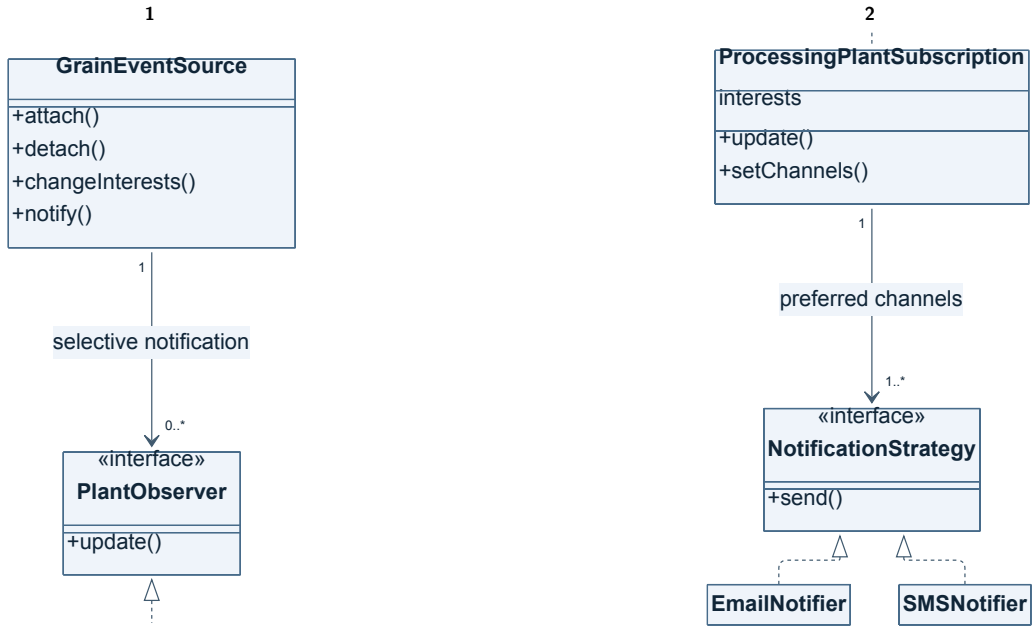


Figure 24: Hard tutorial: selective Observer + channel Strategy — follow numbered panels; crossing arrows continue.

Channel check: interests and delivery channels change independently. Send through each selected channel so "both" is supported; channel creation may use an optional Factory.

Chapter 16: AI engineering and agentic systems

ELI5

Treat an LLM like a fluent assistant who can still misunderstand the task. It can suggest code, tests or tags, but another part of the system must check the result. Asking the assistant to promise 'I am correct' is not the same as measuring correctness.

Answer check: Separate LLM proposals from deterministic validators, policy gates and execution. Valid JSON can still have a wrong shape, disallowed tags or too many tags. An LLM-selected role does not authenticate a user; agent agreement does not prove passing tests.

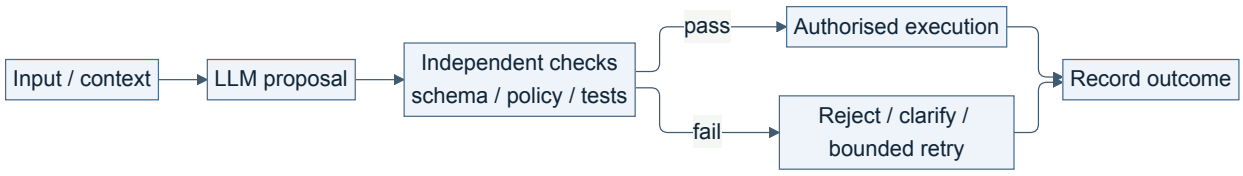


Figure 25: LLM outputs need independent checks; generation is not verification

16.1 AI engineering vocabulary

Term / Definition

AI engineering: The discipline of **building, deploying and maintaining AI-powered systems** in the real world; it fuses AI research (what is possible), software engineering (what works reliably) and domain expertise (what matters)

AI vs ML vs DL: AI = broad science of mimicking human abilities; ML = algorithms that improve through experience; DL = multi-layer neural networks for complex patterns

AI engineering vs ML engineering: The guest decks contrast traditional model development/training with applications built on foundation models; treat this as a focus distinction, not an exclusive definition

LLM pipeline: Tokenisation → embeddings → attention → next-token prediction → (training adjusted the weights)

RAG: Retrieval-augmented generation: fetch relevant documents,

then generate grounded on them

Agent: Plans and executes a **sequence of actions** to achieve a goal — plans across steps, keeps state/memory, uses tools, loops until done (vs a stateless, one-shot chatbot)

ReAct loop: THINK → ACT → OBSERVE → THINK ... with tools, memory, and a **condition** for termination

MCP: Model Context Protocol — standard way to expose tools/ context to models

Agent risks: infinite loops · **indirect prompt injection** · **irreversible actions**

16.2 AI failure modes worth remembering

Failure / Description

Hallucination: Confidently stating factually wrong things (e.g. non-existent case citations)

Bias: Amplifying historical discrimination (e.g. a hiring model downgrading résumés)

Prompt injection: Malicious instructions hidden in content the model reads (direct vs **indirect**)

Over-reliance: Automation bias — humans stop checking the output

Evaluation challenges: natural complexity · open-ended tasks with multiple valid answers · black-box nature · benchmark saturation · emerging capabilities.

16.3 TDD vs BDD vs SDD (the one comparison worth knowing)

Question it answers

TDD (test-driven development): *How do you verify **one behaviour**?* Write a test first, then code to make it pass

BDD (behaviour-driven development): *How do **users/business** expect it to act?* Write scenarios in business language, implement against them

SDD (spec-driven development): *What should the **system** do?* Write the spec first, implement to match it

The SDD loop: Discuss → Specify → Delegate → Evaluate, with a **human review checkpoint** for every submitted change. Anti-pattern: *vibe coding* — vague request → plausible code → hope ("fragile & ungrounded").

Mapping back onto SC2006 (the useful part):

SDD phase / SC2006 equivalent

Discuss: Requirements **elicitation** (interview/workshop, learn the problem domain, name what is out of scope)

Specify: **SRS** + use case descriptions + data dictionary (the "written spec that survives the session")

Delegate: Implementation (object design → code)

Evaluate: **Validation & verification**, acceptance testing, UAT

16.4 AI engineering practice: adapting, evaluating and operating systems

The practice deck develops three layers: **application development** (interface, prompts, context, evaluation), **model development** (adaptation/training, datasets, inference optimisation, evaluation) and **infrastructure** (compute/data management, serving, monitoring).

- **Prompt engineering:** specify task, instructions, examples and context. Fast to change; it does not update the model's weights or make unsupported output automatically correct.
- **RAG:** retrieve relevant external documents and supply them as generation context. Useful for private or changing knowledge; retrieval quality, latency and document maintenance still matter.
- **Fine-tuning:** adapt model weights using suitable labelled examples for a narrow task or desired behaviour. It needs data and training effort; distinguish it from adding context at inference time.
- **Evaluation:** use representative tests and quantitative/ qualitative review throughout development, including component tests and end-to-end behaviour. The Understanding deck also covers fairness, privacy, hallucination, prompt injection and monitoring after deployment.
- **Operations:** the practice deck includes REST APIs, containerised dependencies, Kubernetes for orchestration, and CI/CD/continuous training. Architecture combines the application, model, retrieval/data and serving layers; no single tool guarantees quality.

The agentic-development deck adds **automated**, **observed** and **human** evaluation evidence: unit/integration tests, type checks/lint/builds; runtime/UI behaviour and logs; requirements traceability, diff review, security/privacy/ maintainability. It advocates short feedback loops, enforced checks and searchable structured logs. Passing those checks establishes only what the checks cover, not absence of bugs.

16.5 Architecture extension in the Thursday system-design deck

`Lecture_12_System_Design_2026_Thursday.pdf`, pp. 8–14, discusses intelligence, a three-level capability system (**foundational, integrative, applied**), LLM agent architecture and brain-inspired agent architecture. These are exploratory architecture examples, not new GES requirements or additional GoF patterns. Their assessment status is not stated in the files.

Chapter 17: Examination answer checklist

ELI5

An open-book reference is a labelled toolbox, not a substitute for knowing which tool a question needs. Read the command word, find the right template, then fill it using this case's facts. Every answer should make the examiner's checking job easy.

Final answer checks

1. Use this paper's names, units, role permissions and constraints consistently across all four questions.
2. Diagram: external actors; goal names; correct arrows; multiplicities at both association ends; guarded decisions; bars for concurrency.
3. Design: justify each component/pattern against a case requirement; include a trade-off, not only a name.
4. Tests: valid baseline, exactly one invalid field per defensive row, exact boundary equality and an independently justified expected result.

Pattern answer template: Problem [what must vary]; pattern [name and mechanism]; roles [case classes]; consequence [benefit and cost].

Command-word decoder for exam prompts

Prompt says / ***They want***

"Atomise": Numbered, single-assertion, verifiable statements

"Identify the design problem": The *smell* + the *force*, before any pattern name

"Apply the appropriate pattern": Name + class diagram + role table

"Systematically design test cases": A **table** with one row per test case and an oracle column

"Discuss": Two sides + a verdict + a trade-off

"With justification": One sentence of *why* per decision

"Draw": Correct **notation**, not prose

Source index

Lecture citations (L1–L18), tutorial numbers and section references throughout identify the evidence. All source PDFs are in the adjacent `lectures/`, `tutorial/` and `pyps/` folders. The editable study guide is `../SC2006_Study_Guide.md`.

- **Chapters 1–6:** Introduction; Requirements Elicitation 1–3; Conceptual Models; Dynamic Models; Writing SRS; UMLSpecNotes.
- **Chapters 7–8:** Software Processes – Basics; Agile Software Development – Basics, including its image slides.
- **Chapters 9–11:** System Design (both decks), Object Design (both decks), Design Patterns, MVC (both versions). The file named `Lecture_13_System_Design_2026.pdf` opens as Object Design.
- **Chapters 12–13:** Test Fundamentals/Testing Introduction; Equivalence Class Testing; Boundary Value Testing (2026 and other supplied versions).
- **Chapters 14–15:** Tutorial Questions and the scanned GES supplement. Tutorial-only examples remain in Chapter 15; shared answers appear once in their topic chapter.
- **Chapter 16:** Understanding AI Engineering; AI Engineering as a practice – NTU; Agentic Software Engineering; Thursday System Design extension. These guest topics have no separately confirmed assessment status in the supplied files.

Source conflicts: use hollow-triangle UML generalization; keep the course's historical Scrum wording distinct from current-standard claims; distinguish graph bases from feasible traces; use the problem's stated mortgage upper limit (8000 versus 80000); 800001 in the separate summary is a typo for 80001. Full BV triplets and reduced course selections are labelled separately. Local GES and paper-specific assumptions remain beside the relevant answer.

Past-year papers and worked answers

Paper format observed: two hours, four compulsory 25-mark questions. Budget about 30 minutes per question, including reading/checking. The linked originals supply exact wording and Appendix A; prompts here are summaries. Worked designs are examples, not unique solutions.

Paper	Scenario	Main modelling variation
AY2022/23 S1	Car sales	Hold-car use case, conceptual classes, dialog map
AY2022/23 S2	Shopping chatbot	Replacement use case, conceptual classes, activity diagram
AY2023/24 S1	DevSecOps	Security-check use case, classes, activity diagram
AY2023/24 S2	CodeAgent	Reviewer use case, classes, QA-Checker state machine
AY2024/25 S1	Task management	Assignment use case, classes, task lifecycle activity diagram
AY2024/25 S2	Disaster Warning System	Stakeholders/requirements, sensor state machine, linking activity
AY2025/26 S1	Smart home	Guest access use case, controller state machine, voice activity

Continued on next page

Paper	Scenario	Main modelling variation
AY2025/26 S2	Restaurant recommendations	JSON-check activity, recommendation classes, conversation state machine

Common testing conventions: reset/isolate rows; keep fixtures and other inputs valid; combine valid coverage but use exactly one invalid field per defensive row. Accept means input validation passes, not that later external operations succeed. Reject means controlled failure without data mutation/crash. Oracles come from requirements; unclear behaviour is labelled TBD. Full triplets are retained; categorical membership has no numeric boundary.

Answer map: Q1 uses Chapters 2–3; modelling uses 4–5; process/Scrum uses 7–8; architecture/patterns uses 9–11; testing uses 12–13; AI checks use 16. Apply the method to the case rather than copying an earlier case's classes or limits.

Drawing convention: Mermaid approximates UML use-case/activity notation. In an examination, use UML actors, boundaries, ellipses, arrowheads, initial/final nodes, bars and guarded diamonds as explained in Chapters 3–5.

Older papers: grouped question bank

The four older papers (AY2022/23 and AY2023/24) are grouped by question type. Each case retains its own prompt, diagrams, constraints and testing rows. The four newer papers follow in chronological order.

Older case	Q1	Q2	Q3	Q4
Car sales	p. 34	p. 38	p. 42	p. 45
Shopping chatbot	p. 35	p. 39	p. 43	p. 46
DevSecOps	p. 36	p. 40	p. 44	p. 47
CodeAgent	p. 37	p. 41	p. 44	p. 48

AY2022/23 S1: Car sales

Source: Question paper, Q1–Q4 on PDF pp. 1–3; Appendix A on p. 4.

Case facts. Visitors browse/search new and used cars. A registered, logged-in user can hold one car by paying a deposit through the dealer's payment system. A car has only one holder at a time; another user cannot hold or buy it. Cancellation/expiry causes a refund. A buyer completes purchase with dealer staff. Staff log in, view car information, manage orders and complete delivery only after full-payment confirmation. A sales manager additionally manages car listings and generates sales reports.

AY2022/23 S2: Shopping chatbot

Source: Question paper, Q1–Q4 on PDF pp. 1–4; Appendix A on p. 5.

Case facts. The chatbot answers product/pricing/availability queries and helps logged-in customers manage carts, orders, payment/delivery details, status, replacements and refunds. Free replacement/refund is available within seven days of delivery, with item photos, purchase receipt and reasons. The company confirms or rejects the request; the chatbot communicates instructions or rejection.

AY2023/24 S1: DevSecOps

Source: Question paper, Q1–Q4 on PDF pp. 1–3; Appendix A on p. 4.

Case facts. A developer supplies a local project or GitHub link. The platform analyses security and quality using four external toolsets: SCA, SAST, DAST and IAST. It generates a report and offers mitigation recommendations. Project-selection help is optional. Developers may rerun selected toolsets to verify fixes and generate reports on request.

AY2023/24 S2: CodeAgent

Source: Question paper, Q1–Q4 on PDF pp. 1–4; Appendix A on p. 5.

Case facts. Seven internal agents are User, CEO, CPO, CTO, Reviewer, Coder and QA-Checker. The first six correspond to user roles; users log in to interact with the corresponding agents. Code review recognises input modality/language; Reviewer checks commit-message consistency, vulnerability injection and formatting; Coder revises code; Coder/CTO/CEO summarise the work. Paired agents converse under QA-Checker supervision until the answer is appropriate or a dialogue limit is reached. Logs/reports are generated on request.

Older Q1: Use-case models and descriptions

Shared method: use the boundary/relationship and description checks in Chapter 3, p. 5. Each case retains its own actors, flow and failure rules.

AY2022/23 S1: Car sales — Q1 (25 marks)

(a) Identify actors/use cases and draw their relationships (17). (b) Describe holding a car: actors, preconditions, flow and alternatives (8).

(a) Actors: Visitor, Registered Customer, Dealer Staff, Sales Manager and external Dealer Payment System. Registered Customer can do the visitor's browsing; Sales Manager inherits staff capabilities. The system itself, its database and a car are not actors.

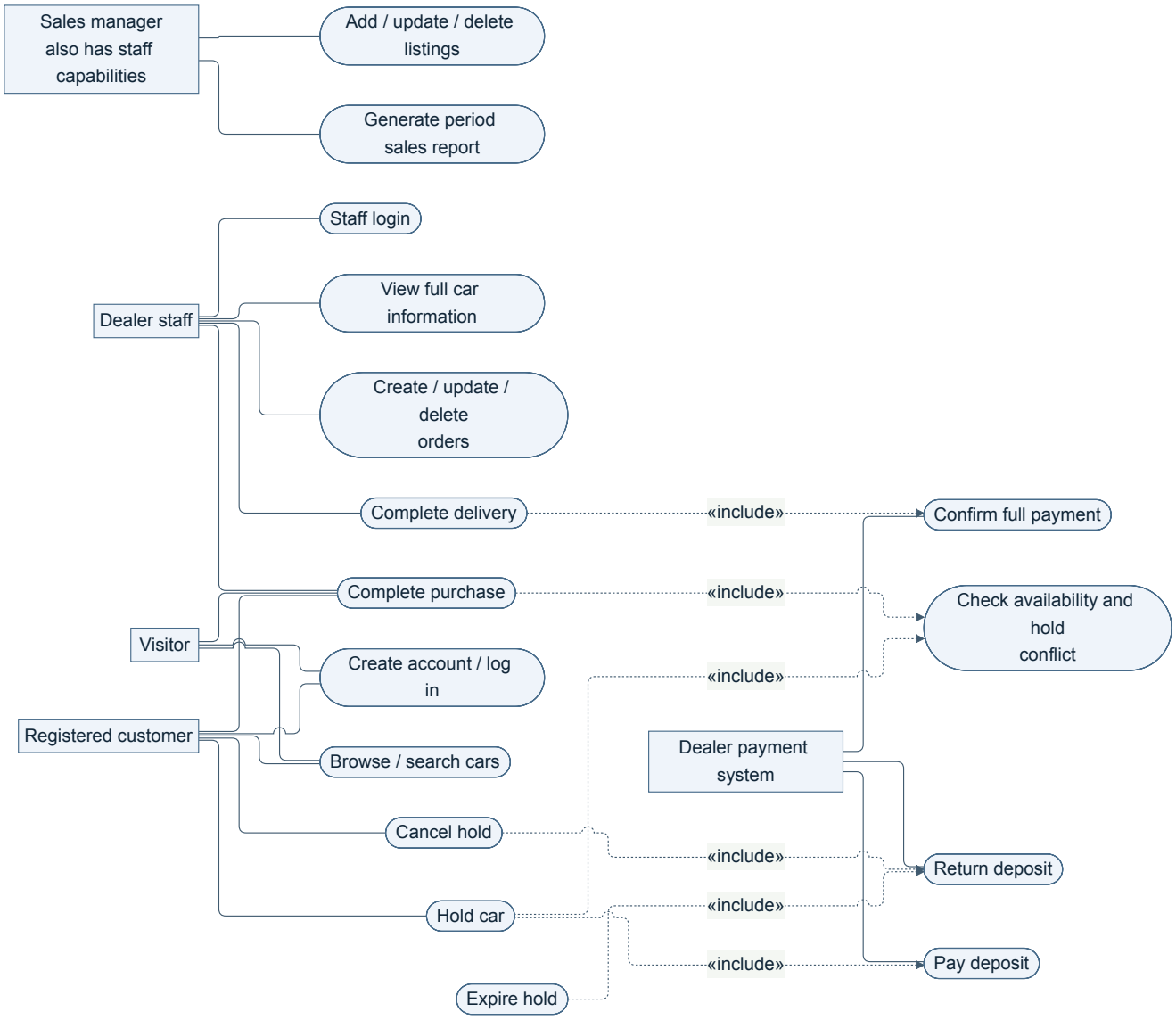


Figure 26: Car sales: Q1 — Use cases (25 marks)

Generalization: Customer to Visitor; Sales Manager to Staff. Expiry is an internally scheduled system action; a timer need not be invented as an external actor. The appendix does not explicitly require an account/deposit for a direct purchase with staff; those requirements apply to holding a car. A failed deposit is an alternative flow. The question says use relationships **where appropriate**, so an artificial extension is unnecessary.

(b) Use case: Hold Car. Primary actor: Registered Customer. Supporting actor: Dealer Payment System. Preconditions: customer is authenticated; selected car exists, is unsold and is not already under an active hold. Normal flow:

1. Customer selects a car and requests a hold.
2. System checks current availability and displays deposit/hold terms. The duration and deposit amount come from dealer policy; the paper gives no numbers.
3. Customer confirms and authorises the deposit through the payment system.
4. Payment system confirms success. System creates the hold for that customer and marks the car unavailable to other customers until expiry.
5. System displays the hold reference and expiry time.

Alternatives: failed/cancelled payment creates no active hold; a concurrent hold/purchase causes rejection rather than double allocation; expiry or user cancellation ends the hold and requests a refund. A payment confirmed after an allocation conflict must be compensated/refunded. **Design assumption:** availability checks and allocation are made atomic, with a pending reservation/payment reconciliation mechanism. Do not imply that an initial availability check alone prevents races.

AY2022/23 S2: Shopping chatbot — Q1 (25 marks)

(a) Actors/use cases and relationships (17). (b) Item-replacement description with actors, preconditions, normal/alternative flows (8).

(a) The chatbot is the system being modelled, not an external actor. Customer is the primary actor. Existing Web-Based Shopping System is a supporting actor outside the chatbot boundary; company approval is supplied through that system. If company staff directly operate a chatbot approval interface, they become another actor, but that interface is not specified.

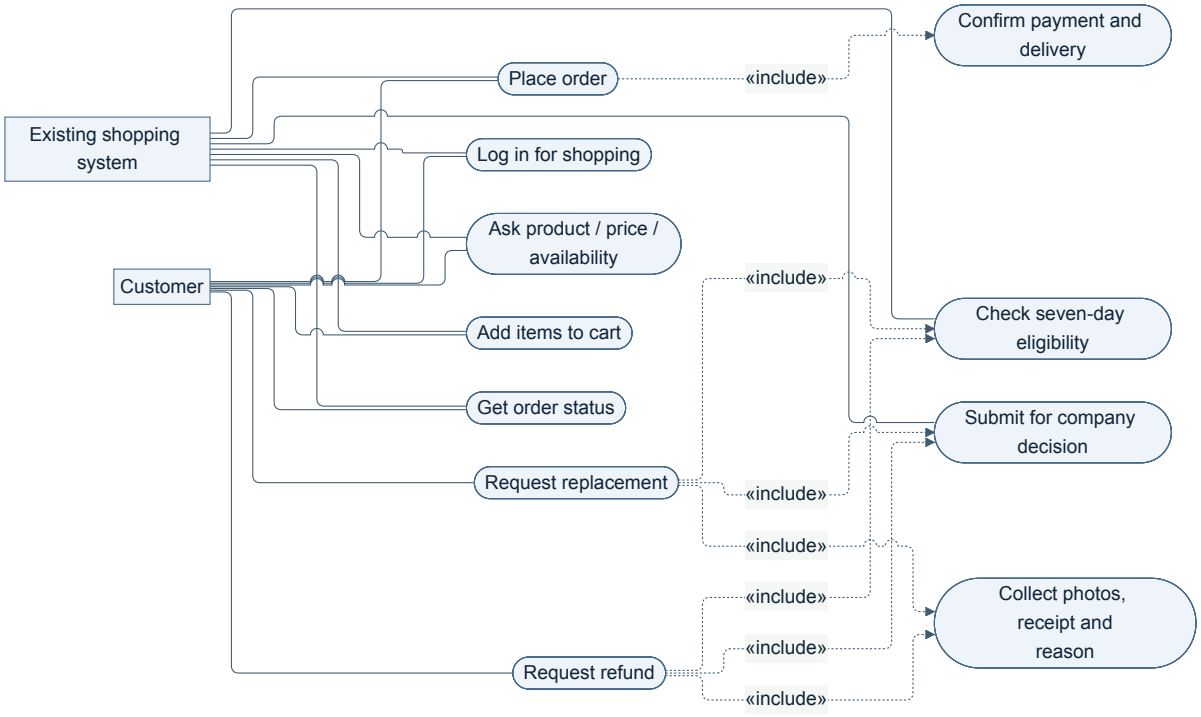


Figure 27: Shopping chatbot: Q1 — Use cases (25 marks)

Approval leads to replacement/refund instructions; rejection is an alternative flow. Shopping actions require a valid authenticated session, while product enquiries can precede login.

(b) Use case: Request Item Replacement. Actors: Customer, Existing Shopping System (including company decision service). Preconditions: customer is logged in; the delivered purchase can be identified. Normal flow:

1. Customer asks for replacement and selects the purchased item/order.
2. Chatbot retrieves purchase/delivery details and checks the seven-day rule.
3. Chatbot asks for item photos, receipt and replacement reason; customer supplies them.
4. Chatbot validates completeness and submits the request to the existing system for company confirmation.
5. On approval, chatbot sends the company-provided replacement procedure and request reference.

Alternatives: outside the eligibility period, reject the free replacement; missing evidence, request correction; company rejects, communicate rejection; integration failure, report pending/failure without falsely confirming approval. **Assumption:** "within seven days" includes the seventh day; the exact timestamp/timezone policy needs agreement. Company review may be asynchronous, so "submitted/pending" is distinct from "approved".

AY2023/24 S1: DevSecOps — Q1 (25 marks)

(a) Actors, use cases and relationships (17). (b) Describe the security-check functionality (8).

(a) Primary actor: Developer. Supporting actors: external SCA/SAST/DAST/IAST services and GitHub for remote project retrieval. The platform's own scanner coordinator and report store are internal components, not actors.

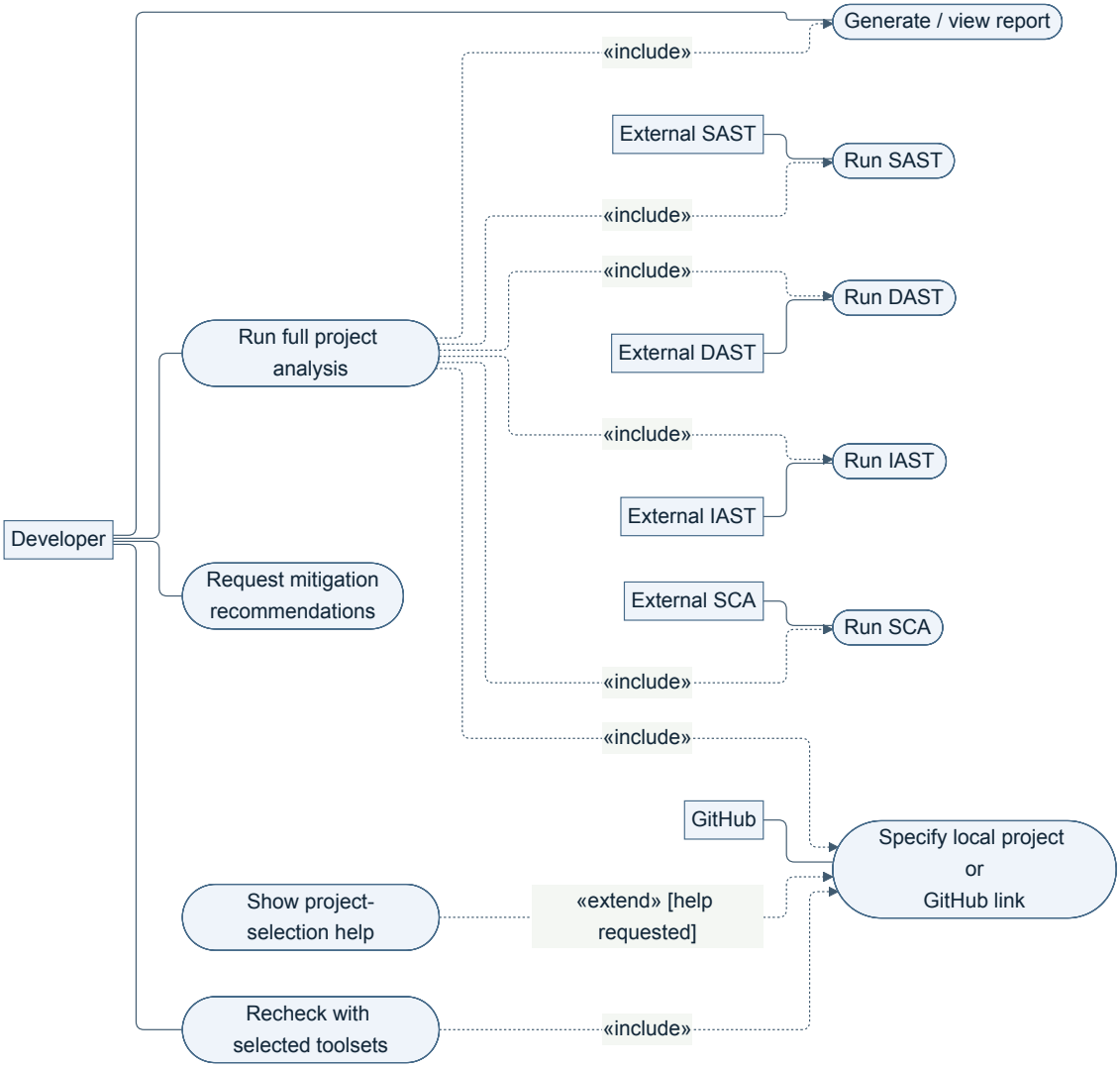


Figure 28: DevSecOps: Q1 — Use cases (25 marks)

Selected reruns invoke only chosen engines; the diagram's four mandatory includes apply to the **full-analysis** use case. GitHub participates only in the remote-source alternative. Mitigation is available after a report; it need not be automatically included in every scan. Selection help is a genuine optional extension.

(b) Use case: Check Project Security. Actors: Developer; selected external analysis toolsets; GitHub when applicable. Preconditions: project is accessible; required engines are available/configured. Normal flow:

1. Developer specifies local source or a GitHub link; platform obtains the project.
2. Platform runs all four toolsets for an initial full analysis, or the developer selects toolsets for a verification rerun.
3. Each selected toolset returns its findings; platform associates results with the project/version and combines them.
4. Platform generates and displays a security/quality report.
5. On request, platform supplies mitigation recommendations.

Alternatives: developer requests selection help; inaccessible source produces a corrective message; unsupported/failed/timed-out engine is reported with explicit incomplete status, not a clean result; developer fixes issues and reruns selected checks. Version tagging and incomplete-result status are proposed design details needed for trustworthy results. The paper does not specify a login requirement, so do not invent one as a case fact.

AY2023/24 S2: CodeAgent — Q1 (25 marks)

(a) Actors/use cases/relationships (19). (b) Reviewer-agent use case description (6).

(a) With the **whole CodeAgent platform** as the system boundary, agents are internal software, not external human actors. Model the six corresponding external user roles and their authenticated interactions. Appendix wording is unusual; this boundary/role interpretation is explicit rather than silently treating every program as an actor.

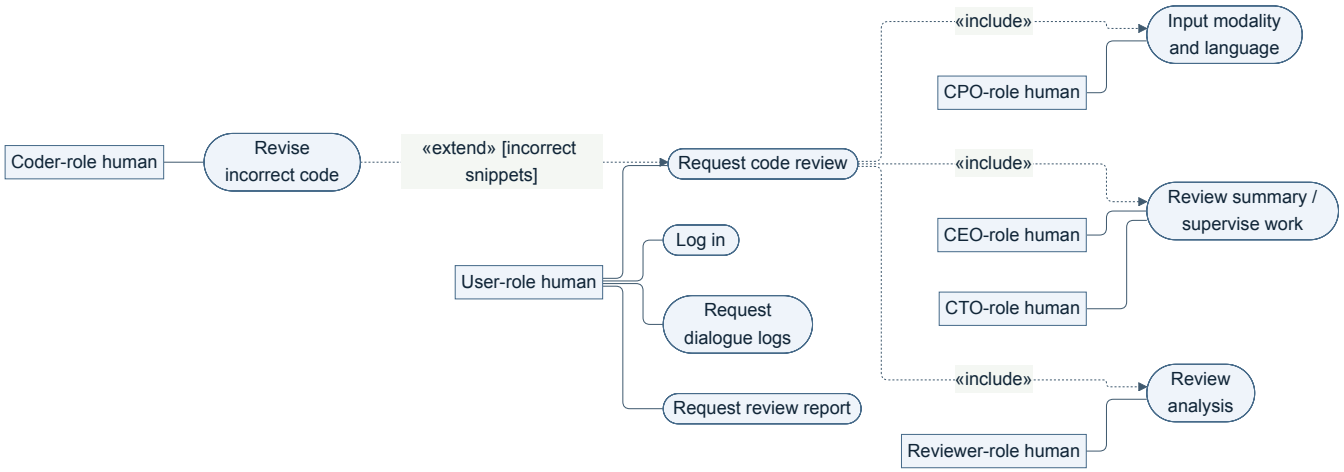


Figure 29: CodeAgent: Q1 — Use cases (25 marks)

Detail view of the same **Perform review analysis** use case:

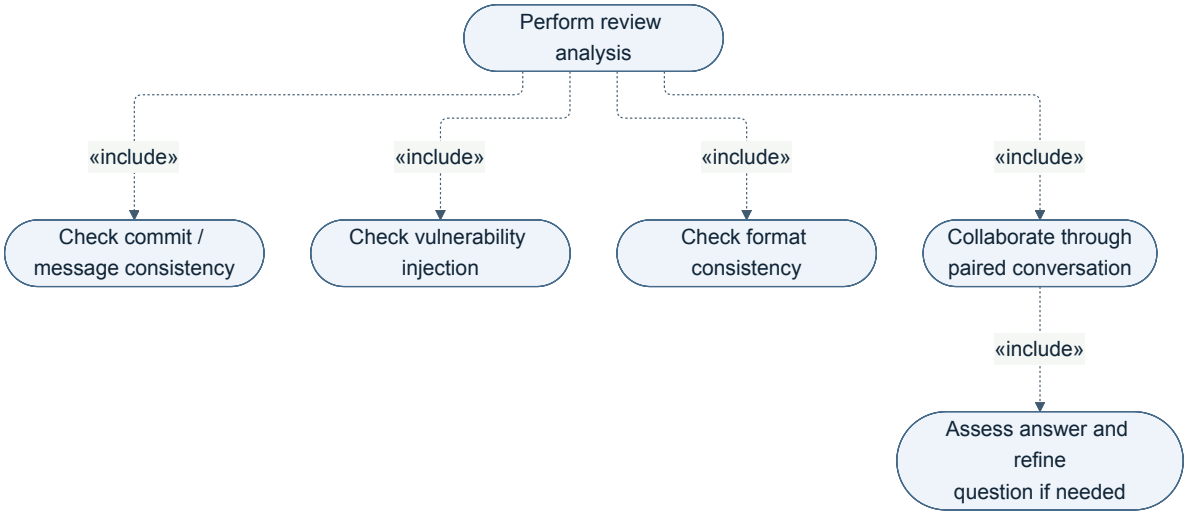


Figure 30: CodeAgent: Q1 — Use cases (25 marks)

All six role users can log in, interact with their corresponding agents, and request permitted outputs; show generalization to a common Authenticated User actor in exact UML rather than implying that only User-role humans log in. Internal CEO/CPO/Coder collaboration and Coder/CTO/CEO summarisation occur through agent orchestration; external users need not manually perform every subtask. QA-Checker needs no extra human actor.

(b) Use case: Review Submitted Code. Primary actor: authenticated requesting user/Reviewer-role user, according to the interface boundary chosen above. Preconditions: commit, commit message and original files are available; prior input recognition identifies modality/language. Flow: submit review request; Reviewer obtains code/context with Coder's help; analyse commit-message consistency, introduced vulnerabilities and formatting; conduct supervised conversations where needed; create an analysis report with observations and suggested revisions; provide it to Coder/user. Alternatives: missing/unreadable inputs, ask for correction; inappropriate answers, QA-Checker appends instructions and retries within the limit; limit exhausted, report unresolved/incomplete analysis rather than claiming success. Coder's subsequent revision is conditional on finding incorrect snippets.

Older Q2: Conceptual models, behaviour and process

Shared method: apply Chapters 4–5 to the named object/workflow; use W, p. 15 and S, p. 18 for process questions.

AY2022/23 S1: Car sales — Q2 (25 marks)

(a) Hold-car conceptual classes with boundary/control/entity stereotypes (8). (b) Dialog map for holding a car (8). (c) Can customer involvement fix Waterfall's response to changing requirements? (4). (d) Estimate Scrum progress and timely completion (5).

(a) The external payment system is represented by a payment boundary, not an entity.

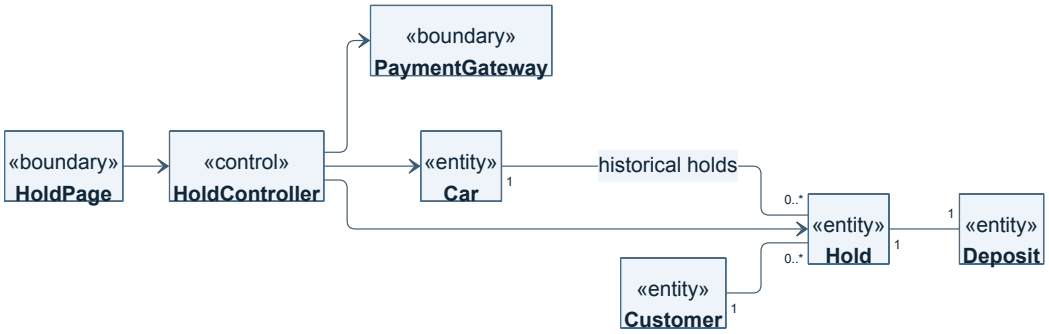


Figure 31: Car sales: Q2 — Analysis and process (25 marks)

Constraint: each Car has **at most one active Hold**, even though historical holds are many. Customer may have multiple holds on different cars; Appendix A does not prohibit that. Attributes/operations are deliberately omitted for this analysis question.

(b) The hold dialog models screens, not physical car states.

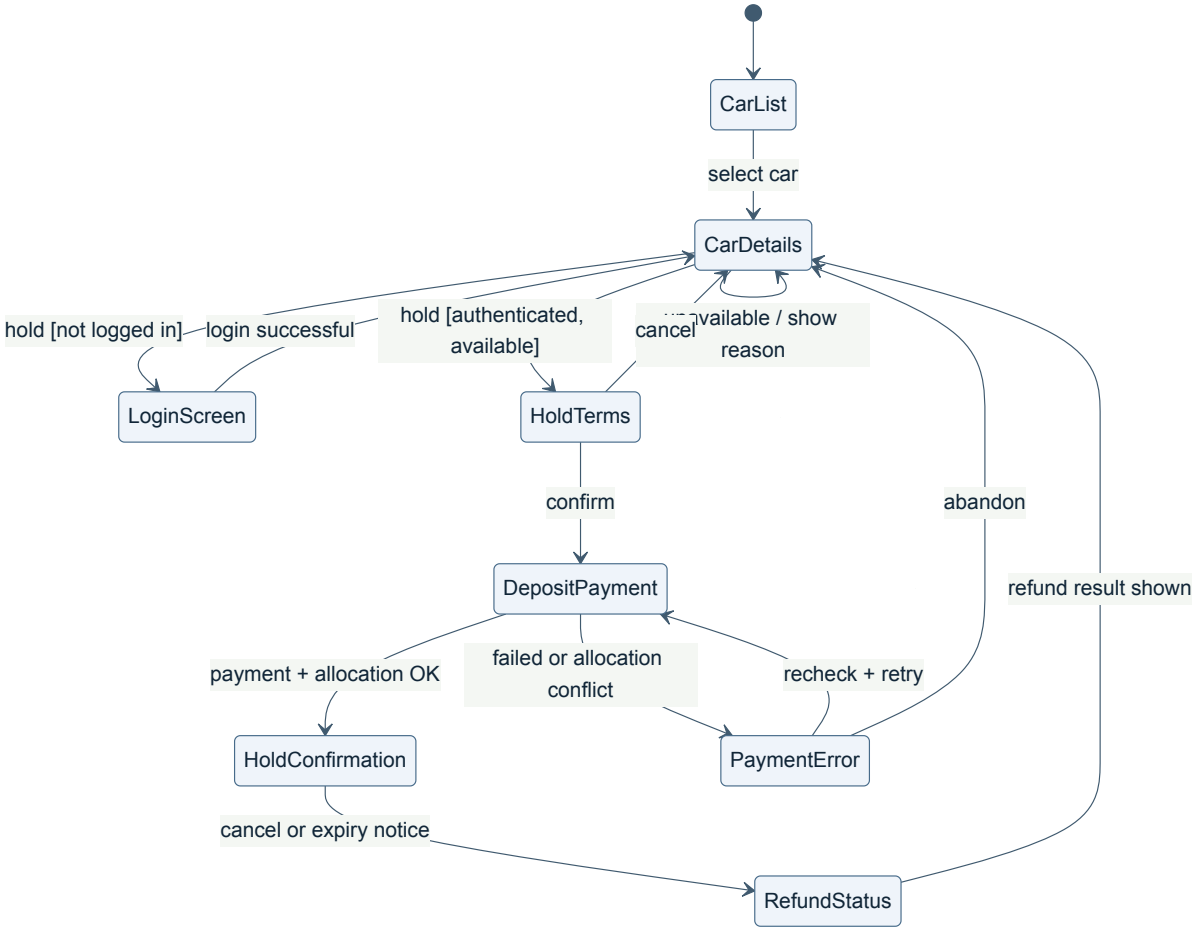


Figure 32: Car sales: Q2 — Analysis and process (25 marks)

Terms/error/refund screens are proposed interface details. The business invariants from Q1 still apply.

(c) Use W, p. 15: customer reviews expose change early; agreed downstream impact/replanning is still necessary.

(d) Use S, p. 18: track remaining effort/velocity with a burndown; forecast capacity and negotiate priorities as scope changes.

AY2022/23 S2: Shopping chatbot — Q2 (25 marks)

(a) Replacement conceptual classes with stereotypes (7). (b) Replacement activity with user/chatbot/existing-system participants (8). (c) Why reuse-based integration/configuration can force requirement compromises, and mitigation (4). (d) Scrum Master response to a bug threatening sprint completion (6).

(a)

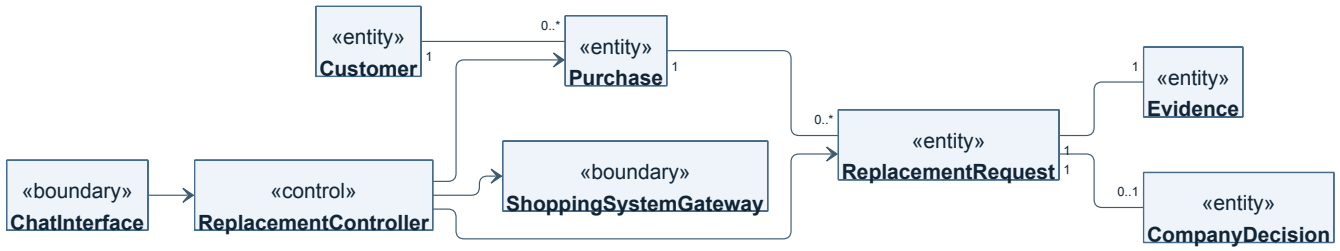


Figure 33: Shopping chatbot: Q2 — Analysis and development process (25 marks)

Decision is absent while pending. Evidence groups photos, receipt and reason; its presence does not imply that an incomplete submission is approved.

(b) Subgraphs approximate activity swimlanes. Company approval is an activity of the existing system's organisation, not an autonomous chatbot decision.

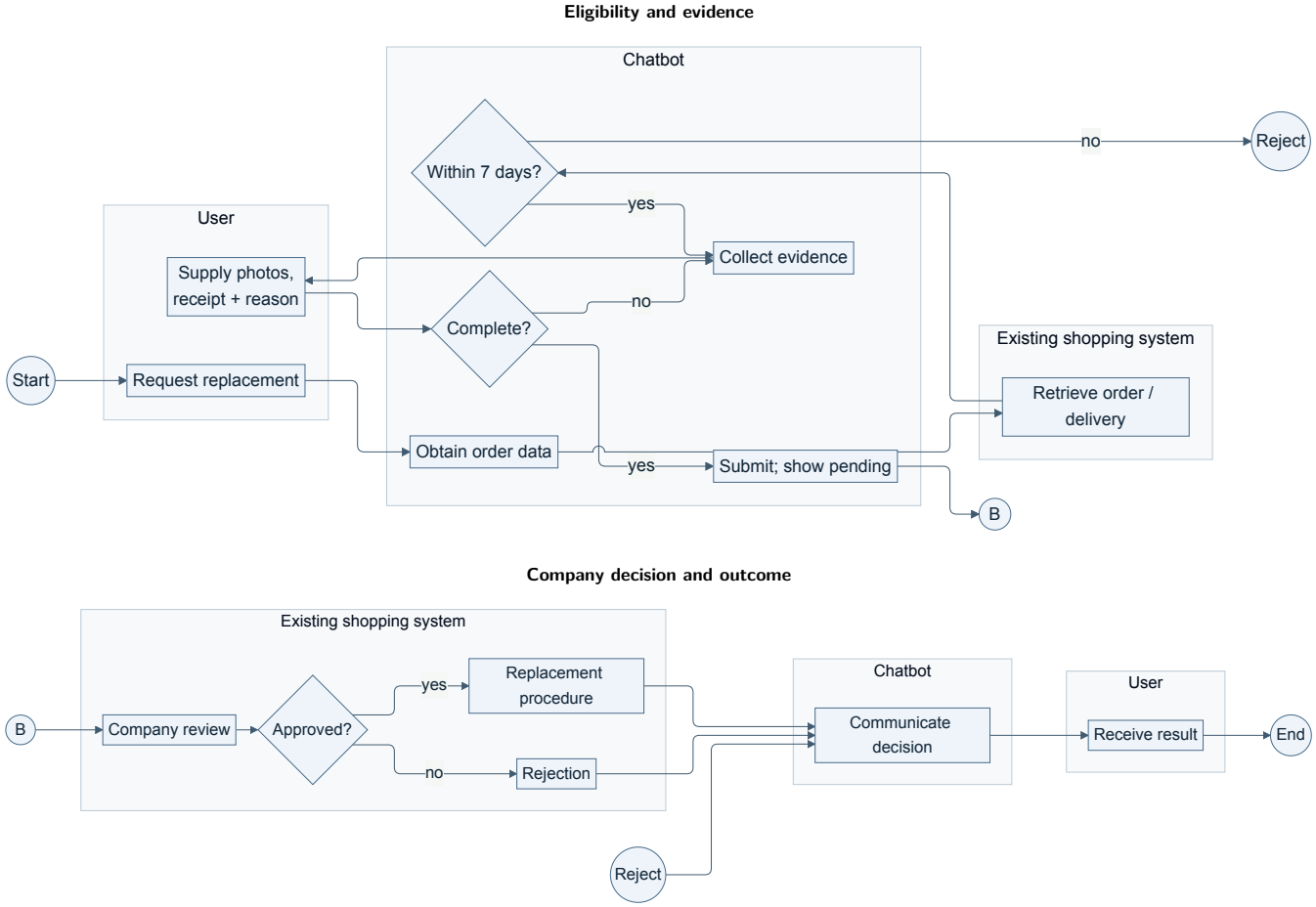


Figure 34: Shopping chatbot: Q2 — Analysis and development process (25 marks)

(c) Use W, p. 15 for reuse: compare the existing shop with must-have requirements, prototype its interfaces, negotiate acceptable compromises and adapt/build unmet essentials.

(d) Apply S, p. 18: expose the bug, let developers investigate, assess Sprint Goal/severity, negotiate less essential work and review prevention. The Scrum Master facilitates.

AY2023/24 S1: DevSecOps — Q2 (25 marks)

(a) Security-check conceptual classes/stereotypes (8). (b) Activity diagram with swimlanes (8). (c) Explain how incremental development can be plan-driven or agile (4). (d) Why may agile struggle with widely varying team skills? (5).

(a)

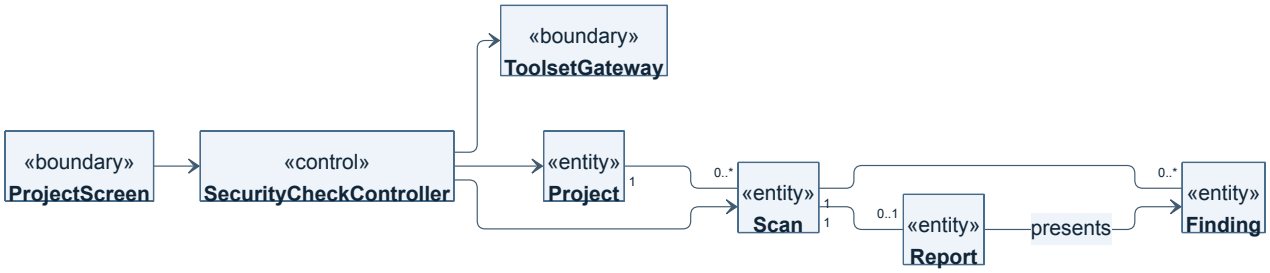


Figure 35: DevSecOps: Q2 — Analysis and agile (25 marks)

(b) The external tool lane represents selected engines, not people inside the platform. Activities may be scheduled in parallel if dependencies permit; that scheduling is not specified by Appendix A.

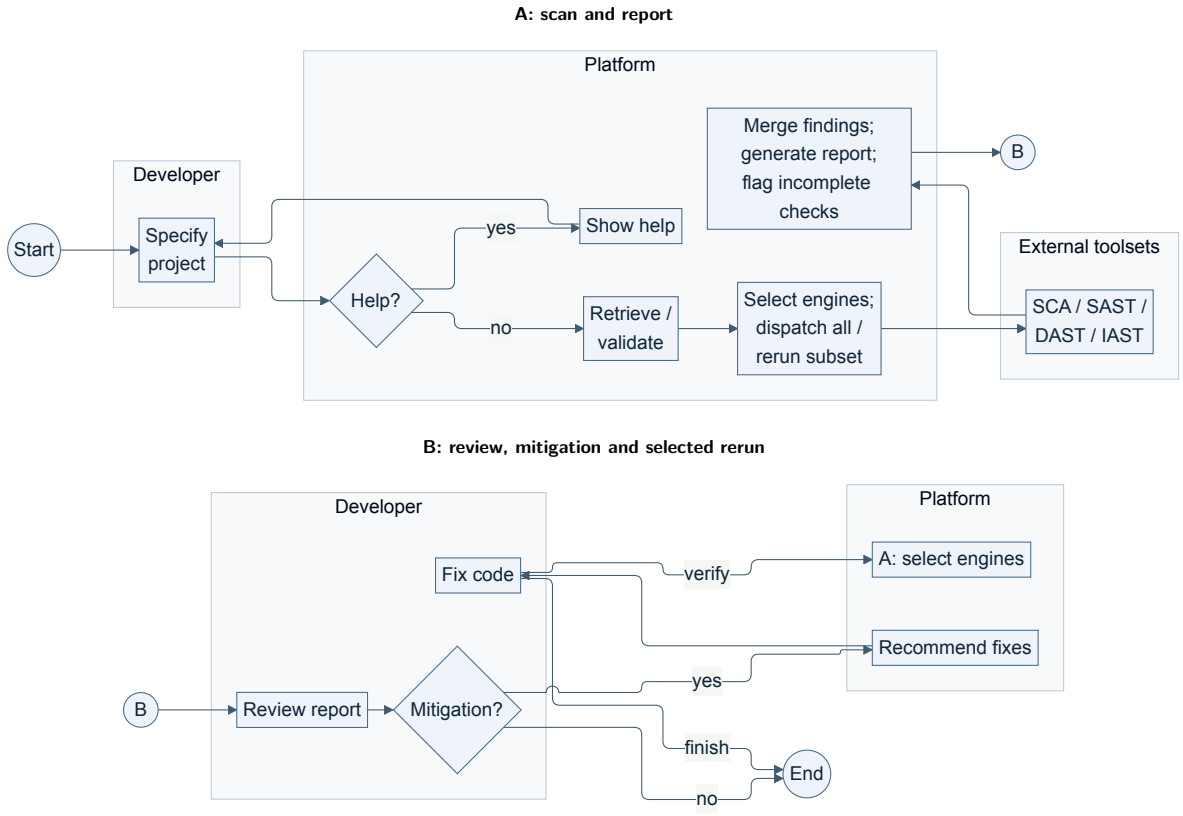


Figure 36: DevSecOps: DevSecOps activity workflow — combined scan/report and review/rerun phases; B continues below, A returns to engine selection.

(c) Use W, p. 15: increments may be preplanned or reprioritised through feedback; delivery structure alone does not establish agile management.

(d) Use S, p. 18: uneven skills risk quality, estimates, expert bottlenecks and integration. Pair/mentor, standardise and automate tests; diversity itself is not disqualifying.

AY2023/24 S2: CodeAgent — Q2 (25 marks)

(a) Reviewer conceptual classes/stereotypes (9). (b) QA-Checker state machine (7). (c) Two ways plan-driven projects can handle changing requirements (4). (d) Importance of Scrum velocity (5).

(a)

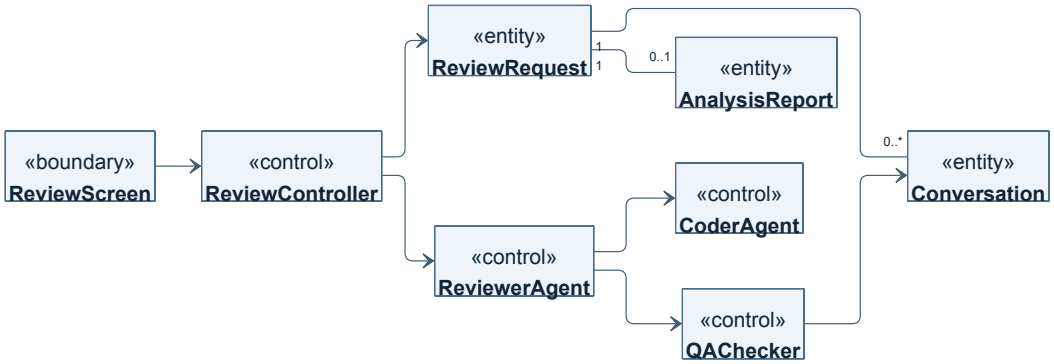


Figure 37: CodeAgent: Q2 — Reviewer analysis, QA states and process (25 marks)

Agents coordinate behaviour; they are not merely stored domain records. ReviewRequest groups commit, message and original files. Report is absent before completion; unresolved status is possible.

(b) Let `n` count answers received and `max` be the configured maximum dialogue count. Assume `max` $\geq$ `1`. On the first received answer set `n=1`.

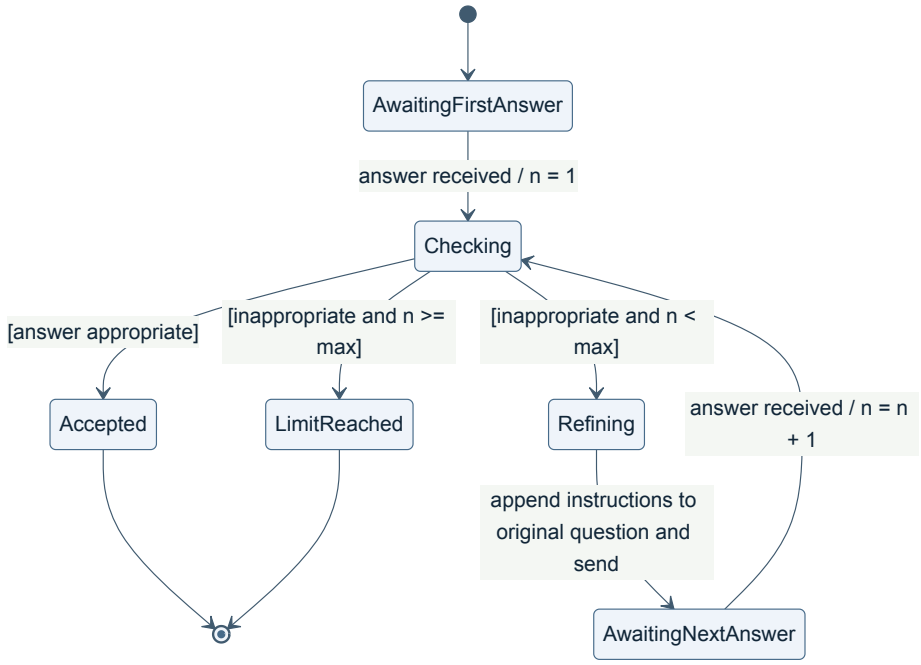


Figure 38: CodeAgent: Q2 — Reviewer analysis, QA states and process (25 marks)

Accept an appropriate answer even on the last permitted round. LimitReached means unresolved, not accepted. This models supervision of **one conversation**; a new conversation starts a new instance/reset.

(c) Apply W, p. 15 with two measures: early prototypes/reviews or planned increments; approved change control updating affected baselines, schedule, budget and tests.

(d) Use S, p. 18: completed velocity supports feasible sprint selection and remaining-effort forecasts. Reforecast capacity; exclude unfinished work and inflated estimates.

Older Q3: Architecture, NFRs and patterns

Shared method: use A, p. 19, P, p. 20, M, p. 23 and C, p. 21. The case answers supply the concrete responsibilities, constraints and trade-offs.

AY2022/23 S1: Car sales — Q3 (25 marks)

(a)(i) Architecture and detailed classes (8); (ii) alternative using a different pattern and its advantages (6). (b)(i) Combined-pattern car-sales scenario and classes (8); (ii) when patterns can be combined/merged, with an example (3).

(a)(i) Apply the shared layered frame. HoldController enforces allocation/refund rules rather than putting them into the page. Extend Q2's conceptual model as follows:

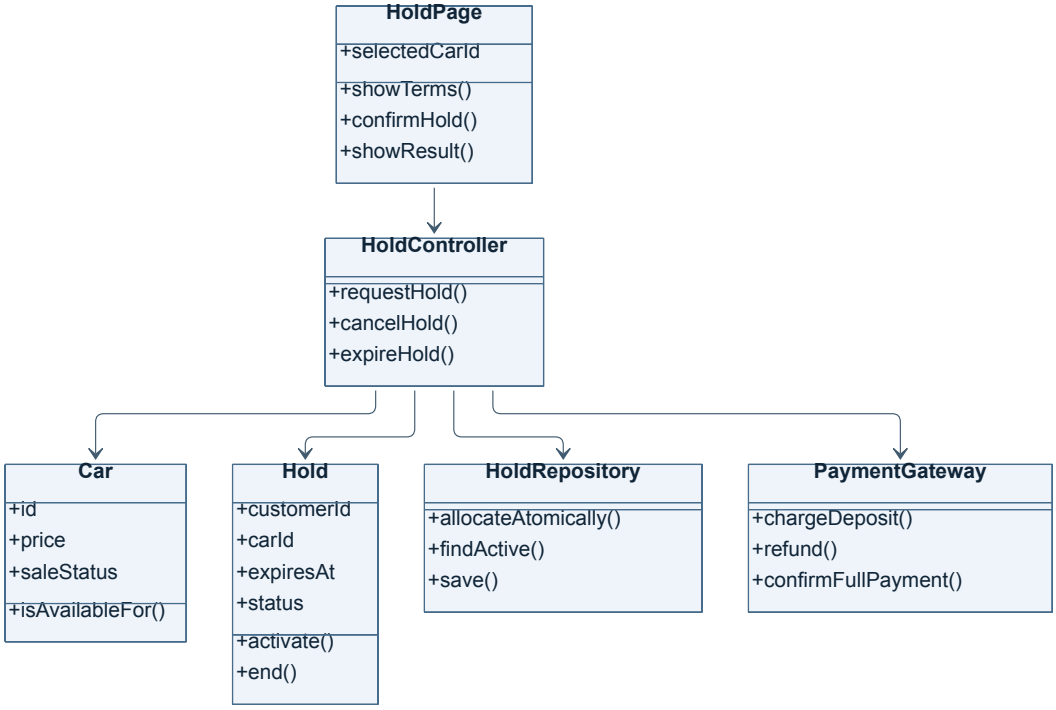


Figure 39: Car sales: Q3 — Architecture and pattern composition (25 marks)

Order/delivery controls similarly require full-payment confirmation before delivery. Persistence must enforce active-hold uniqueness; payment callbacks need reconciliation to avoid recording inconsistent states.

(a)(ii) Apply M, p. 23. Model: Car/Hold plus services; Views: listing/details/status screens; Controller: HoldController. Notify both staff and customers when hold state changes; layered persistence can remain.

(b)(i) Combine **Strategy** with **Simple Factory** for selecting a dealer payment provider. This is a proposed extension; the appendix only requires integration with the dealer payment system.

Class diagram: use P, p. 20. Context = `HoldController`; Strategy = `PaymentStrategy`; Factory = `PaymentFactory`. Concrete strategies: `CardPayment`, `WalletPayment`. Interface/concrete operations: `charge()`, `refund()`. Provider selection is a proposed extension; retain availability, allocation and refund rules.

Patterns do not replace validation or transaction handling.

(b)(ii) Use C, p. 21. MVC combines Observer/Strategy; the payment example separates behaviour from construction. Shared classes are acceptable when responsibilities stay clear.

AY2022/23 S2: Shopping chatbot — Q3 (25 marks)

(a)(i) Architecture/detailed classes (8); (ii) alternative pattern and advantages (6). (b)(i) Combined patterns and class diagram (8); (ii) structural method for comparing architectures (3).

(a)(i) Apply the shared layered frame with a gateway to the existing shop. Keep order/replacement decisions in deterministic services; natural-language interpretation identifies the requested operation.

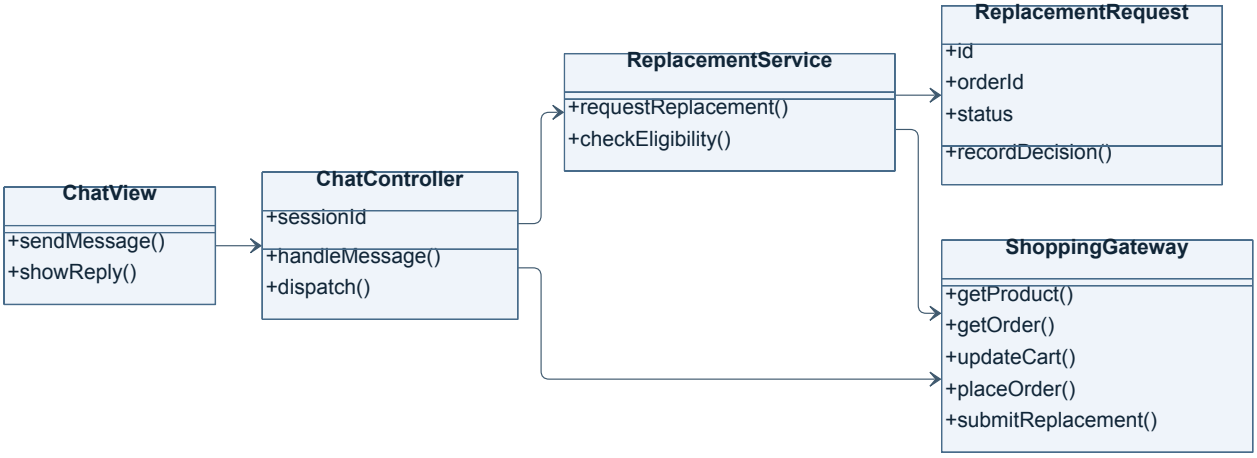


Figure 40: Shopping chatbot: Q3 — Chatbot architecture and combined patterns (25 marks)

(a)(ii) Apply M, p. 23. Models: conversation/cart/replacement request; Views: chat and order status; Controllers: input handling. Notify both views when the company decision arrives; retain layered services underneath.

(b)(i) Combine Strategy for handling different intents with Simple Factory for selecting the handler:

Class diagram: use P, p. 20. Context = `ChatController`; Strategy = `IntentHandler`; Factory = `HandlerFactory`. Concrete strategies: `ProductQueryHandler`, `ReplacementHandler`, `RefundHandler`. Interface/concrete operations: `execute()`. The chatbot selects behaviour; the existing shop still makes the company approval decision.

(b)(ii) Use A, p. 19: compare cohesion, coupling, cycles and change propagation. A changed shopping API should affect its gateway, not every handler/view; raw class/edge counts alone prove little.

AY2023/24 S1: DevSecOps — Q3 (25 marks)

(a)(i) Important NFRs (6); (ii) detailed architecture/classes addressing them (8). (b)(i) Combined-pattern scenario/classes (6); (ii) can AI design software architectures? (5).

(a)(i) Apply A, p. 19 to three proposed NFRs: source confidentiality/isolated execution; replaceable interoperable engines; responsive, reliable long-running scans. Agree measurable criteria rather than inventing guaranteed numbers.

(a)(ii) Add plug-in engine boundaries to the shared layered frame. A scan coordinator implements a Facade for the UI; a stable Engine interface supports replacement tool adapters. Proposed queued jobs and isolated execution address responsiveness and fault containment.

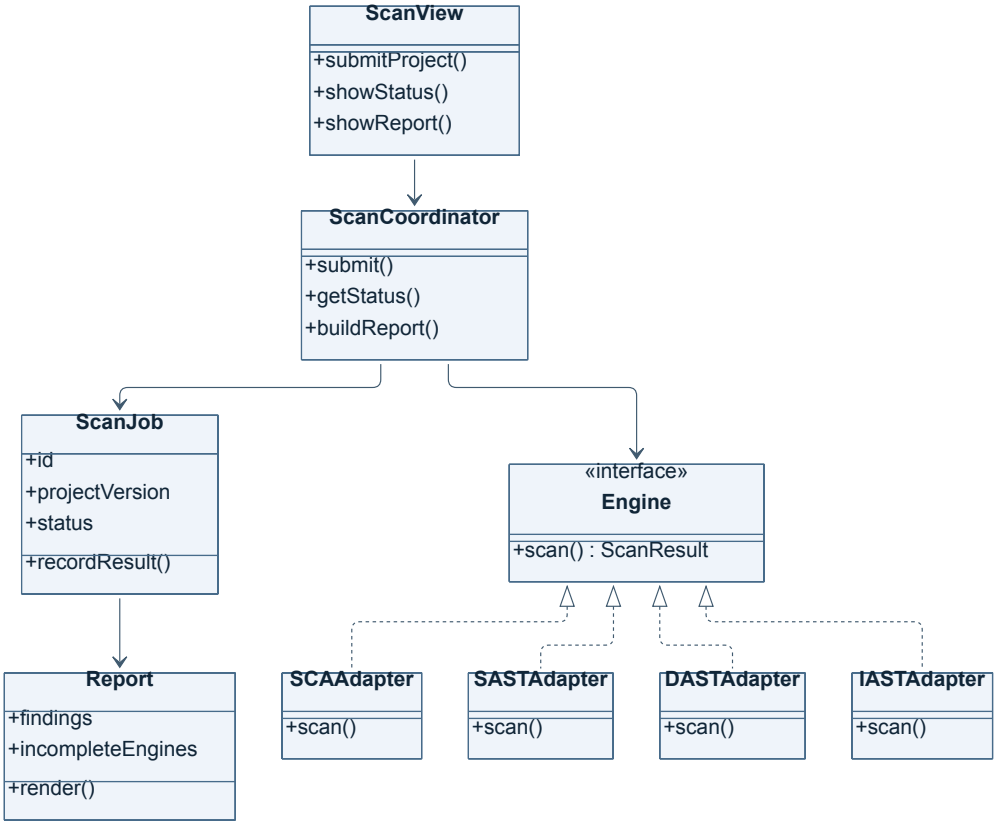


Figure 41: DevSecOps: Q3 — Quality attributes and architecture (25 marks)

Isolate source access and credentials at adapter/execution boundaries; apply timeouts and preserve partial-failure information. Adding an engine should add an adapter rather than change UI or report-domain rules. These choices directly connect the architecture to the proposed NFRs.

(b)(i) Combine Strategy and Simple Factory: engines share the Engine contract while an EngineFactory creates configured adapters. ScanCoordinator uses the resulting strategies without knowing their concrete classes.

Class diagram: use P, p. 20. Context = `ScanCoordinator`; Strategy = `Engine`; Factory = `EngineFactory`. Concrete strategies: `SASTAdapter`, `SCAAdapter`, `DASTAdapter`, `IASTAdapter`. Interface/concrete operations: `scan()`. Create configured tool adapters; preserve the engine result/partial-failure contract.

(b)(ii) AI can propose candidate NFRs, architectures, diagrams and trade-offs. It cannot set missing stakeholder priorities or prove security/performance/integration fitness. Ground it in requirements, review traceability and validate risky decisions with prototypes/tests and human judgement.

AY2023/24 S2: CodeAgent — Q3 (25 marks)

(a)(i) Two additional agent roles (3); (ii) one important NFR (3); (iii) detailed architecture addressing it (8). (b)(i) Combined-pattern scenario/classes (6); (ii) can CodeAgent replace development teams? (5).

(a)(i) A **Test Engineer agent** creates/executes candidate regression tests and reports evidence; a **Requirements Analyst agent** clarifies requests and checks traceability between requirements and reviewed changes. These are proposed additions, distinct from the existing conversation-focused QA-Checker.

(a)(ii) Choose **extensibility/maintainability**: new agent roles and analysis strategies should be addable without changing all existing agents. The request to introduce roles makes this a defensible priority; correctness/security would also require attention in a real platform.

(a)(iii) Use layered UI/orchestration/domain boundaries with plug-in agents. A coordinator schedules phases and supervised conversations; agents implement a common contract; configuration/registry selects roles. This avoids hard-coded peer dependencies throughout the UI.

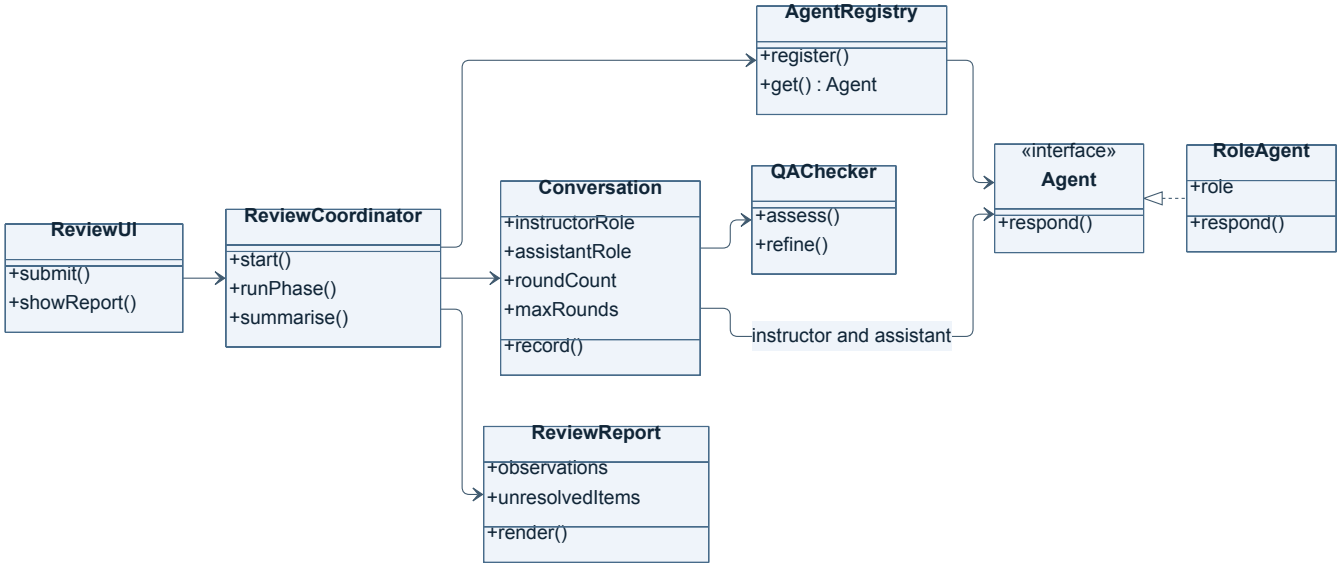


Figure 42: CodeAgent: Q3 — Extending CodeAgent and architecture (25 marks)

Configure RoleAgent for User/CEO/CPO/CTO/Reviewer/Coder, or add new Agent implementations. Persist logs and preserve unresolved status when conversation limits are reached. The registry approach supports the chosen extensibility NFR; storage and orchestration should not depend on one concrete role.

(b)(i) Combine Strategy for analysis methods with Simple Factory for constructing the requested checks:

Class diagram: use P, p. 20. Context = `ReviewerAgent`; Strategy = `ReviewCheck`; Factory = `CheckFactory`. Concrete strategies: `ConsistencyCheck`, `VulnerabilityCheck`, `FormatCheck`. Interface/concrete operations: `analyse()`. Choose the requested review check without hard-coding it in the reviewer.

Reviewer's three analyses retain separate responsibilities, while creation is centralised. A new check can be registered without duplicating the review workflow.

(b)(ii) CodeAgent assists review/revision, but does not replace stakeholder negotiation, requirements clarification, architecture judgement, independent validation, operations or accountability. Agent agreement is not proof of correct software; accept changes using reviewed outputs and executable tests.

Older Q4: Input testing and applied testing/maintenance

Shared input interpretation. The older questions specify an ASCII identifier/description, a fixed-digit integer, a calendar-date range and an allowed category set. Assume positive numeric integers (leading zeroes do not count), inclusive complete years and one-day date resolution. Parsing policy is an explicit assumption; do not invent ASCII length, printable-only or nonempty restrictions. **Shared EC/BV checks.** Each category set is one valid EC; testing all members adds coverage, not extra ECs. ASCII membership and unordered categories have no specified ordered application boundary. Malformed numbers/nonexistent dates are extra EC tests. Keep all boundary triplets and exactly one invalid field per defensive row. All original rows follow under their own cases.

AY2022/23 S1: Car sales — Q4 (25 marks)

(a)(i) Derive ECs for Car Model, Price, Manufacturing Date and Color (4). (ii) Give on/below/above boundary values (4). (iii) Design defensive tests (5).

Input interpretation. Car Model must be ASCII; Price is a 6-digit integer; Manufacturing Date is between 2010 and 2022; Color is one of `White`, `Black`, `Blue`, `Red`, `Others`. Apply the shared input interpretation above.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Car Model	ASCII text	Text containing non-ASCII characters
Price	Integer 100000. . . 999999	Integer below 100000; integer above 999999; malformed/ non-integer input at the input boundary
Manufacturing Date	Real calendar date 2010-01-01. . . 2022-12-31	Real date before 2010-01-01; real date after 2022-12-31; malformed/nonexistent date
Color	One EC containing {White, Black, Blue, Red, Others}	Any unlisted value

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Price: minimum	99999 (reject)	100000 (accept)	100001 (accept)
Price: maximum	999998 (accept)	999999 (accept)	1000000 (reject)
Manufacturing Date: earliest	2009-12-31 (reject)	2010-01-01 (accept)	2010-01-02 (accept)
Manufacturing Date: latest	2022-12-30 (accept)	2022-12-31 (accept)	2023-01-01 (reject)

(iii) Defensive suite:

Car Model	Price	Manufacturing Date	Color	Expected / purpose
`CAR-A`	`100000`	`2010-01-01`	`White`	Accept — valid boundaries / category coverage
`CAR-A`	`100001`	`2010-01-02`	`Black`	Accept — valid boundaries / category coverage
`CAR-A`	`999998`	`2022-12-30`	`Blue`	Accept — valid boundaries / category coverage
`CAR-A`	`999999`	`2022-12-31`	`Red`	Accept — valid boundaries / category coverage
`CAR-A`	`549999`	`2016-07-01`	`Others`	Accept — interior baseline / category coverage
`café`	`549999`	`2016-07-01`	`White`	Reject — non-ASCII Car Model
`CAR-A`	`99999`	`2016-07-01`	`White`	Reject — Price below minimum
`CAR-A`	`1000000`	`2016-07-01`	`White`	Reject — Price above maximum
`CAR-A`	`549999`	`2009-12-31`	`White`	Reject — Manufacturing Date below minimum
`CAR-A`	`549999`	`2023-01-01`	`White`	Reject — Manufacturing Date above maximum
`CAR-A`	`549999`	`2016-07-01`	`Unknown`	Reject — unlisted Color
`CAR-A`	`12x`	`2016-07-01`	`White`	Reject — malformed integer; extra defensive case
`CAR-A`	`549999`	`2010-02-30`	`White`	Reject — nonexistent calendar date; extra defensive case

(b)(i) Give at least two unit/system testing differences (4). (ii) Why might grey-box testing be better in some cases? (6). (iii) When is refactoring needed? (2).

(i) Unit: isolate a component (e.g. Hold expiry) with test doubles; fast, cheap failure localisation. System: verify the integrated application against requirements, e.g. payment creates a hold and delivery requires full payment. It exposes end-to-end interaction faults.

(ii) Grey-box testing uses partial knowledge of interfaces/state/storage to target cases while keeping externally specified oracles. Separate hold/payment records suggest callback-order and reconciliation tests. It can improve fault targeting/cost, but does not guarantee structural coverage or replace black-box requirement checks.

(iii) Refactor costly duplication, coupling or unclear responsibilities. Preserve observable behaviour and check with regression tests; adding features or changing requirements is not refactoring.

AY2022/23 S2: Shopping chatbot — Q4 (25 marks)

(a)(i) Derive ECs for Product ID, Quantity, Payment Date and Shipping method (4). (ii) Give on/below/above boundary values (4). (iii) Design defensive tests (5).

Input interpretation. Product ID must be ASCII; Quantity is a 7-digit integer; Payment Date is between 2010 and 2023; Shipping method is one of `Flat rate`, `Express`, `Next-day Delivery`, `Pickup`, `Others`. Apply the shared input interpretation above.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Product ID	ASCII text	Text containing non-ASCII characters
Quantity	Integer 1000000. . . 9999999	Integer below 1000000; integer above 9999999; malformed/non-integer input at the input boundary
Payment Date	Real calendar date 2010-01-01. . . 2023-12-31	Real date before 2010-01-01; real date after 2023-12-31; malformed/nonexistent date
Shipping method	One EC containing {Flat rate, Express, Next-day Delivery, Pickup, Others}	Any unlisted value

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Quantity: minimum	999999 (reject)	1000000 (accept)	1000001 (accept)
Quantity: maximum	9999998 (accept)	9999999 (accept)	10000000 (reject)
Payment Date: earliest	2009-12-31 (reject)	2010-01-01 (accept)	2010-01-02 (accept)
Payment Date: latest	2023-12-30 (accept)	2023-12-31 (accept)	2024-01-01 (reject)

(iii) Defensive suite:

Product ID	Quantity	Payment Date	Shipping method	Expected / purpose
`P100`	`1000000`	`2010-01-01`	`Flat rate`	Accept — valid boundaries / category coverage
`P100`	`1000001`	`2010-01-02`	`Express`	Accept — valid boundaries / category coverage
`P100`	`9999998`	`2023-12-30`	`Next-day Delivery`	Accept — valid boundaries / category coverage

Continued on next page

Product ID	Quantity	Payment Date	Shipping method	Expected / purpose
P100	9999999	2023-12-31	Pickup	Accept — valid boundaries / category coverage
P100	5499999	2016-07-01	Others	Accept — interior baseline / category coverage
café	5499999	2016-07-01	Flat rate	Reject — non-ASCII Product ID
P100	999999	2016-07-01	Flat rate	Reject — Quantity below minimum
P100	10000000	2016-07-01	Flat rate	Reject — Quantity above maximum
P100	5499999	2009-12-31	Flat rate	Reject — Payment Date below minimum
P100	5499999	2024-01-01	Flat rate	Reject — Payment Date above maximum
P100	5499999	2016-07-01	Unknown	Reject — unlisted Shipping method
P100	12x	2016-07-01	Flat rate	Reject — malformed integer; extra defensive case
P100	5499999	2010-02-30	Flat rate	Reject — nonexistent calendar date; extra defensive case

(b)(i) Use requirement understanding to assist testing (3). (ii) Use code summarisation for testing (3). (iii) Give two maintainability uses (6).

(i) Turn requirements into a traceable candidate test catalogue: normal/alternative use-case flows, input ECs/BVs and expected results. Review cases against the actual requirements, then implement automated tests. The company-confirmation rule must not disappear merely because a generated test assumes automatic approval.

(ii) Summarise branches, state changes and dependencies to identify candidate unit tests, branch/path cases and integration failures. Compare the code's apparent behaviour with the independently specified intended behaviour; a summary of a bug must not become the test oracle. Run tests and inspect coverage rather than treating generated prose as evidence of execution.

(iii) Two useful applications: **program comprehension/ documentation**, where code summaries explain responsibilities and dependencies to maintainers; and **change/refactoring assistance**, where candidate patches remove duplication or address a change request, accompanied by regression tests and human review. Generated explanations and patches can be wrong, so validate them against code, requirements and tests. These are applied suggestions, not official marking points or guarantees of an AI product's capabilities.

AY2023/24 S1: DevSecOps — Q4 (25 marks)

(a)(i) Derive ECs for Project ID, Scan Time Limit (minutes), Last Commit Date and Scan Engines (4). (ii) Give on/below/ above boundary values (4). (iii) Design defensive tests (5).

Input interpretation. Project ID must be ASCII; Scan Time Limit (minutes) is a 6-digit integer; Last Commit Date is between 2000 and 2023; Scan Engines is one of `SAST`, `SCA`, `DAST`, `IAST`, `All Engines`. Apply the shared input interpretation above.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Project ID	ASCII text	Text containing non-ASCII characters
Scan Time Limit (minutes)	Integer 100000. . . 999999	Integer below 100000; integer above 999999; malformed/ non-integer input at the input boundary
Last Commit Date	Real calendar date 2000-01-01. . . 2023-12-31	Real date before 2000-01-01; real date after 2023-12-31; malformed/nonexistent date
Scan Engines	One EC containing {SAST, SCA, DAST, IAST, All Engines}	Any unlisted value

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Scan Time Limit (minutes): minimum	99999 (reject)	100000 (accept)	100001 (accept)
Scan Time Limit (minutes): maximum	999998 (accept)	999999 (accept)	1000000 (reject)
Last Commit Date: earliest	1999-12-31 (reject)	2000-01-01 (accept)	2000-01-02 (accept)
Last Commit Date: latest	2023-12-30 (accept)	2023-12-31 (accept)	2024-01-01 (reject)

(iii) Defensive suite:

Project ID	Scan Time Limit (minutes)	Last Commit Date	Scan Engines	Expected / purpose
PROJ1	100000	2000-01-01	SAST	Accept — valid boundaries / category coverage
PROJ1	100001	2000-01-02	SCA	Accept — valid boundaries / category coverage
PROJ1	999998	2023-12-30	DAST	Accept — valid boundaries / category coverage
PROJ1	999999	2023-12-31	IAST	Accept — valid boundaries / category coverage
PROJ1	549999	2011-07-01	All Engines	Accept — interior baseline / category coverage
café	549999	2011-07-01	SAST	Reject — non-ASCII Project ID

Continued on next page

Project ID	Scan Time Limit (minutes)	Last Commit Date	Scan Engines	Expected / purpose
PROJ1	99999	2011-07-01	SAST	Reject — Scan Time Limit (minutes) below minimum
PROJ1	1000000	2011-07-01	SAST	Reject — Scan Time Limit (minutes) above maximum
PROJ1	549999	1999-12-31	SAST	Reject — Last Commit Date below minimum
PROJ1	549999	2024-01-01	SAST	Reject — Last Commit Date above maximum
PROJ1	549999	2011-07-01	Unknown	Reject — unlisted Scan Engines
PROJ1	12x	2011-07-01	SAST	Reject — malformed integer; extra defensive case
PROJ1	549999	2000-02-30	SAST	Reject — nonexistent calendar date; extra defensive case

(b) Apply unit, integration, system and acceptance testing to the platform (8). (c) Design projects that exercise the four engines (4).

(b) Unit: test validators, result parsers and report aggregation with tool doubles. Integration: test source retrieval, each real tool adapter, result interchange and timeout/error handling. System: run an end-to-end local/GitHub submission through scans, report and mitigation, including selected reruns and partial failures. Acceptance: developers evaluate agreed workflows and quality criteria on representative projects, checking that reports are useful and findings trace to source/version. Begin test design early and automate regression checks; one level cannot substitute for all others.

(c) Prepare controlled projects with known ground truth and clean counterparts: vulnerable dependency/component for SCA; a known insecure code construct for SAST; a safely deployed application with an observable runtime vulnerability for DAST; an instrumented test application exercising a known vulnerable path for IAST. Check detection, location, severity/report mapping and absence of false positives on corrected versions. Include unsupported inputs, tool failure and mixed-engine cases. Run intentionally vulnerable fixtures only in an isolated test environment. "Found at least one issue" alone is not a sufficient oracle.

AY2023/24 S2: CodeAgent — Q4 (25 marks)

(a)(i) Derive ECs for Project Description, Development Time Limit (minutes), Launch Date and Application Type (4). (ii) Give on/below/above boundary values (4). (iii) Design defensive tests (7).

Input interpretation. Project Description must be ASCII; Development Time Limit (minutes) is a 4-digit integer; Launch Date is between 2000 and 2026; Application Type is one of `Desktop Application`, `Web Site`, `Mobile App`, `AI Application`, `All Types`. Apply the shared input interpretation above.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Project Description	ASCII text	Text containing non-ASCII characters
Development Time Limit (minutes)	Integer 1000. . . 9999	Integer below 1000; integer above 9999; malformed/ non-integer input at the input boundary
Launch Date	Real calendar date 2000-01-01. . . 2026-12-31	Real date before 2000-01-01; real date after 2026-12-31; malformed/nonexistent date
Application Type	One EC containing {Desktop Application, Web Site, Mobile App, AI Application, All Types}	Any unlisted value

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Development Time Limit (minutes): minimum	999 (reject)	1000 (accept)	1001 (accept)
Development Time Limit (minutes): maximum	9998 (accept)	9999 (accept)	10000 (reject)
Launch Date: earliest	1999-12-31 (reject)	2000-01-01 (accept)	2000-01-02 (accept)
Launch Date: latest	2026-12-30 (accept)	2026-12-31 (accept)	2027-01-01 (reject)

(iii) Defensive suite:

Project Description	Development Time Limit (minutes)	Launch Date	Application Type	Expected / purpose
`Build service`	1000	2000-01-01	`Desktop Application`	Accept — valid boundaries / category coverage
`Build service`	1001	2000-01-02	`Web Site`	Accept — valid boundaries / category coverage
`Build service`	9998	2026-12-30	`Mobile App`	Accept — valid boundaries / category coverage
`Build service`	9999	2026-12-31	`AI Application`	Accept — valid boundaries / category coverage
`Build service`	5499	2013-07-01	`All Types`	Accept — interior baseline / category coverage

Continued on next page

Project Description	Development Time Limit (minutes)	Launch Date	Application Type	Expected / purpose
café	5499	2013-07-01	Desktop Application	Reject — non-ASCII Project Description
Build service	999	2013-07-01	Desktop Application	Reject — Development Time Limit (minutes) below minimum
Build service	10000	2013-07-01	Desktop Application	Reject — Development Time Limit (minutes) above maximum
Build service	5499	1999-12-31	Desktop Application	Reject — Launch Date below minimum
Build service	5499	2027-01-01	Desktop Application	Reject — Launch Date above maximum
Build service	5499	2013-07-01	Unknown	Reject — unlisted Application Type
Build service	12x	2013-07-01	Desktop Application	Reject — malformed integer; extra defensive case
Build service	5499	2000-02-30	Desktop Application	Reject — nonexistent calendar date; extra defensive case

(b) Improve QA-Checker with course testing techniques (6). (c) Extend CodeAgent for maintenance/evolution (4).

(b) Derive EC/BV tests from requirements for candidate code, including invalid inputs and limits. Generate normal/alternative scenario tests and independently reviewed expected outputs. Use structural analysis for branch/basis-path coverage where code is available, then run unit, integration and system tests in isolation. Regression suites compare revised code with required existing behaviour; known-defect fixtures check whether the agent detects real issues. Record actual results/coverage and return failures to Coder. QA-Checker must distinguish conversational relevance from executable correctness; generated tests and oracles need review because agents may share the same mistake.

(c) Add versioned change requests and impact analysis linking requirements, affected components and tests. Maintain code/documentation/version history and decision logs. Let maintenance agents propose patches/refactorings, run regression and compatibility tests, and require review before acceptance. Update reports/documentation and retain a rollback path. These additions support continuing evolution rather than regenerating the entire project without preserving behaviour.

Paper 5: AY2024/25 Semester 1 — Collaborative task management

Source: Question paper, Q1–Q4 on PDF pp. 1–3; Appendix A on p. 4.

Case facts. Registered users log in; access is role based. Members create tasks; team leaders/project managers verify and assign them to an individual or team, setting deadlines/milestones. Tasks have categories, priorities and tags. Reassignment is allowed, but a task cannot have two assignments simultaneously. Finished tasks can be updated/deleted. Chat, comments and documents support collaboration; dashboards show progress/overdue work/workloads; email and in-app notifications cover deadlines, upcoming tasks and priority changes, with configurable alerts.

Q1 — Use cases (25 marks)

(a) Actors/use cases/relationships (19). (b) Task-assignment use case with actors, preconditions and alternative flows (6).

(a) Actors: Team Member, Team Leader, Project Manager and external Email Server. Leaders/managers have member capabilities; model actor generalization in UML. The dashboard and notification subsystem are internal, not actors.

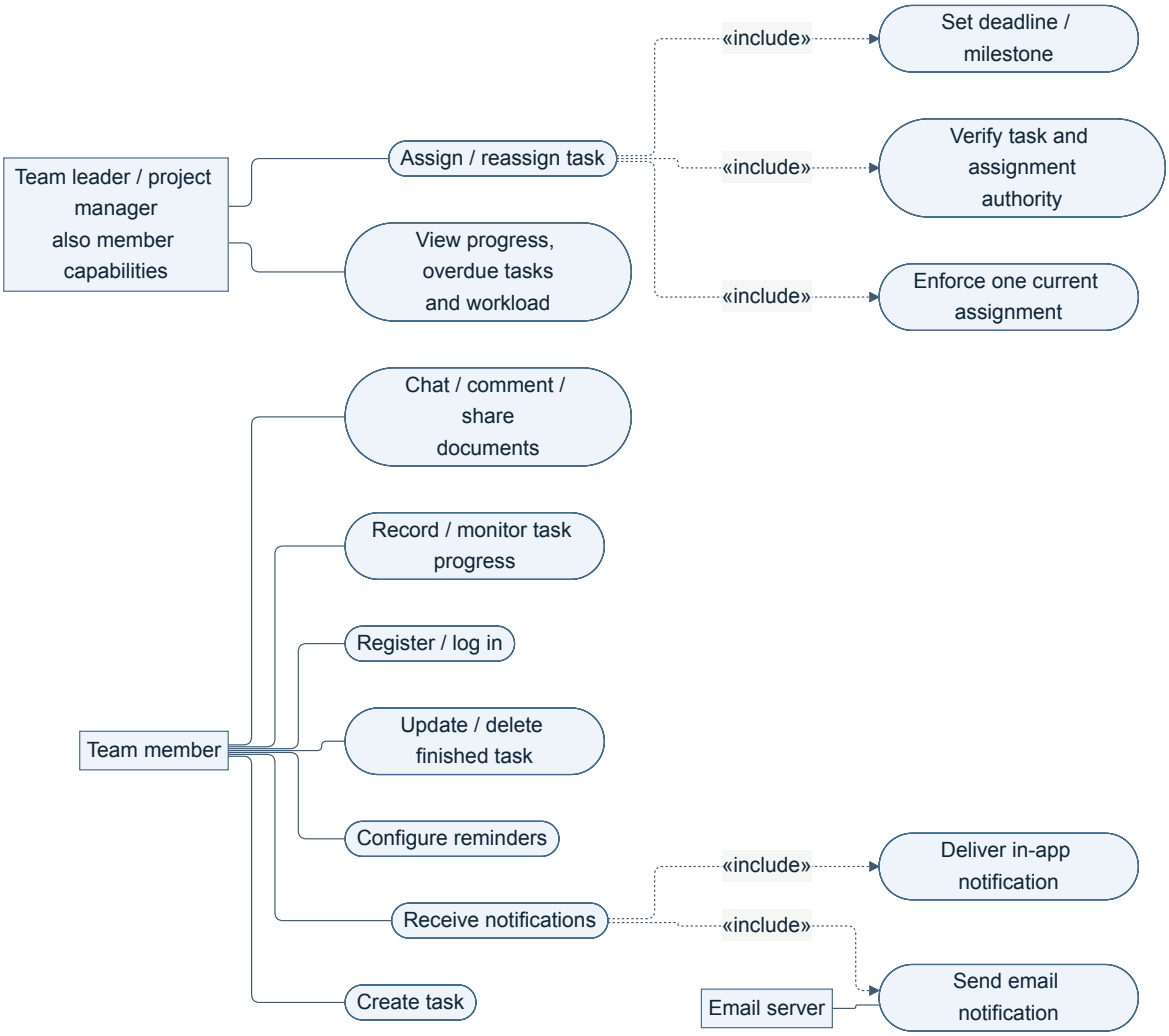


Figure 43: Q1 — Use cases (25 marks)

Every protected action requires a valid session and appropriate role. Member permissions for modifying/deleting tasks must follow the agreed access policy; the appendix does not say every member can modify every task. Include categorisation/priority/tag fields in Create/Update Task descriptions rather than treating each field entry as a separate user goal. No optional extension is needed simply to satisfy the diagram wording.

(b) Use case: Assign Task. Primary actor: authenticated Team Leader or Project Manager. Preconditions: task exists and actor is authorised; chosen individual/team exists. Normal flow: actor selects task; system presents task and current assignment; actor verifies task, chooses one assignee and specifies deadline/milestones; system validates rights and details; system atomically records the assignment and exposes updated workload/progress. Alternatives: unverified/invalid task or unavailable assignee, request correction; unauthorised actor, reject; concurrent assignment change, reject/reload; reassignment, replace the old assignment atomically instead of appending a second active assignment. **Assumption:** "one assignment" means one assignee target, which may be an individual or a team, not only one person within a team.

Q2 — Analysis, lifecycle and process (25 marks)

(a) Assignment conceptual classes/stereotypes (9). (b) Activity/swimlane diagram from task creation to deletion (7). (c) Waterfall response to drastic mid-project requirement change (4). (d) Improve customer satisfaction after a disappointing first agile sprint (5).

(a)

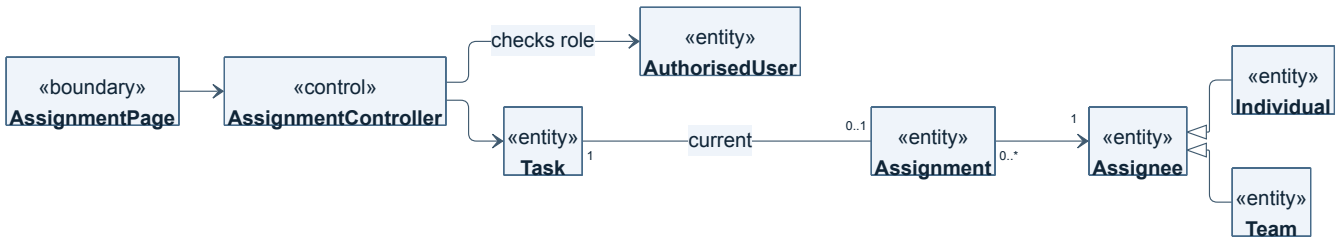


Figure 44: Q2 — Analysis, lifecycle and process (25 marks)

If assignment history is added, use `0..*` historical assignments plus an explicit **at most one active assignment** constraint; do not remove the invariant.

(b) The appendix permits update/deletion after completion, so those actions appear after Finished. Reassignment is an atomic replacement. Deletion is optional, not an automatic consequence of finishing.

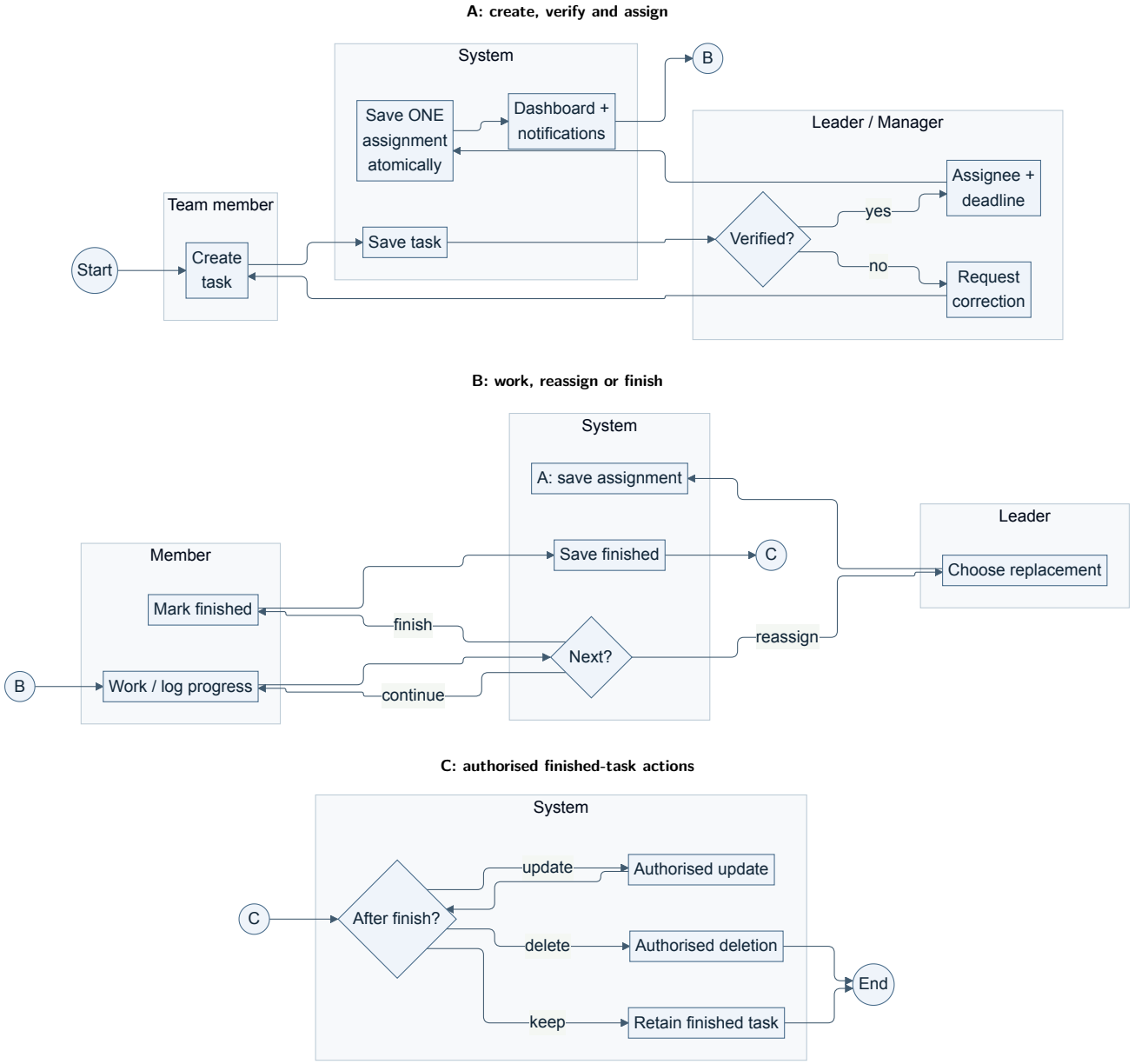


Figure 45: Q2 — Analysis, lifecycle and process (25 marks) — B/C continue below; A returns to atomic assignment.

(c) Apply W, p. 15 to the drastic task-system change; update affected artefacts and agree revised scope/schedule. Persistent change may justify incremental/agile delivery.

(d) Apply S, p. 18: clarify unmet needs/acceptance criteria, reprioritise value with the PO, choose smaller usable increments and retrospect on estimates/capacity/integration.

Q3 — Architecture, NFR and patterns (25 marks)

(a)(i) Detailed architecture/classes (8); (ii) one relevant NFR and alternative architecture addressing it (8). (b)(i) Combined-pattern scenario/classes (6); (ii) is combining patterns always beneficial? (3).

(a)(i) Use a layered task application: UI boundaries, application services, task/assignment entities and repositories/notification gateways.

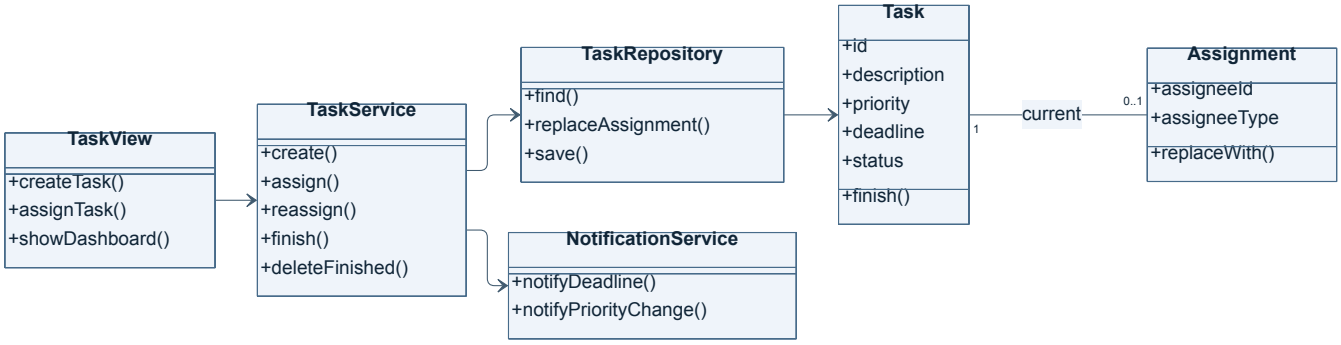


Figure 46: Q3 — Architecture, NFR and patterns (25 marks) — assignment replacement is atomic

(a)(ii) Choose **security/access control**, explicitly motivated by role-based access and sensitive team information. An alternative centralised **client–server architecture with a security Facade** places all task operations behind authenticated server-side authorisation. Browser/UI

role checks alone cannot protect the data.

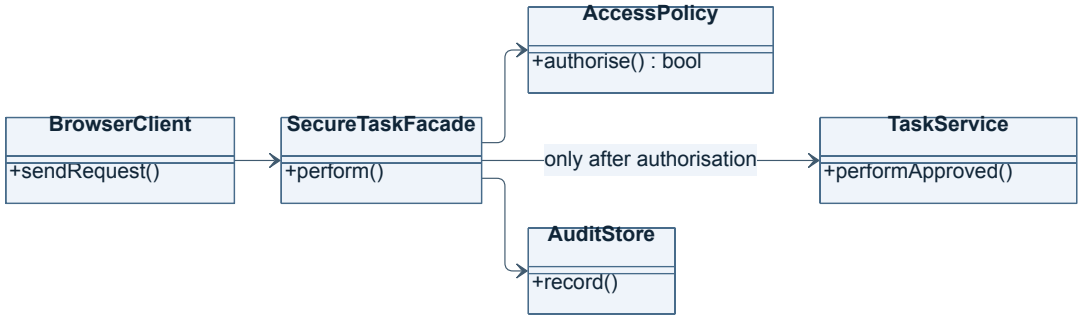


Figure 47: Q3 — Architecture, NFR and patterns (25 marks)

Keep Task/Assignment/Repository from Q3(a)(i) behind TaskService. Centralisation makes policy consistent across clients and supports audited denials; it adds a server/network dependency. Audit storage is a proposed security measure, not an explicit appendix feature. Layering and client–server are compatible architectural dimensions; the alternative adds a trust boundary rather than claiming that these styles are mutually exclusive.

(b)(i) Combine Observer with Simple Factory for email and in-app notifications. A task subject notifies both subscribed channels on relevant events; a factory creates/configures channel observers. Deadline timers also publish events rather than requiring a task to mutate every second.

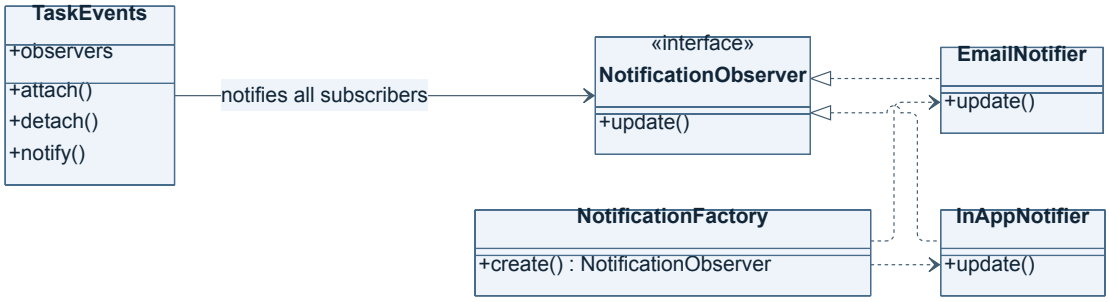


Figure 48: Q3 — Architecture, NFR and patterns (25 marks)

Factory governs creation; Observer governs runtime notification without coupling task code to concrete delivery channels. The requirements call for email **and** in-app messages, not arbitrarily selecting only one.

(b)(ii) No: apply C, p. 21. Email and in-app notifications justify separate channels; unrelated patterns add cost without solving another force.

Q4 — Task inputs, shift-left and maintenance (25 marks)

(a)(i) Derive ECs for Task Description, Resource needed, Deadline and Task Priority (4). (ii) Give on/below/above boundary values (4). (iii) Design defensive tests (5).

Input interpretation. Task Description must be ASCII; Resource needed is a 4-digit integer; Deadline is between 2024 and 2026; Task Priority is one of `Critical`, `High`, `Normal`, `Low`, `Very Low`. Assume a positive 4-digit numeric value (not a string whose leading zeroes count as digits), inclusive complete calendar years, and one-day date resolution. These interpretations are stated because the brief does not specify parsing details. Do not invent an ASCII-field length limit.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Task Description	ASCII text	Text containing non-ASCII characters
Resource needed	Integer 1000. . . 9999	Integer below 1000; integer above 9999; malformed/non-integer input at the input boundary
Deadline	Real calendar date 2024-01-01. . . 2026-12-31	Real date before 2024-01-01; real date after 2026-12-31; malformed/nonexistent date
Task Priority	One EC containing {Critical, High, Normal, Low, Very Low}	Any unlisted value

The category set is **one valid EC**, although testing all its few members is useful. No empty-string prohibition or printable-only ASCII restriction is supplied; clarify those separately instead of asserting an invented oracle.

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Resource needed: minimum	999 (reject)	1000 (accept)	1001 (accept)
Resource needed: maximum	9998 (accept)	9999 (accept)	10000 (reject)
Deadline: earliest	2023-12-31 (reject)	2024-01-01 (accept)	2024-01-02 (accept)
Deadline: latest	2026-12-30 (accept)	2026-12-31 (accept)	2027-01-01 (reject)

ASCII membership and unordered categories have no specified application-level ordered boundary. Malformed values are additional EC

tests, not numeric/date neighbours.

(iii) Defensive suite. Use the common testing conventions below; valid rows combine boundaries, invalid rows isolate one field. The full boundary suite is retained.

Task Description	Resource needed	Deadline	Task Priority	Expected / purpose
`Implement API`	`1000`	`2024-01-01`	`Critical`	Accept — valid boundaries / category coverage
`Implement API`	`1001`	`2024-01-02`	`High`	Accept — valid boundaries / category coverage
`Implement API`	`9998`	`2026-12-30`	`Normal`	Accept — valid boundaries / category coverage
`Implement API`	`9999`	`2026-12-31`	`Low`	Accept — valid boundaries / category coverage
`Implement API`	`5499`	`2025-07-01`	`Very Low`	Accept — interior baseline / category coverage
`café`	`5499`	`2025-07-01`	`Critical`	Reject — non-ASCII Task Description
`Implement API`	`999`	`2025-07-01`	`Critical`	Reject — Resource needed below minimum
`Implement API`	`10000`	`2025-07-01`	`Critical`	Reject — Resource needed above maximum
`Implement API`	`5499`	`2023-12-31`	`Critical`	Reject — Deadline below minimum
`Implement API`	`5499`	`2027-01-01`	`Critical`	Reject — Deadline above maximum
`Implement API`	`5499`	`2025-07-01`	`Unknown`	Reject — unlisted Task Priority
`Implement API`	`12x`	`2025-07-01`	`Critical`	Reject — malformed integer; extra defensive case
`Implement API`	`5499`	`2024-02-30`	`Critical`	Reject — nonexistent calendar date; extra defensive case

(b) Why is shift-left testing useful? (4). (c) Four maintenance challenges and mitigations (8).

(b) Reviewing requirements and designing tests early exposes ambiguity before it spreads into design/code. Early unit/integration automation provides fast feedback, reduces downstream rework and makes changes safer. Examples: identify the one-assignment invariant during analysis and test the service before building the dashboard. Shift-left complements rather than eliminates later system, performance and acceptance testing.

(c)

Challenge	Mitigation
Understanding unfamiliar/poorly documented legacy code	Maintain architecture/decision notes, clear interfaces and traceable change history; use code reviews and onboarding.
Regressions caused by changes	Automate regression tests, check integration/compatibility, and use reviewed version-controlled changes with rollback.
Increasing technical debt and coupling	Refactor incrementally with behaviour-preservation checks; keep responsibilities and boundaries clear.
Changing requirements/dependencies and operational environments	Perform impact analysis, prioritise changes, maintain supported dependency versions, and update requirements/tests/documentation together.

Tie each challenge to its remedy rather than listing four generic "best practices" without explanation.

Paper 6: AY2024/25 Semester 2 — Disaster Warning System (DWS)

Source: Question paper, Q1–Q4 on PDF pp. 1–4; Appendix A on p. 5.

Case facts. A local government deploys a configurable DWS. Its manager registers/deregisters physically connected sensors and alert systems, links/unlinks registered sensor–alert pairs, and chooses safe measurement ranges. Registered sensors send integer updates at any time; an out-of-range value triggers the linked alert. Changes and received/sent messages are recorded. More than 24 hours without an update from a registered sensor causes a manager alert. The manager can retrieve the full record.

Q1 — Stakeholders, requirements and use cases (25 marks)

(a) Four stakeholders and interests (4). (b) Three FRs and three NFRs (6). (c) Actors/use cases/relationships (15).

(a) Local government: public safety, budget and dependable operation. DWS manager: correct configuration, sensor health and usable records. Emergency responders/alert recipients: timely, accurate, actionable warnings. Residents: protection from disasters and avoidance of missed/false alarms. A stakeholder need not directly operate the software and therefore need not be an actor.

(b) Three FRs: (1) accept sensor/alert registration only when physical connection is detected; (2) maintain configured links/ranges and trigger linked alerts for out-of-range integer measurements; (3) notify the manager when a registered sensor has supplied no update for over 24 hours. Recording and record retrieval are additional FRs.

Three proposed NFRs: (1) flexibility/interoperability across different deployments and sensor/alert implementations; (2) availability/reliability for ongoing monitoring and fault recovery; (3) performance for timely processing and delivery of alerts under the agreed peak sensor workload. Flexibility is explicit in the case; measurable availability, latency and recovery targets must be agreed, not invented. "Log all messages" is functional behaviour, not automatically an NFR merely because it concerns auditing.

(c) Actors: DWS Manager, external Sensor, external Alert System. Time passage is an event, not necessarily an actor.

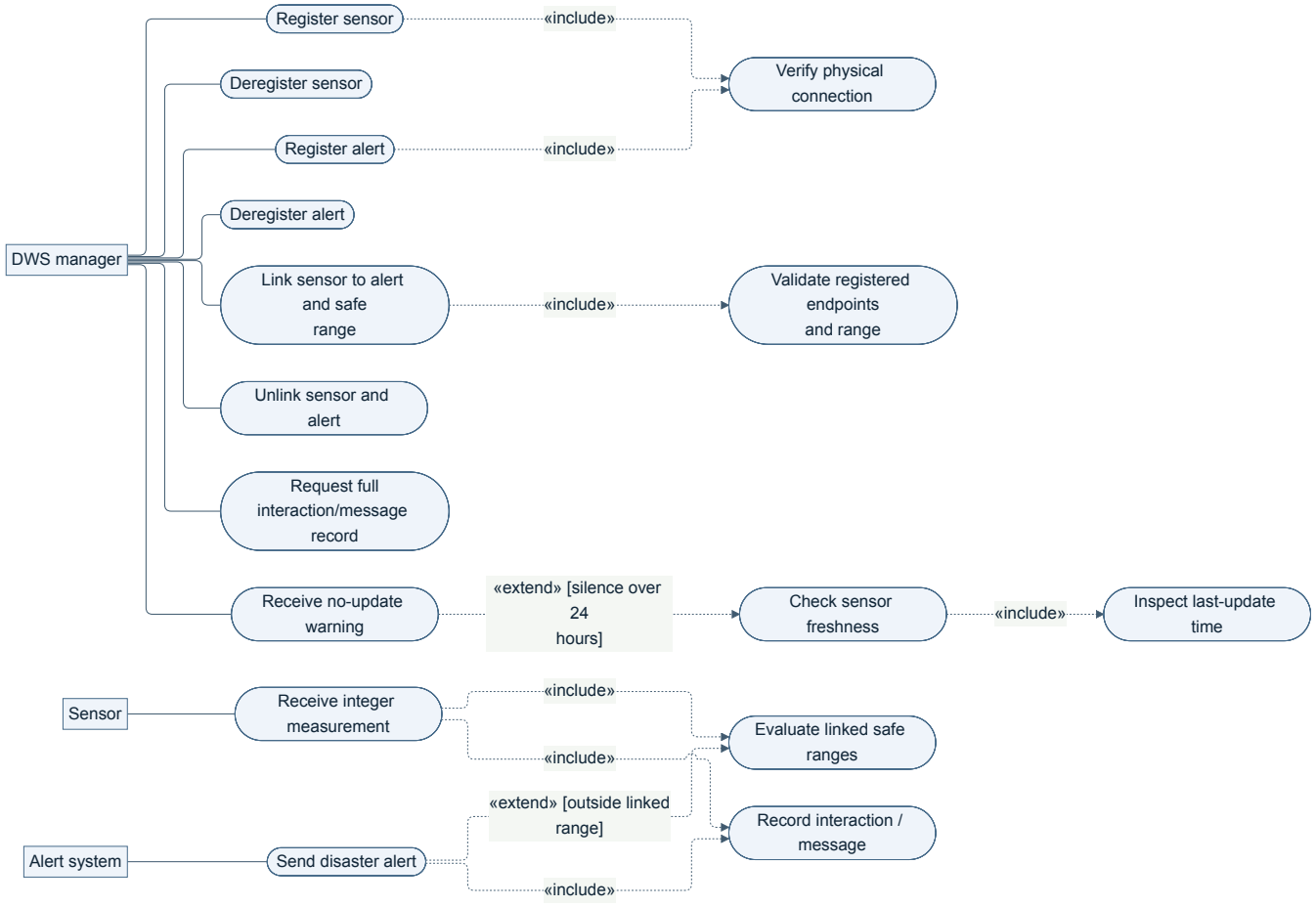


Figure 49: Q1 — Stakeholders, requirements and use cases (25 marks)

Registration, deregistration, linking and unlinking also include recording; add their arrows to the shared Record use case in exact UML. The physical-connection check applies only to registration. Safe-range alerting is conditional, making «extend» reasonable here. The system must not let an unregistered sensor bypass its configured registration policy.

Q2 — Sensor behaviour and process (25 marks)

(a) Sensor-object state machine (6). (b) Linking activity/ swimlanes (7). (c)(i) Choose Waterfall or Agile for changing requirements (3); (ii) manage changes (3). (d) Scrum response to an urgent mid-sprint feature request (6).

(a) Model the sensor's **DWS-side lifecycle**, since this software cannot dictate the internal hardware's physical operating states. A registered sensor is monitored even when quiet; silence is not automatic deregistration. A registration timestamp starts monitoring until the first update, a necessary stated assumption because the appendix does not define that initial timer.

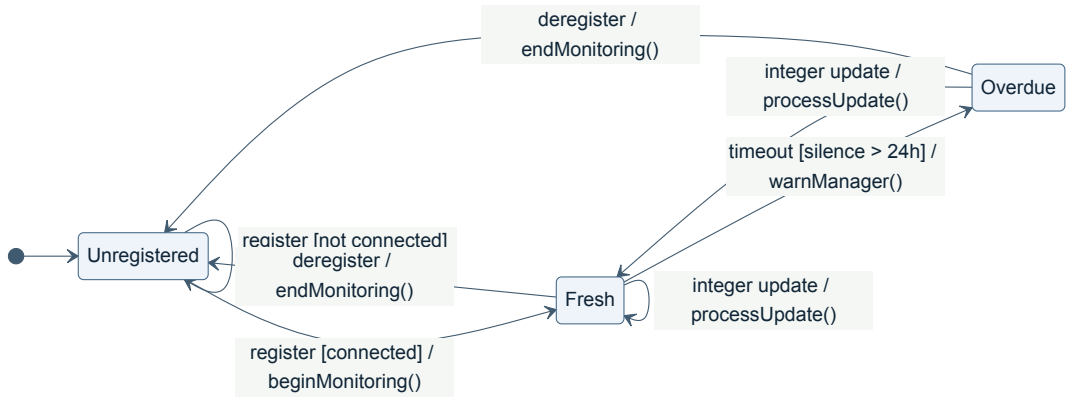


Figure 50: Q2 — Sensor behaviour and process (25 marks)

`beginMonitoring()` records registration and starts freshness timing; `processUpdate()` records the valid measurement, evaluates linked ranges and resets freshness; `endMonitoring()` records deregistration and stops monitoring/subscriptions. Exactly 24 hours is not "over 24 hours". Repeated warning frequency is unspecified; assume one warning per overdue episode unless the client defines reminders. Invalid-type messages are rejected/logged by a defensive integration boundary, not treated as valid measurements.

(b)

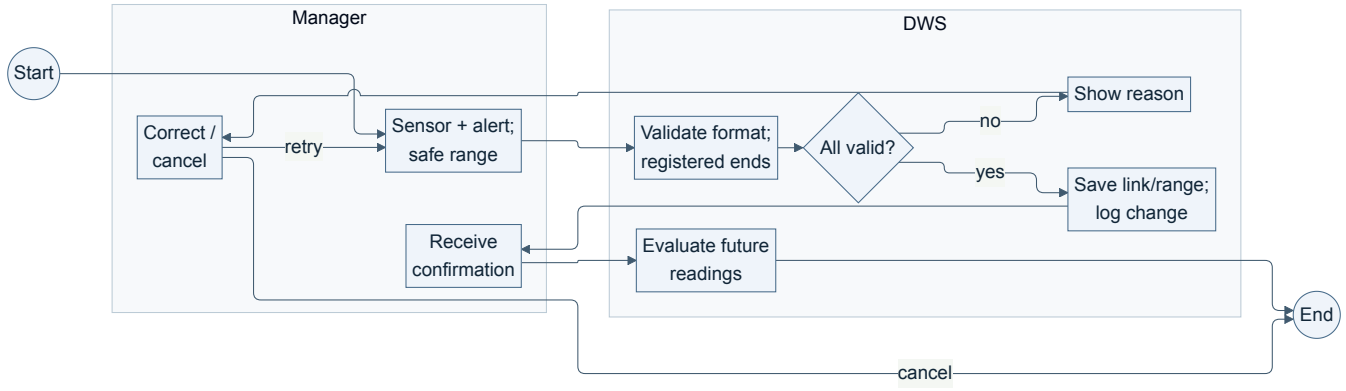


Figure 51: Q2 — Sensor behaviour and process (25 marks)

Linking does not require an immediate disaster measurement, nor does it automatically send an alert. Preserve the existing configuration if validation fails.

(c)(i) Recommend Agile/incremental delivery because requirements may change and early stakeholder feedback can refine configuration and warning workflows. Retain appropriate safety reviews, documentation and rigorous testing; "agile" does not mean no planning.

(c)(ii) Capture changes in a prioritised backlog, clarify acceptance criteria, analyse safety/interface impacts and select feasible increments. Protect the current sprint's goal; negotiate later scope and re-estimate affected work, then validate changes with regression and acceptance tests.

(d) Use S, p. 18: PO/developers assess urgency and goal impact; normally backlog the feature for a later sprint. Negotiate transparent scope changes; only the PO cancels an obsolete goal.

Q3 — DWS architecture, Observer and responsibility (25 marks)

(a) Architecture/justification (3). (b)(i) Most applicable pattern (3); (ii) detailed classes (10). (c)(i) One reason for/against blaming the vendor for a crash on a string measurement (2); (ii) two other responsible parties (4); (iii) three prevention/risk-reduction measures (3).

(a) An event-driven architecture, with layered configuration, monitoring/domain and device-integration services, fits asynchronous measurements. A registry manages runtime membership, a monitor evaluates configured links, and gateways deliver alerts while a record store preserves interactions. Isolate malformed or failing devices so one sensor cannot crash monitoring for all others.

(b)(i) Observer: sensor subjects publish measurements to a DWS observer that can subscribe/unsubscribe at runtime. This avoids tight coupling to a fixed sensor set and avoids requiring the DWS to synchronously poll for every update. Distinguish domain registration from subscription: the registration service coordinates both, but a list of objects alone is not Observer.

(b)(ii)

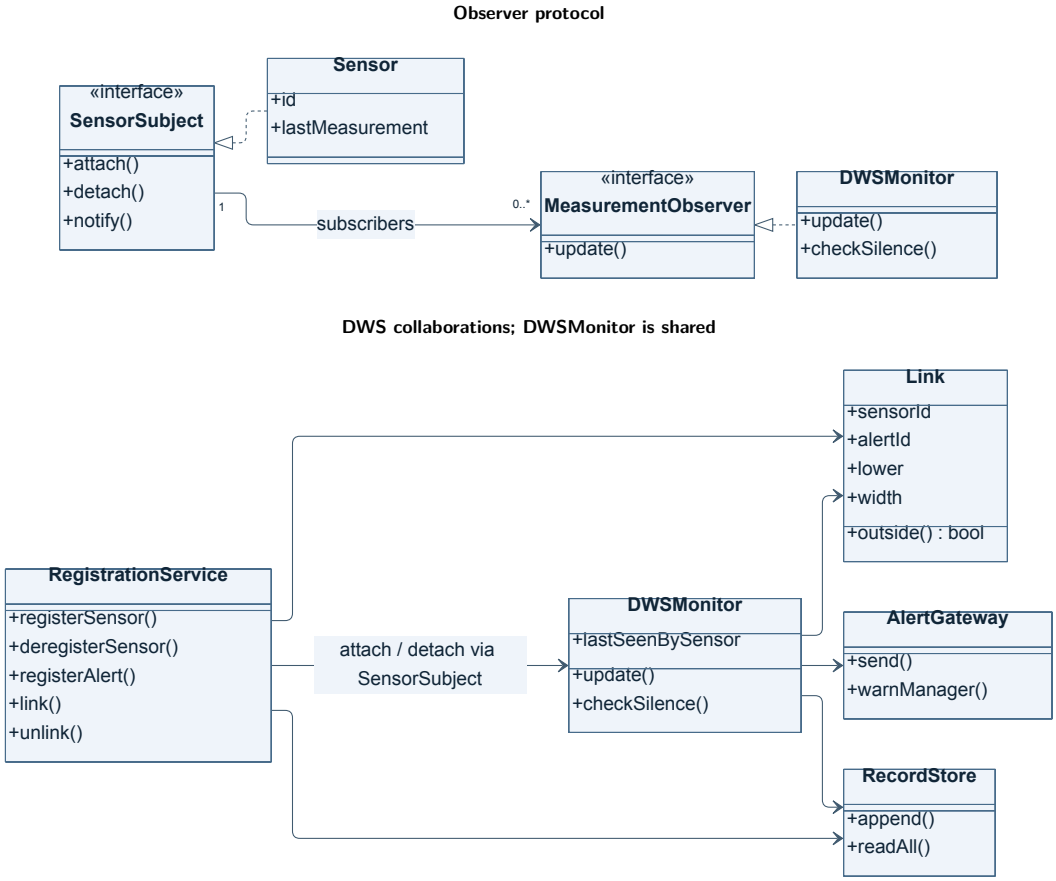


Figure 52: Q3 — DWS architecture, Observer and responsibility (25 marks)

For heterogeneous external devices, a sensor adapter can expose SensorSubject while translating the real protocol. `update` validates integer messages, records them, updates freshness, evaluates every matching Link, and records any sent alerts. Registration verifies connection before subscribing. Link changes are recorded. Deregistration stops monitoring/subscriptions; the policy for dependent links must be agreed rather than leaving dangling references. Use overflow-safe arithmetic for safe-range calculations.

(c)(i) Engineering arguments **for** responsibility: a foreseeable malformed external input should be rejected/isolated rather than causing a system-wide outage, especially in a warning system. **Against:** the third-party sensor violated the specified integer interface contract; the DWS vendor did not control that sensor's implementation. Both facts can be relevant; neither alone settles all responsibility.

(c)(ii) Sensor manufacturer: emitted a value outside the agreed protocol or failed to validate its output. Client/integrator: installed or approved incompatible equipment without verifying conformance and deployment integration. Explain the actual contribution rather than assigning blame solely because a party exists.

(c)(iii) Three measures: (1) validate types/ranges and contain exceptions at every device boundary; (2) run defensive, fault-injection and third-party conformance/integration tests, including string messages and disconnected devices; (3) document interface contracts, supported-device requirements and acceptance evidence, with monitoring and clear failure reporting. Documentation reduces ambiguity but does not substitute for robust error handling.

Q4 — DWS defensive tests and dependent inputs (25 marks)

Given: deployed sensor `smoke concentration`, alert `fire`; signed five-digit decimal `N` (e.g. +83220, −00123); unsigned five-digit non-negative decimal `M` (e.g. 12345, 00020). Safe upper bound is `N+M`. Individually, integer values outside −32768...32767 must be rejected.

(a) ECs for the four inputs (5). (b) Boundary triplets (4). (c) Defensive test suite (8). (d) Consequences of additionally restricting `N+M` to signed 16-bit range (4). (e)(i) Difficulty of one generic deployment-independent suite (2); (ii) solution (2).

(a) Treat formatting and parsed numeric value separately. Assume exactly one sign and five ASCII digits for N, and exactly five ASCII digits without a sign for M. Leading zeroes are required/allowed as in the paper. ASCII names must also identify registered endpoints in this deployment.

Field	Valid EC	Invalid ECs
Sensor name	Exact registered name `smoke concentration`	Other ASCII name not registered; name containing non-ASCII characters
Alert name	Exact registered name `fire`	Other ASCII name not registered; name containing non-ASCII characters
N	Format `[+-][0-9]{5}` and parsed value $-32768 \leq N \leq 32767$	Well-formed value below −32768; well-formed value above 32767; malformed representation
M	Format `[0-9]{5}` and parsed value $0 \leq M \leq 32767$	Well-formed value above 32767; malformed representation, including a negative/sign-bearing input

These partitions are disjoint when numeric invalid classes are restricted to correctly formatted values. +83220 illustrates the required shape but is **invalid under the additional 16-bit rule**. Negative M cannot satisfy its unsigned format. No assumption that every ASCII name is registered is allowed.

(b) Numeric triplets, written using the required formats:

Boundary	Just below	On boundary	Just above
N minimum −32768	−32769 reject	−32768 accept	−32767 accept
N maximum 32767	+32766 accept	+32767 accept	+32768 reject
M minimum 0	−00001 reject (format)	00000 accept	00001 accept
M maximum 32767	32766 accept	32767 accept	32768 reject

Name membership has no ordered boundary. Extra/missing digits and signs are format tests, not numeric boundary neighbours. The just-below-zero M representation is necessarily invalid in format; there is no negative, unsigned, well-formed M string.

(c) In this initial version, the **sum has no extra 16-bit rejection rule**. For each row keep all other inputs valid. Abbreviations in the table: `S = smoke concentration`, `A = fire`; `other sensor` and `other alert` are stipulated unregistered. A means accepted link/configuration, R means clean rejection without changing links or crashing.

Sensor	Alert	N	M	Expected / purpose
S	A	+00000	00020	A — baseline
S	A	−32768	00000	A — N lower, M lower
S	A	−32767	00001	A — valid lower neighbours
S	A	+32766	32766	A — valid upper neighbours; sum restriction absent
S	A	+32767	32767	A — valid maxima; sum restriction absent
S	A	−32769	00020	R — N too small
S	A	+32768	00020	R — N too large
S	A	+00000	32768	R — M too large
S	A	+00000	−00001	R — M negative / forbidden sign
other sensor	A	+00000	00020	R — sensor not registered
`fumée`	A	+00000	00020	R — sensor non-ASCII
S	other alert	+00000	00020	R — alert not registered
S	`feu-é`	+00000	00020	R — alert non-ASCII
S	A	00000	00020	R — N missing sign
S	A	+0000	00020	R — N too few digits
S	A	+000000	00020	R — N too many digits

Continued on next page

Sensor	Alert	N	M	Expected / purpose
S	A	+00a00	00020	R — N non-digit
S	A	+00000	+00020	R — M forbidden sign
S	A	+00000	0020	R — M too few digits
S	A	+00000	000020	R — M too many digits
S	A	+00000	00a20	R — M non-digit

Use fresh/reset fixtures for each row so an existing link does not alter the oracle. Exactly one **parameter** is invalid in each rejected row; a malformed parameter can violate more than one format predicate. Supplement with physical-connection/registration tests at the appropriate integration boundary.

(d) Now M's valid numeric range depends on N:

$0 \leq M \leq \texttt{min(32767, 32767} - \texttt{N)}$, with valid N already in $-32768...32767$.

The sum cannot underflow because M is non-negative, but it can overflow. Partition the **pair** (N,M) into sum-valid and sum-too-large cases in addition to independent format/range classes. For N=+32760, M=00006, 00007, 00008 exercise below/on/above the upper sum boundary; the last is now rejected. At N=+32767, only M=00000 is valid. Reclassify earlier maximum-pair tests rather than reusing their old acceptance oracles. Compute the sum in a wider/checked representation before testing its range.

(e)(i) Registered names and even available devices vary by deployment. A name valid in one installation can be invalid in another; a supposedly "invalid" hard-coded name may actually be registered elsewhere. Physical connections and existing links also affect results.

(e)(ii) Parameterise the suite with known registered names, verified absent ASCII names and resettable configuration. Prefer a controlled fixture that registers a test sensor and alert (using connected simulators/test devices), runs the suite, and restores state. Validate fixture preconditions; do not label deployment differences as software defects.

Paper 7: AY2025/26 Semester 1 — Smart Home Control System

Source: Question paper, Q1–Q4 on PDF pp. 1–3; Appendix A on p. 4.

Case facts. Owners control lights, AC and door locks through a mobile app or LLM-powered voice assistant. The assistant interprets voice requests and can ask clarifying questions. Owners grant/revoke temporary guest access to specific doors. Guests may lock/unlock only during their permitted period; denied out-of-window attempts are logged. Administrators manage accounts, credentials and access rights but do not control devices. All guest-access activity is logged.

Q1 — Use cases (25 marks)

(a) Actors/use cases/relationships (18). (b) Guest door-access description (7).

(a) Actors: Homeowner, Guest, Administrator and external controlled devices (Lights, AC, Door Locks). App and voice assistant are interfaces/components of the SHCS, not additional human users. An external hosted speech/LLM service can be a supporting actor if that deployment boundary is chosen, but hosting is not specified in this appendix.

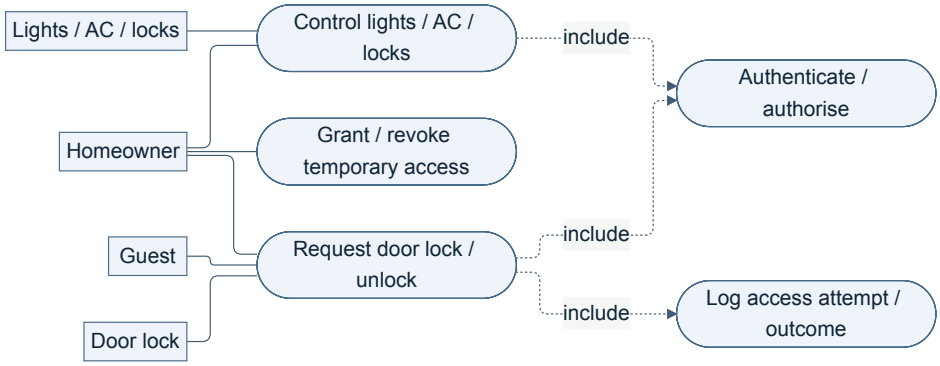


Figure 53: Q1 — Use cases (25 marks) — device control and access

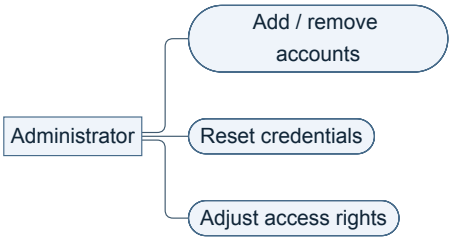


Figure 54: Q1 — Use cases (25 marks) — account administration

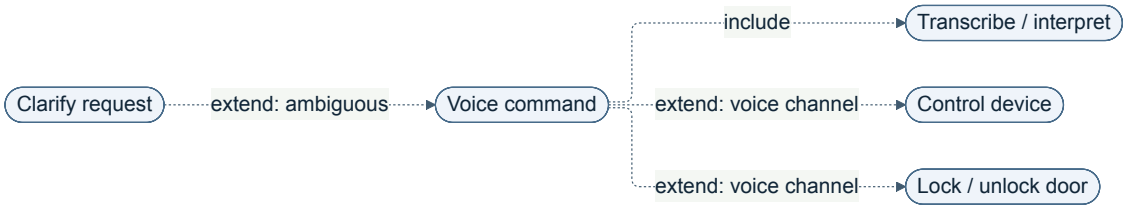


Figure 55: Q1 — Use cases (25 marks) — optional voice channel and clarification

Authorisation includes a guest's door-specific grant, revocation status and current time. Do not generalise Administrator from Homeowner or give administrators device-control associations. An owner-granted access period is not a one-time login token or permission for every door. Both permitted and denied guest attempts must reach the log.

(b) Use case: Guest Door Access. Primary actor: Guest. Supporting actor: Door Lock; voice interface if selected. Preconditions: guest identity can be authenticated; door and guest grant can be identified. Do **not** require "current time is valid" as the only precondition if the description must cover out-of-window denial. Flow: guest requests lock/unlock for a specific door through app or voice; system interprets/clarifies the request as needed; authenticate guest; check action, door, grant, revocation and time; execute permitted command through the lock interface; record request/time/outcome and show confirmation. Alternatives: no grant, revoked grant, wrong door or outside period, deny and log; ambiguous command, clarify without unlocking; device failure, log failure and report it rather than claiming success. A voice utterance alone is not proof of identity.

Q2 — Access states, voice activity and process (25 marks)

(a) Temporary guest access-controller state machine (4). (b) Guest voice-unlock activity/swimlanes (8). (c) Waterfall/Agile comparison: flexibility, customer involvement, delivery and one advantage each (5). (d) Two sprint advantages and two challenges, with examples (8).

(a) Model one guest–door grant. **Assumption:** a time window is half-open `[start,end)`; the paper requires a strict window but does not specify endpoint inclusion. Agree the exact policy before implementation/testing.

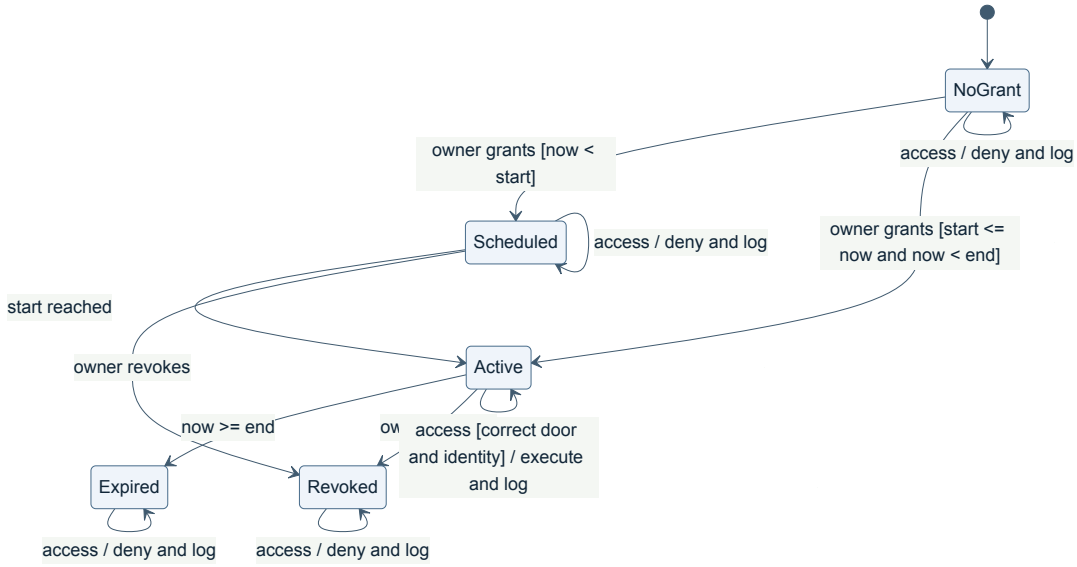


Figure 56: Q2 — Access states, voice activity and process (25 marks)

A new grant can create a fresh controller/grant instance. A command arriving late must recheck time before execution; being previously Active is not enough.

(b)

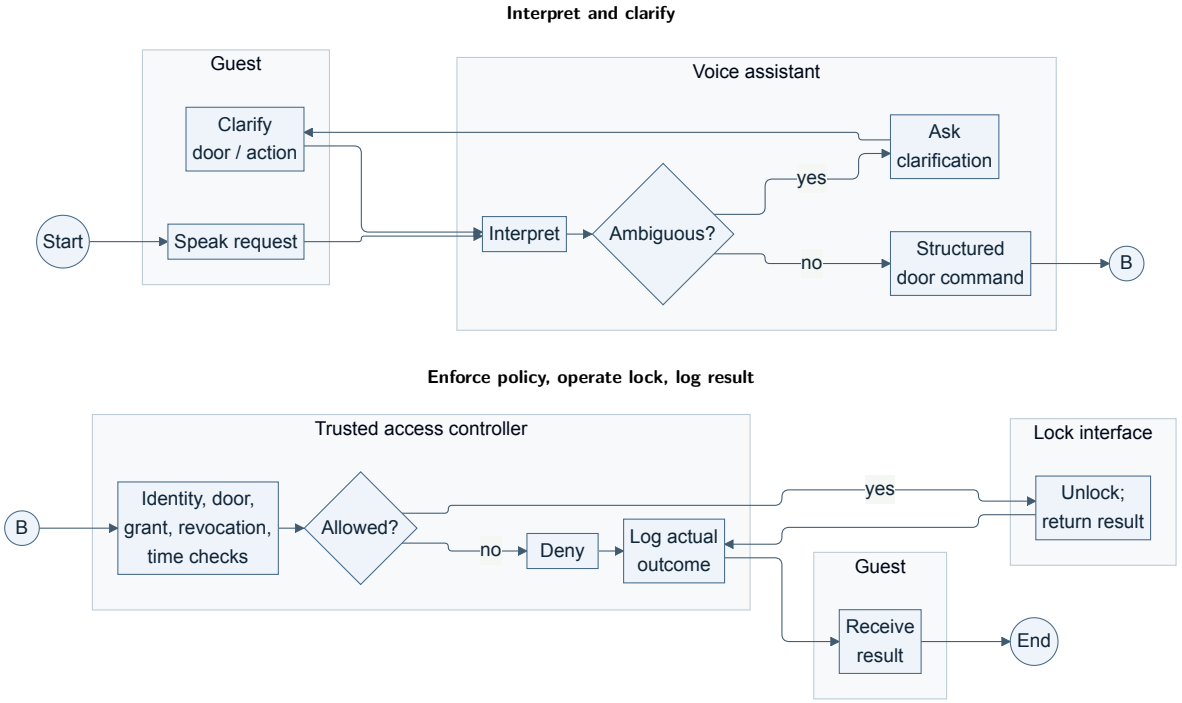


Figure 57: Q2 — Access states, voice activity and process (25 marks)

LLM output proposes an action; deterministic access rules decide whether it can occur. Failure to interpret/authenticate must not unlock the door.

(c) Compare flexibility, customer involvement, delivery and strengths using W, p. 15. Apply the comparison to owner/guest/device access increments.

(d) Apply S, p. 18. Two benefits: owner feedback on voice clarification; visible tested guest-access increments. Two challenges: oversized voice/device work (split/spike) and hardware/urgent-change dependencies (doubles, early integration and backlog negotiation).

Q3 — Smart-home architecture and pattern judgement (25 marks)

(a)(i) Architecture/detailed classes (8); (ii) one NFR and alternative architecture (8). (b)(i) Combined-pattern scenario/ classes (6); (ii) evaluate the cost of pattern overuse (3).

(a)(i) Use layered channel interfaces, application command control, domain access policy and device gateways. Both app and voice requests reach the same trusted policy before actuation.

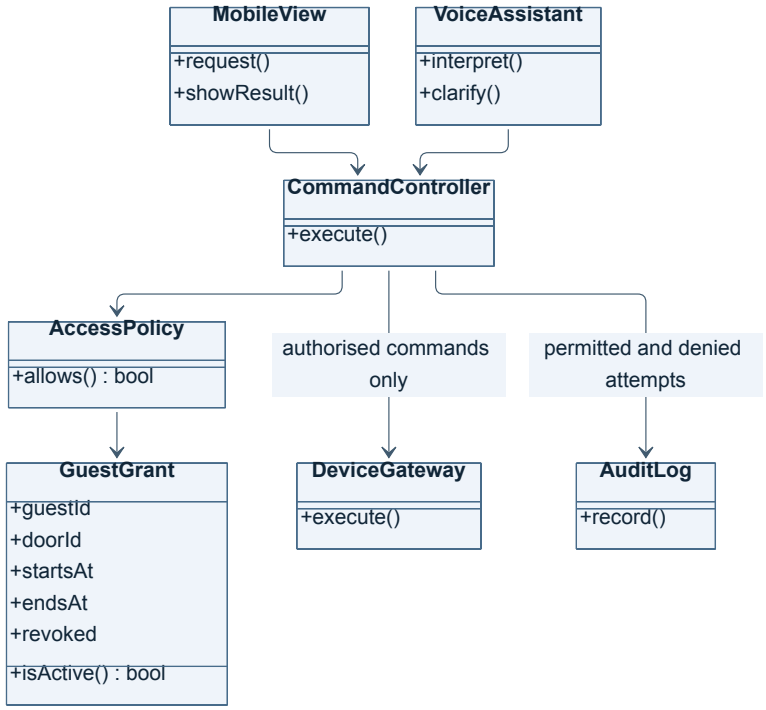


Figure 58: Q3 — Smart-home architecture and pattern judgement (25 marks)

(a)(ii) Choose **security**, explicitly required by authentication, role separation and time-limited door grants. An alternative **client–server architecture with a security Facade/reference gate** treats app and LLM processing as untrusted input channels. Only a trusted server-side access service can invoke device gateways. Account administration uses a separate authorised API and does not confer device-control rights.

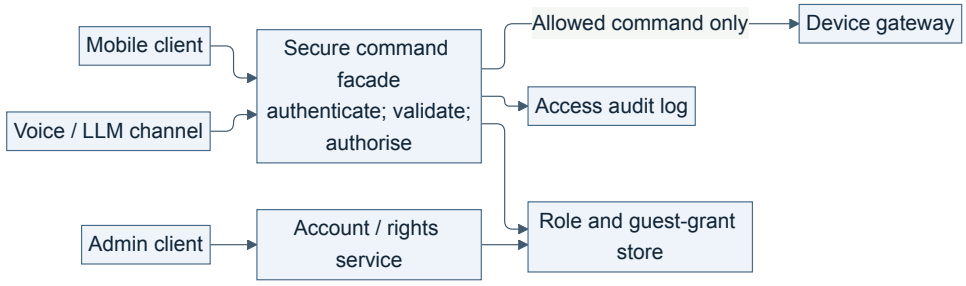


Figure 59: Q3 — Smart-home architecture and pattern judgement (25 marks)

Compared with policy spread among clients, a central gate prevents channel-specific bypasses and keeps one authoritative time/grant check. It introduces connectivity/availability concerns that require explicit failure behaviour; door access must not silently bypass checks when the policy service fails. This is a deployment/trust-boundary alternative compatible with layered internals.

(b)(i) Combine Strategy for role-specific permission policies with Simple Factory for selecting the policy from the **authenticated role**, never an LLM-supplied role string.

Class diagram: use P, p. 20. Context = `AccessService`; Strategy = `AccessPolicy`; Factory = `PolicyFactory`. Concrete strategies: `OwnerPolicy`, `GuestPolicy`, `AdministratorPolicy`. Interface/concrete operations: `allows()`. Select from the authenticated role, never an LLM role string. Guest checks door/time/revocation; Administrator denies device control.

(b)(ii) Agree: use C, p. 21. Trivial access rules may need only a policy table; introduce patterns for actual policy variation/coupling.

Q4 — User inputs and automation (25 marks)

(a)(i) Derive ECs for Name, Phone Number, Access Expiry Date and Type of the User (4). (ii) Give on/below/above boundary values (4). (iii) Design defensive tests (5).

Input interpretation. Name must be ASCII; Phone Number is a 8-digit integer; Access Expiry Date is between 2025 and 2030; Type of the User is one of `Administrator`, `Owner`, `Guest`, `One Time User`. Assume a positive 8-digit numeric value (not a string whose leading zeroes count as digits), inclusive complete calendar years, and one-day date resolution. These interpretations are stated because the brief does not specify parsing details. Do not invent an ASCII-field length limit.

(i) Equivalence classes:

Field	Valid EC	Invalid ECs
Name	ASCII text	Text containing non-ASCII characters
Phone Number	Integer 10000000. . . 99999999	Integer below 10000000; integer above 99999999; malformed/non-integer input at the input boundary
Access Expiry Date	Real calendar date 2025-01-01. . . 2030-12-31	Real date before 2025-01-01; real date after 2030-12-31; malformed/nonexistent date
Type of the User	One EC containing {Administrator, Owner, Guest, One Time User}	Any unlisted value

The category set is **one valid EC**, although testing all its few members is useful. No empty-string prohibition or printable-only ASCII restriction is supplied; clarify those separately instead of asserting an invented oracle.

(ii) Boundary triplets:

Field / boundary	Just below	On boundary	Just above
Phone Number: minimum	9999999 (reject)	10000000 (accept)	10000001 (accept)
Phone Number: maximum	99999998 (accept)	99999999 (accept)	100000000 (reject)
Access Expiry Date: earliest	2024-12-31 (reject)	2025-01-01 (accept)	2025-01-02 (accept)
Access Expiry Date: latest	2030-12-30 (accept)	2030-12-31 (accept)	2031-01-01 (reject)

ASCII membership and unordered categories have no specified application-level ordered boundary. Malformed values are additional EC tests, not numeric/date neighbours.

(iii) Defensive suite. Use the common testing conventions below; valid rows combine boundaries, invalid rows isolate one field. The full boundary suite is retained.

Name	Phone Number	Access Expiry Date	Type of the User	Expected / purpose
`Alex`	`10000000`	`2025-01-01`	`Administrator`	Accept — valid boundaries / category coverage
`Alex`	`10000001`	`2025-01-02`	`Owner`	Accept — valid boundaries / category coverage
`Alex`	`99999998`	`2030-12-30`	`Guest`	Accept — valid boundaries / category coverage
`Alex`	`99999999`	`2030-12-31`	`One Time User`	Accept — valid boundaries / category coverage
`Alex`	`54999999`	`2027-07-01`	`Administrator`	Accept — interior baseline / category coverage
`café`	`54999999`	`2027-07-01`	`Administrator`	Reject — non-ASCII Name
`Alex`	`9999999`	`2027-07-01`	`Administrator`	Reject — Phone Number below minimum
`Alex`	`100000000`	`2027-07-01`	`Administrator`	Reject — Phone Number above maximum

Continued on next page

Name	Phone Number	Access Expiry Date	Type of the User	Expected / purpose
Alex	54999999	2024-12-31	Administrator	Reject — Access Expiry Date below minimum
Alex	54999999	2031-01-01	Administrator	Reject — Access Expiry Date above maximum
Alex	54999999	2027-07-01	Unknown	Reject — unlisted Type of the User
Alex	12x	2027-07-01	Administrator	Reject — malformed integer; extra defensive case
Alex	54999999	2025-02-30	Administrator	Reject — nonexistent calendar date; extra defensive case

The Q4 category One Time User is valid for this input question, although Appendix A does not define that role's device permissions. Do not invent its access policy from the category name.

(b) How can LLMs assist automated testing? (4). (c) At least four challenges of fully automated development (8).

(b) Use requirements and code context to propose tests, fixtures, boundary cases and automation scripts; run them through a real test runner and report actual results. Generate guest-access cases for wrong door, revocation and exact time edges, and validate expected results against the agreed policy. Review generated code/oracles and measure meaningful coverage. An LLM's claim that a test passed is not execution evidence; voice-language tests also need representative ambiguous/adversarial requests and repeatability controls.

(c) Four challenges with consequences: **ambiguous/incomplete requirements** require stakeholder clarification, otherwise correct-looking code solves the wrong problem; **incorrect or nondeterministic generation** can introduce defects/security flaws, requiring independent execution and review; **integration/context limits** make existing interfaces, hardware behaviour and cross-component invariants hard to preserve; **validation and accountability** require trusted oracles, quality-attribute evidence and responsibility for consequential decisions. Maintenance adds further difficulty because changes must preserve compatibility, intent and operational safety across versions. In this case, a plausible voice response cannot establish that a physical door was correctly authorised or operated.

Paper 8: AY2025/26 Semester 2 — Restaurant Recommendation System

Source: Question paper, Q1–Q4 on PDF pp. 1–3; Appendix A on p. 4.

Case facts. Anonymous users converse with an LLM about food preferences. Conversation ends after ten user/assistant message pairs or an earlier Finished action. The same LLM then returns JSON containing up to five tags from a fixed set of twenty. Invalid JSON, out-of-set tags or more than five tags produces an error and a restart invitation. With valid tags, user provides postcode and integer travel radius; recommendations stay within radius and sort by decreasing count of matching tags. Clicking a restaurant website records preferences and selection. Administrators authenticate to maintain restaurant entries and view selection logs.

Q1 — Use cases and JSON-check activity (25 marks)

(a) Actors/use cases/relationships (18). (b) Activity/swimlane diagram for checking JSON output (7).

(a) Actors: Anonymous User, Administrator, and external LLM Service if externally hosted (explicitly the deployment considered in Q3(c)). The database and validation function are internal components. A restaurant website need not be a supporting actor merely because the browser follows its link.

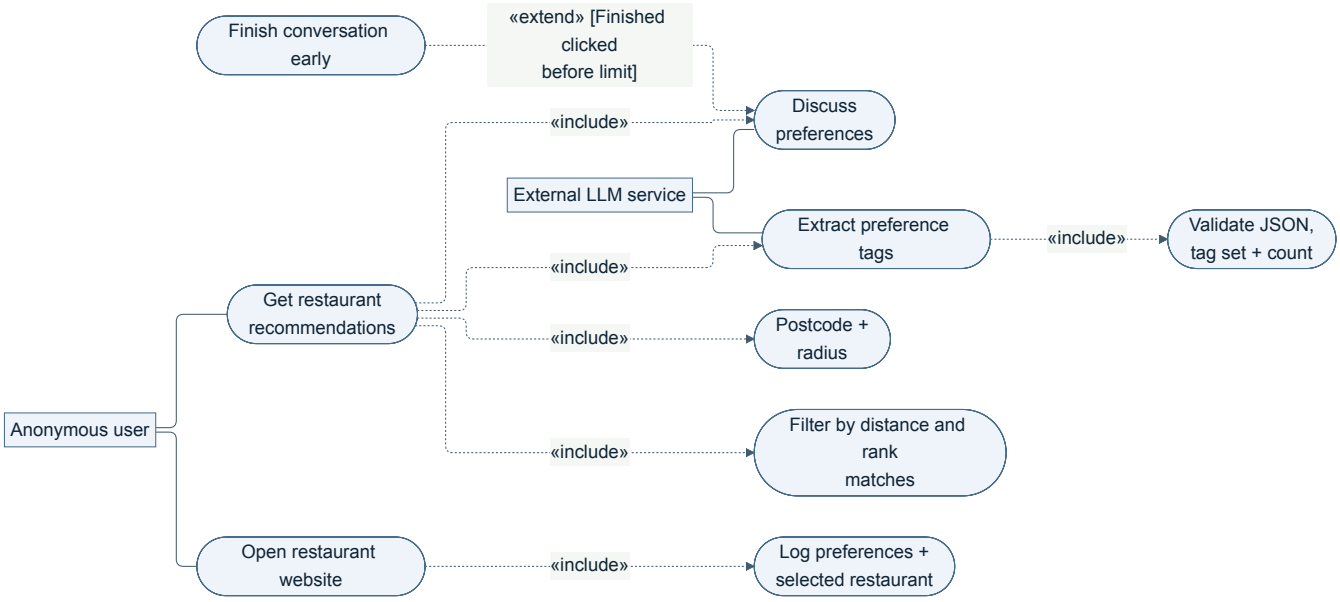


Figure 60: Q1 — Use cases and JSON-check activity (25 marks) — anonymous user and supporting LLM view

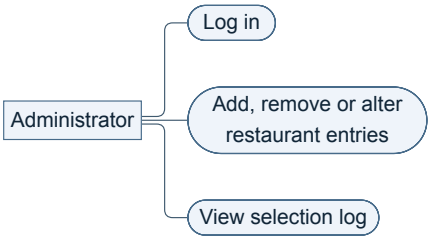


Figure 61: Q1 — Use cases and JSON-check activity (25 marks) — administrator view

Protected administrator actions require authentication; anonymous recommendations do not. Early finish is optional; reaching ten completed pairs also ends the conversation. Location/ranking steps belong to the successful flow; validation failure terminates that attempt before them. Recording a selection happens on a website click, not automatically for every displayed restaurant.

(b) Schema assumption: the LLM contract represents tags as a JSON array of strings (or an agreed object field holding that array). JSON validity alone does not ensure that the parsed shape is usable as a tag list.

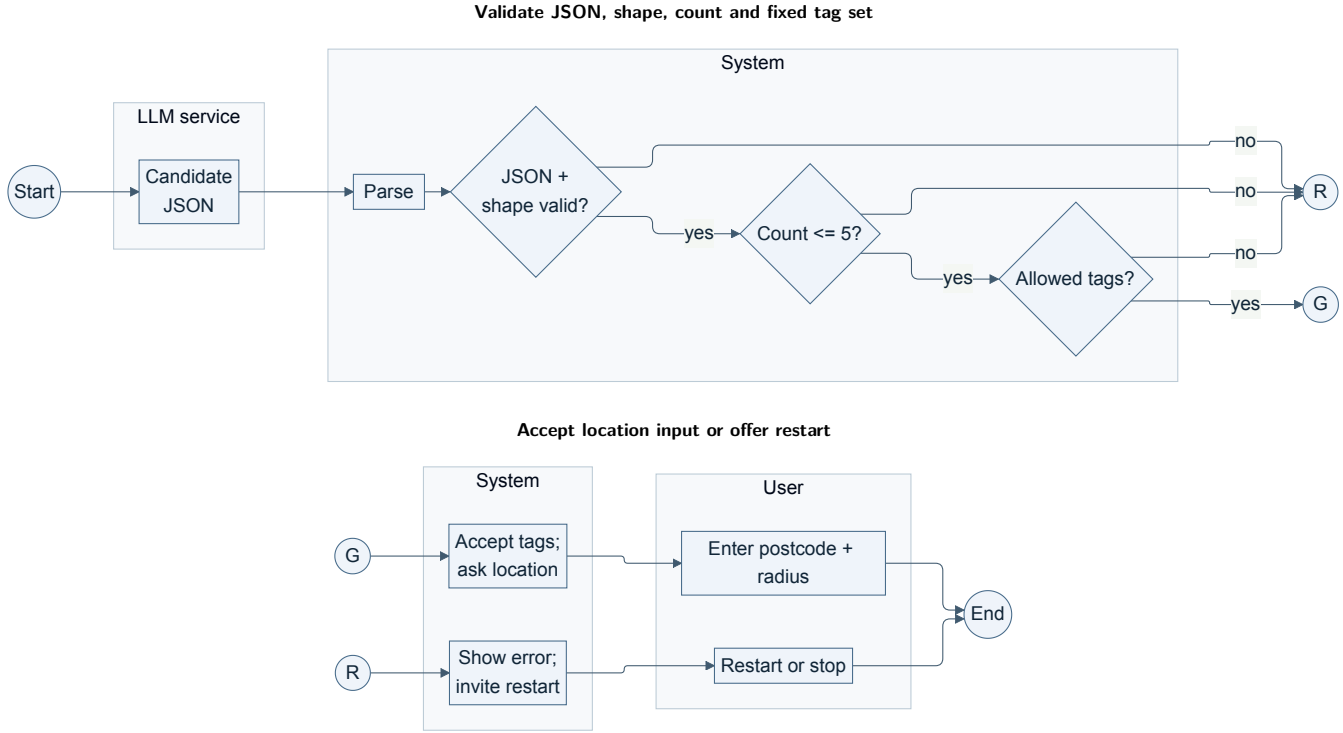


Figure 62: Q1 — Use cases and JSON-check activity (25 marks) — G = accepted tags; R = rejected candidate.

"Up to five" supplies no minimum-one rule: do not reject an empty tag list unless a separate requirement establishes that rule. Duplicate-tag handling is unspecified; do not silently add a rejection requirement. Count entries before any optional deduplication so a six-entry response cannot evade the stated limit.

Q2 — Recommendation classes, conversation states and Scrum (25 marks)

(a) Recommendation-generation conceptual classes/ stereotypes (8). (b) Conversation state machine (6). (c) Waterfall versus Agile when the tag set changes (5). (d) Two causes/remedies for frequently overrun sprints (6).

(a)

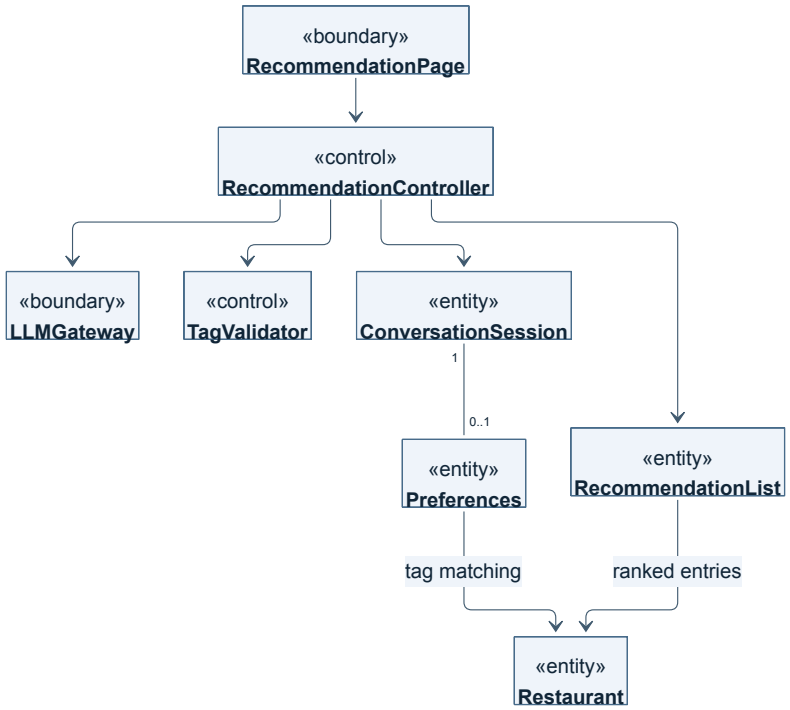


Figure 63: Q2 — Recommendation classes, conversation states and Scrum (25 marks)

Postcode/radius belong to the recommendation request; preferences contain validated tags. No registered-user entity is required for anonymous use.

(b) Count **completed user/assistant pairs**, not individual messages. The limit is ten of each, not ten combined.

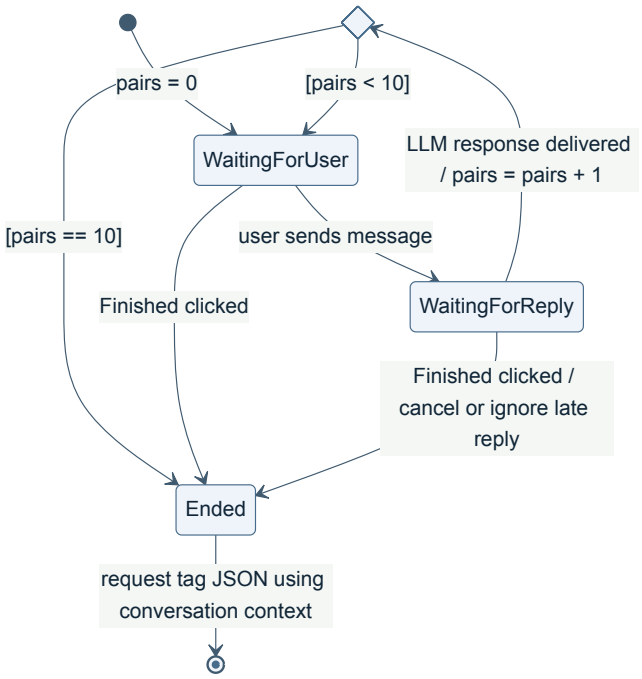


Figure 64: Q2 — Recommendation classes, conversation states and Scrum (25 marks)

Handling Finished while a reply is pending is a stated design choice; it must not reopen the ended conversation when a late reply arrives. Restart after invalid tag JSON starts a fresh session. JSON validation/recommendation are subsequent components, not extra conversation messages.

(c) Apply W, p. 15. A tag-set change affects prompts, validators, restaurant data, UI and tests. Version/configure the vocabulary consistently; changing only the prompt is insufficient.

(d) Use S, p. 18. Oversized/optimistic stories: split and select less work using actual capacity. Hidden LLM/API/integration or support work: investigate early, use doubles and budget support capacity. Replan unfinished work.

Q3 — Architecture and external LLM trade-offs (25 marks)

(a) Architecture (3). (b) Detailed classes with attributes/ methods (12). (c)(i) Two upsides/two downsides of self-hosting an LLM (4); (ii) two other responses to a hosted LLM advertising competitors (6).

(a) Use client–server with layered application/domain services and an external LLM gateway. MVC can organise browser interaction: views display conversation/results, controllers accept inputs, and domain models/services hold conversation, tag-validation and recommendation rules. Keep the restaurant database, logging and administrator authorisation on the server; never trust unvalidated

LLM output.

(b) Two views of one class model keep the diagram readable. `SessionController` is the same class in both views; operations are shown in the view that uses them.

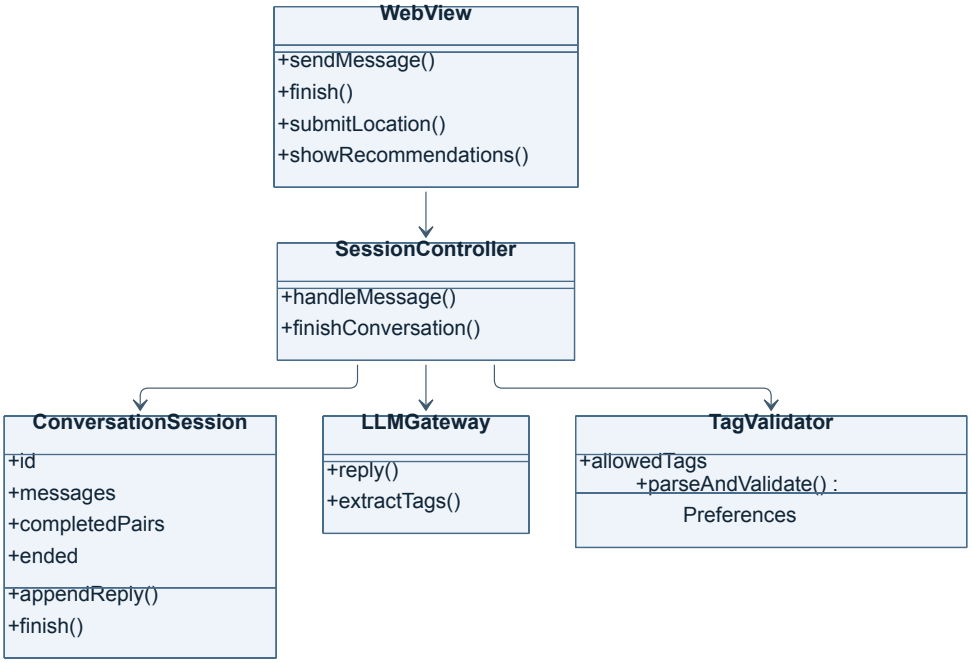


Figure 65: Q3 — Architecture and external LLM trade-offs (25 marks)

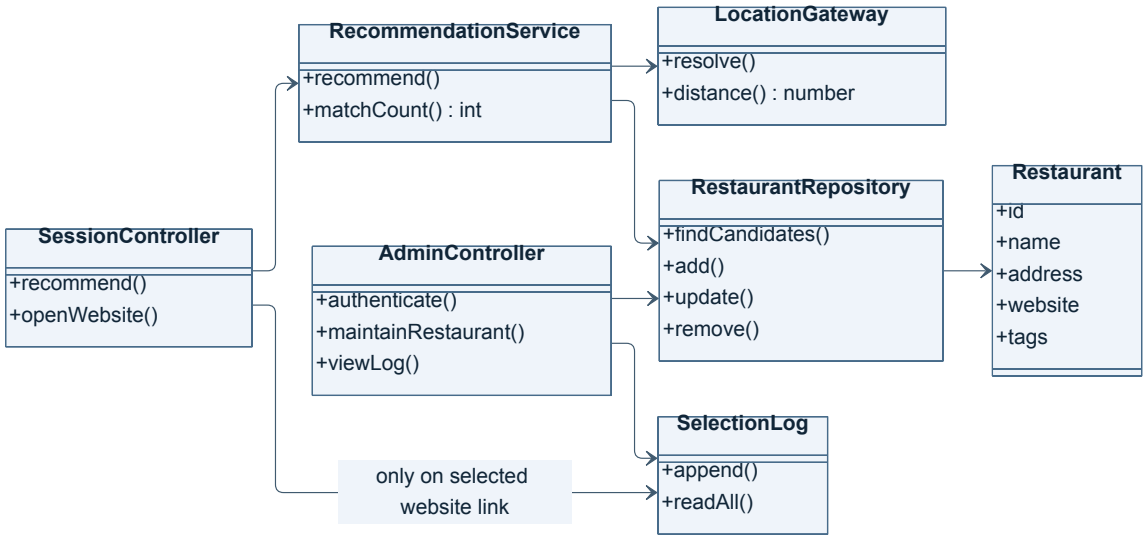


Figure 66: Q3 — Architecture and external LLM trade-offs (25 marks)

A server-side selection endpoint records the preferences/restaurant associated with that anonymous session, then redirects to the restaurant website. Domain algorithm: resolve location; retain restaurants with `distance` $\leq$ `radius`; compute the number of matching preference tags; sort descending; display names and website links. Zero matches are not explicitly excluded. Tie ordering and distance calculation method are unspecified; declare any chosen policy. Location resolution is a proposed integration needed for distance calculation, distinct from Q4's initially format-only validator.

(c)(i) Self-hosting upsides: **control over model behaviour/version/customisation**, and **control over data handling with less dependence on a hosted provider's policies/availability**. Downsides: **compute/infrastructure cost**, and **operational/expertise burden** for serving, monitoring, updates, capacity and security. Self-hosting does not automatically deliver better accuracy or eliminate harmful outputs.

(c)(ii) Two approaches other than self-hosting:

1. **Switch to another externally hosted model/provider**, through the gateway, after testing against conversation and tag-extraction requirements. Establish service/content expectations and regression checks; avoid merely assuming every alternative behaves better.
2. **Add a server-side output-policy boundary.** Check user-visible replies for unwanted promotional content, block affected responses and request a constrained regeneration with bounded retries; provide a controlled error/fallback when checks fail. Keep tag extraction under strict structural/membership validation as well. A prompt instruction alone is not a reliable enforcement boundary, and valid JSON does not establish that freeform replies contain no advertising. Evaluate false positives/negatives and preserve the required conversation capability.

Q4 — Postcode/distance ECs, external API and internationalisation (25 marks)

Given: `validateUserInput(postcode, distance)` accepts a postcode string of six numerical digits and an integer distance from 0 to 100 km inclusive. **(a) ECs/assumptions (4). (b) Boundary**

triplets (4). (c) Defensive suite (8). (d)(i) EC changes when a public API confirms a real Singapore building (3); (ii) two complications for comprehensive tests (4). (e) Adaptation to multiple countries (2).

(a) Assume "numerical digits" means ASCII 0–9, retain leading zeroes, do not trim/coerce invalid inputs silently, and initially check **syntax only**, not real-building existence. The function contract already gives string/integer argument types; non-string/non-integer inputs are extra defensive-interface cases if the implementation permits them.

Input	Valid EC	Invalid ECs
Postcode	Exactly six ASCII digits: `[0-9]{6}`	Length other than six; length six with at least one non-digit (including non-ASCII digits)
Distance	Integer $0 \leq d \leq 100$	Integer `d < 0`; integer `d > 100`

The postcode invalid classes are disjoint as defined. 000000 and 999999 are syntactically valid even if they do not identify buildings. Postcode is not an integer range from 100000 to 999999.

(b) Distance triplets: -1, 0, 1 and 99, 100, 101. Postcode **length** triplet: length 5, 6, 7, e.g. 00000, 000000, 0000000. Character membership is a discrete format rule; use a six-character string containing a letter/space rather than inventing an overall numeric postcode boundary.

(c) Reset/isolate each test. "Accept" means this validator returns acceptance; it does not promise that later geocoding/recommendation will succeed.

Postcode string	Distance (km)	Expected / purpose
612345	50	Accept — baseline
000000	50	Accept — leading zeroes, syntax only
999999	50	Accept — six digits, syntax only
612345	0	Accept — lower distance boundary
612345	1	Accept — lower valid neighbour
612345	99	Accept — upper valid neighbour
612345	100	Accept — upper distance boundary
612345	-1	Reject — distance below minimum
612345	101	Reject — distance above maximum
00000	50	Reject — five characters
0000000	50	Reject — seven characters
12345a	50	Reject — six characters with non-digit
123 45	50	Reject — embedded space
[six full-width digits]	50	Reject — non-ASCII digits under stated assumption
empty string	50	Reject — wrong length

All rejected rows contain exactly one invalid parameter. If runtime types are not enforced, add type-check tests such as fractional distance and a numeric rather than string postcode, with explicit rejection policy.

(d)(i) Split the old valid-syntax postcode class into **API-confirmed real-building postcodes** and **well-formed postcodes confirmed not to identify a building**. Keep malformed classes. Timeout, unavailable service, malformed API response or rate limiting is a dependency-failure outcome, not proof that the postcode is invalid. Define fail/retry/error behaviour explicitly.

(d)(ii) Two complications: **changing external data**, so a postcode's building association/validity may change and make hard-coded oracles stale; **nondeterministic service behaviour**, including network failures, latency, limits and unexpected response formats. Use versioned fixtures/test doubles for deterministic unit/contract cases and a smaller controlled live integration suite with recorded expectations. Exercise confirmed-valid, confirmed-absent and each failure response; do not claim that a mocked suite verifies live API availability.

(e) Add a country identifier or country-specific validation policy: `validateUserInput(countryCode, postcode, distance)`. Select the appropriate format and, where required, country-specific lookup service. Preserve postcodes as strings because countries may use letters, spaces and leading zeroes. Define normalisation and accepted formats per country and parameterise tests; do not impose Singapore's six-digit rule globally.